

# Competitive Programming Reference

Combined & reorganized master document

|                                                                              |              |
|------------------------------------------------------------------------------|--------------|
| <b>1. CP Cheat Sheet (C++17 quick reference)</b>                             | <b>p. 2</b>  |
| <b>2. Theoretical CS Cheat Sheet (math / complexity reference)</b>           | <b>p. 12</b> |
| <b>3. Contest Snippets</b>                                                   | <b>p. 22</b> |
| Index of 71 snippets                                                         | p. 22        |
| Snippet contents (#1 dijkstra → #71 Suffix_Automaton)                        | p. 23        |
| Appendix: Original snippet-site index pages (superseded, kept for reference) | p. 103       |

# CP Cheat Sheet

C++17 · Competitive Programming Reference — reformatted as real monospace code

## Main Template

prefix: tcp

```
#include <bits/stdc++.h>
using namespace std;

template<class T> istream& operator>>(istream& in, vector<T>& v){
    for(auto& x:v) in>>x; return in;
}
template<class T> ostream& operator<<(ostream& out, vector<T>& v){
    for(auto& x:v) out<<x<<' '; return out;
}

#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define endl '\n'
#define int long long
#define str string
#define vi vector<int>
#define vill vector<ll>
#define read(v) for (auto &x : v) cin >> x;
#define cin(v) for (auto &x : v) cin >> x;
#define cout(v) for (auto &x : v) cout << x << ' ';

using ll = long long;
using ull = unsigned long long;
using u128 = unsigned __int128;
using i128 = __int128;

const int MOD = 1e9 + 7;
const long long INF = 1e18;

static const int IO_SPEEDUP = [](){
    ios::sync_with_stdio(false);
    cout.tie(nullptr); cin.tie(nullptr);
    return 0;
}();

int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

void solve() { }

const int TESTCASES = 1;
// 1 = read T, 2 = local only, else = single run

signed main() {
    ull TTT = 1;
    if (TESTCASES == 1) cin >> TTT;
    else if (TESTCASES == 2) { #ifndef LOCAL cin >> TTT; #endif }
    while (TTT--) solve();
}
```

## Quick Snippets

VSCode

### Chmin / Chmax

prefix: ch

```
template<class T> bool chmin(T& a, const T& b){
    if(b<a){a=b;return 1;} return 0;
}
template<class T> bool chmax(T& a, const T& b){
    if(a<b){a=b;return 1;} return 0;
}
```

### Modpow

prefix: powm

```
ll modpow(ll a, ll e, ll mod){
    ll r=1;
    while(e){
        if(e&1) r=r*a%mod;
        a=a*a%mod; e>>=1;
    }
    return r;
}
```

### Binpow (no mod)

prefix: binpow

```
ll binpow(ll a, ll b){
    ll res=1;
    while(b>0){
        if(b&1) res=res*a;
        a=a*a; b>>=1;
    }
    return res;
}
```

### Lcm

prefix: lcm

```
ll lcm(ll a, ll b){ return a/gcd(a,b)*b; }
```

### Mex

prefix: mex

```
int getMEX(const vi& a, int L, int R){
    int len=R-L+1; vector<char> s(len+2);
    for(int i=L;i<=R;i++)
        if(a[i]>=0 && a[i]<=len) s[a[i]]=1;
    int m=0; while(s[m]) m++;
    return m;
}
int getMEX(const vi& a){ return getMEX(a,0,sz(a)-1); }
```

### Vector input

prefix: vec

```
vector<int> arr(n);
for (auto &x : arr) cin >> x;
```

## Bitwise Operations

*bit tricks*

### Basic checks

```
bool is_even(ll n) { return !(n & 1); }
bool is_odd(ll n) { return (n & 1); }
bool is_pow2(ll n) { return n>0 && !(n&(n-1)); }
```

### K-th bit (0-indexed)

```
bool get_bit(ll n, int k) { return (n >> k) & 1; } // read
ll set_bit(ll n, int k) { return n | (1LL << k); } // turn ON
ll clr_bit(ll n, int k) { return n & ~(1LL << k); } // turn OFF
ll tog_bit(ll n, int k) { return n ^ (1LL << k); } // flip
```

### GCC builtins

```
__builtin_popcountll(n) // count set bits
__builtin_clzll(n) // leading zeros
__builtin_ctzll(n) // trailing zeros
```

### LSB tricks

```
n & (n-1) // remove lowest set bit 12(1100)->8(1000)
n & (-n) // extract lowest set bit 12(1100)->4(0100)
```

### Advanced patterns

```
// XOR to find unique element
int xr=0; for(int x:a) xr^=x;

// Enumerate all 2^N subsets
for(int mask=0;mask<(1<<n);mask++)
    for(int i=0;i<n;i++)
        if((mask>>i)&1) /* a[i] in subset */;
```

## Number Theory

*primes · factors · divisors*

### Sieve of Eratosthenes

$O(N \log \log N)$

```
vector<bool> sieve(int n){
    vector<bool> p(n+1,true); p[0]=p[1]=0;
    for(int i=2;i*i<=n;i++)
        if(p[i]) for(int j=i*i;j<=n;j+=i) p[j]=0;
    return p;
}
// O(1) query after
```

### SPF sieve (fast factorization)

$O(N \log \log N)$  pre ·  $O(\log N)$  query

```
const int N = 1500000;
vector<bool> isPrime(N+1,1);
vector<int> spf(N+1); // smallest prime factor
isPrime[0]=isPrime[1]=0;
for(int i=2;i<=N;i++){
    if(isPrime[i]){
        spf[i]=i;
        for(int j=i*i;j<=N;j+=i){
            isPrime[j]=0;
            if(!spf[j]) spf[j]=i;
        }
    }
}
vi factorize(int n){
    vi ret;
    while(n!=1){ ret.push_back(spf[n]); n/=spf[n]; }
    return ret;
}
```

### Prime factorization

$O(\sqrt{N})$

```
vi primeFactors(ll n){
    vi p;
    for(ll i=2;i*i<=n;i++)
        while(n%i==0){ p.push_back(i); n/=i; }
    if(n>1) p.push_back(n);
    return p;
}
```

### Divisors

$O(\sqrt{N})$

```
vi divisors(ll n){
    vi d;
    for(int i=1;i*i<=n;i++)
        if(n%i==0){ d.push_back(i); d.push_back(n/i); }
    return d;
}
// unsorted; sort if needed
```

### All divisors precompute

$O(N \log N)$

```
vector<int> divs[N+1];
for(int i=1;i<=N;i++)
    for(int j=i;j<=N;j+=i)
        divs[j].push_back(i);
```

## Modular Arithmetic

*overflow-safe*

| Rule       | Formula                      |
|------------|------------------------------|
| Add        | $(a \% n + b \% n) \% n$     |
| Sub        | $(a \% n - b \% n + n) \% n$ |
| Mul        | $(a \% n * b \% n) \% n$     |
| Neg mod    | $(a \% n + n) \% n$          |
| Idempotent | $(a \% n) \% n == a \% n$    |
| Divisible  | $a \% n == 0 \iff n   a$     |

### Safe accumulate

```
ans = (ans*y + x*y) % y
```

### Modular inverse (Fermat)

N prime

```
// inv(a) = a^(MOD-2) mod MOD
ll inv(ll a){ return modpow(a, MOD-2, MOD); }
```

## DSU / Union-Find

*prefix: dsu ·  $O(\alpha)$*

```
struct DSU{
    vector<int> p, sz;
    DSU(int n):p(n),sz(n,1){ iota(p.begin(),p.end(),0); }

    int find(int x){
        return p[x]==x ? x : p[x]=find(p[x]);    // path compression
    }
    bool unite(int a, int b){
        a=find(a); b=find(b);
        if(a==b) return 0;
        if(sz[a]<sz[b]) swap(a,b);                // union by size
        p[b]=a; sz[a]+=sz[b];
        return 1;
    }
};
// DSU dsu(n+1); dsu.unite(u,v); dsu.find(x);
```

## Segment Tree (Range Sum)

*prefix: segt ·  $O(\log n)$*

```

struct SegTree{
    int n; vector<ll> t;
    SegTree(int n):n(n),t(2*n){}

    void update(int i, ll v){                // point update
        for(t[i+=n]=v; i>1; i>>=1)
            t[i>>1]=t[i]+t[i^1];
    }
    ll query(int l, int r){                  // [l, r] inclusive
        ll s=0;
        for(l+=n,r+=n; l<=r; l>>=1,r>>=1){
            if(l&1) s+=t[l++];
            if(!(r&1)) s+=t[r--];
        }
        return s;
    }
};

```

## Graph — DFS / BFS

$O(V+E)$

### Setup

```

vector<vector<int>> adj(n+1);
vector<bool> vis(n+1, false);
// adj[u].push_back(v); adj[v].push_back(u);

```

### DFS (recursive)

```

void dfs(int u){
    vis[u]=1;
    for(auto v:adj[u])
        if(!vis[v]) dfs(v);
}

```

### BFS (distance)

prefix: bfs

```

void bfs(int s, vector<int>& d){
    queue<int> q;
    d.assign(sz(adj), -1);
    d[s]=0; q.push(s);
    while(!q.empty()){
        int u=q.front(); q.pop();
        for(int v:adj[u])
            if(d[v]==-1){ d[v]=d[u]+1; q.push(v); }
    }
}

```

### Connected components

```

int comp=0;
for(int u=1;u<=n;u++){
    if(!vis[u]){ comp++; dfs(u); }
}

```

### Tree check (undirected)

```

// tree <-> connected AND edges == n-1
(comp==1 && n-1==m) // single tree
(n-comp==m)         // forest

```

## Dijkstra & 0-1 BFS

*shortest path*

### Dijkstra

$O((V+E) \log V)$

```
// adj[u] = { {v, w}, ... }
vector<ll> dist(n+1, INF); dist[src]=0;
priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
pq.push({0, src});
while(!pq.empty()){
    auto [d,u]=pq.top(); pq.pop();
    if(d>dist[u]) continue;          // stale
    for(auto[v,w]:adj[u])
        if(dist[u]+w<dist[v])
            dist[v]=dist[u]+w, pq.push({dist[v],v});
}
```

## 0-1 BFS

weights 0 or 1

```
vector<int> dist(n+1, INF); dist[src]=0;
deque<int> dq; dq.push_back(src);
while(!dq.empty()){
    int u=dq.front(); dq.pop_front();
    for(auto[v,w]:adj[u])
        if(dist[u]+w<dist[v]){
            dist[v]=dist[u]+w;
            if(w==0) dq.push_front(v);    // high prio
            else     dq.push_back(v);     // low prio
        }
}
```

## Cycle detection (directed, DFS)

```
vi vis(0); bool cyclic=0;
void dfs(int u){
    vis[u]=2;                                // in stack
    for(auto v:adj[u]){
        if(!vis[v]) dfs(v);
        else if(vis[v]==2) cyclic=1;
    }
    vis[u]=1;                                // done
}
```

## Graph Representations

3 ways

### Adjacency list

preferred in CP

```
vector<vector<int>> adj(n+1);
adj[u].push_back(v); adj[v].push_back(u);
```

### Adjacency matrix

$O(N^2)$  space

```
vector<vector<int>> mat(n+1, vector<int>(n+1,0));
mat[u][v]=mat[v][u]=1;    // weighted: = w
```

### Edge list

MST / sorting

```
set<pair<int,int>> edges;
edges.insert({u,v}); edges.insert({v,u});
// or: vector<tuple<int,int,int>> {w,u,v}
```

### Weighted adjacency

```
vector<vector<pair<int,int>>> adj(n+1);
adj[u].push_back({v, w});
// access: for(auto [v,w] : adj[u])
```

## Prefix / Suffix Arrays

$O(n)$  build ·  $O(1)$  query

### 1D prefix sum

```
vector<int> pref(n+1, 0);
for(int i=1;i<=n;i++)
    pref[i]=pref[i-1]+a[i-1];
// sum [l..r] (1-indexed): pref[r] - pref[l-1]
```

### Prefix max / min

```
pMax[i] = max(pMax[i-1], a[i]);
pMin[i] = min(pMin[i-1], a[i]);
```

### Suffix sum

```
suf[n-1] = a[n-1];
for(int i=n-2;i>=0;i--)
    suf[i]=a[i]+suf[i+1];
```

### 2D prefix sum

```
// Build: row prefix then col prefix
for(int i=1;i<=n;i++)
    for(int j=1;j<=m;j++)
        arr[i][j]+=arr[i][j-1];
for(int j=1;j<=m;j++)
    for(int i=1;i<=n;i++)
        arr[i][j]+=arr[i-1][j];

// Query [r1,c1] -> [r2,c2]:
arr[r2][c2]-arr[r1-1][c2]-arr[r2][c1-1]+arr[r1-1][c1-1]
```

### Frequency array

```
// Numbers:    freq[a[i]]++
// Lowercase:  freq[s[i]-'a']++
// Uppercase:   freq[s[i]-'A']++
// Digits:      freq[s[i]-'0']++

accumulate: accumulate(all(v), 0LL)
partial_sum: partial_sum(all(v), res.begin())
```

## Data Structures — STL

Set · Map · PQ · Stack

### Set / multiset

```
set<int> s;           // sorted, no dups
multiset<int> ms;     // sorted, dups OK
s.insert(x);          // O(log n)
s.erase(x);           // all occurrences
ms.erase(ms.find(x)); // one occurrence
s.lower_bound(x);      // first E >= x
s.upper_bound(x);      // first E > x
ms.count(x);           // # duplicates
// descending: set<int, greater<int>>
```

### Map

```
map<int,int> m;
m[key]=val; m.erase(key); m.count(key);
for(auto [k,v]:m) cout<<k<<' '<<v;
// unordered_map for O(1) avg
```

### Priority queue

```
priority_queue<int> maxpq;           // max-heap (default)
priority_queue<int,vector<int>,greater<int>> minpq; // min-heap
pq.push(x); pq.top(); pq.pop(); // all O(log n)
// pairs: pq.push({dist, node});
```

### Stack · Queue · Deque

```
stack<int> st; st.push(x); st.top(); st.pop();
queue<int> q; q.push(x); q.front(); q.pop();
deque<int> dq; dq.push_front(x); dq.push_back(x);
// dq.pop_front(); dq.pop_back();
```

# Sorting

STL + algorithms

| Algorithm | Time              | Space       | Stable |
|-----------|-------------------|-------------|--------|
| Bubble    | $O(n^2)$          | $O(1)$      | yes    |
| Merge     | $O(n \log n)$     | $O(n)$      | yes    |
| Heap      | $O(n \log n)$     | $O(1)$      | no     |
| Quick     | avg $O(n \log n)$ | $O(\log n)$ | no     |
| std::sort | $O(n \log n)$     | $O(\log n)$ | no     |

## STL sort

```
sort(v.begin(), v.end());           // ascending
sort(v.rbegin(), v.rend());         // descending
sort(v.begin(), v.end(), greater<int>());
```

## Custom sort (pairs by second, tiebreak by first)

```
sort(all(v), [](const auto& a, const auto& b){
    if(a.second!=b.second) return a.second<b.second;
    return a.first<b.first;
});
```

# PBDS — Policy-Based DS

prefix: os / pbds\_all · MacBook: use g++-15, not g++

## Include

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

## Ordered set (unique ints)

```
using os_easy = tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>;
```

## Generic template

```
template<typename T>
using os = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
```

## Ordered multiset (dups via pair)

```
template<typename T>
using os_multi = tree<pair<T,int>, null_type,
    less<pair<T,int>>, rb_tree_tag,
    tree_order_statistics_node_update>;
```

| Method           | Returns                             |
|------------------|-------------------------------------|
| find_by_order(k) | iterator to k-th smallest (0-based) |
| order_of_key(x)  | # elements strictly < x             |

# String Operations

std::string

| Method                | Description                  |
|-----------------------|------------------------------|
| s.size() / s.length() | length                       |
| s.find("sub")         | first index, npos if missing |
| s.substr(pos, len)    | substring                    |
| s.push_back(c)        | append char                  |

|                          |                  |
|--------------------------|------------------|
| s.pop_back()             | remove last      |
| s.insert(pos, str)       | insert substring |
| s.replace(pos, len, str) | replace range    |
| getline(cin, s)          | read whole line  |
| s.swap(t)                | swap strings     |
| isupper / islower        | char check       |
| toupper / tolower        | char convert     |
| sort(all(s))             | sort chars       |
| reverse(all(s))          | reverse          |

## STL Algorithms

*<algorithm> · <numeric>*

### Accumulate / partial\_sum

```
ll s = accumulate(all(v), 0LL);           // sum
ll p = accumulate(all(v), 1LL,
    [](ll a, ll b){return a*b;});          // product
partial_sum(all(v), res.begin());         // prefix sum
partial_sum(all(v), res.begin(),
    multiplies<ll>());                     // prefix product
```

### Other useful algorithms

```
min_element(all(v));
max_element(all(v));
lower_bound(all(v), x);    // iterator to first >= x
upper_bound(all(v), x);    // iterator to first > x
binary_search(all(v), x);  // bool, requires sorted
reverse(all(v));
unique(all(v));             // after sort, remove dups
v.erase(unique(all(v)), v.end()); // deduplicate
next_permutation(all(v));  // iterate permutations
iota(all(v), 0);           // fill 0,1,2,...
fill(all(v), val);
```

## Pairs & Vectors

*quick ref*

### Pair

```
pair<int,int> p={x,y};
p.first; p.second;
p=make_pair(x,y);
vector<pair<int,int>> vp(n);
for(auto& x:vp) cin>>x.first>>x.second;
```

### Vector tips

```
v.push_back(x); v.pop_back();
v.resize(n);    v.assign(n, val);
v.clear();      v.empty();
*(v.begin()+k); // k-th element via iterator
vector<vi> g(n, vi(m, 0)); // 2D vector

// Output last element without newline:
cout << a[i] << " \n"[i==n-1];
```

## Interactive Template

*prefix: itcp*

```
// Same header as tcp but:
// 1. DO NOT use #define endl '\n' (interactive needs immediate flush)
// 2. Use cout << "\n" then cout.flush() or cout << endl (real std::endl)
// 3. No file redirection (no freopen)

void ask(int x){
    cout << x << "\n";
    cout.flush();          // CRITICAL
}
```

## Debug Include

prefix: debuginc

```
#ifndef ONLINE_JUDGE
#include "Utils/debug.h"
#endif
```

## Small Primes List

prefix: primes

```
const int primes[] = {
    2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53
};
// product of all overflows even ull
```

Always use ll for distances in Dijkstra — total path weight can exceed  $2 \times 10^9$

## Complexity Quick Reference

| Operation                    | Complexity         |
|------------------------------|--------------------|
| Sieve of Eratosthenes        | $O(N \log \log N)$ |
| SPF sieve preprocess         | $O(N \log \log N)$ |
| SPF factorization query      | $O(\log N)$        |
| Prime factorization (sqrt n) | $O(\sqrt{N})$      |
| Divisors (sqrt n)            | $O(\sqrt{N})$      |
| All divisors precompute      | $O(N \log N)$      |
| DFS / BFS                    | $O(V + E)$         |
| Dijkstra                     | $O((V+E) \log V)$  |
| 0-1 BFS                      | $O(V + E)$         |
| DSU find / unite             | $O(\alpha)$        |
| Seg Tree update / query      | $O(\log N)$        |
| PBDS find_by_order           | $O(\log N)$        |
| Set insert / find            | $O(\log N)$        |
| Map access                   | $O(\log N)$        |
| Unordered map                | $O(1)$ avg         |
| PQ push / pop                | $O(\log N)$        |
| PQ top / peek                | $O(1)$             |
| Binary search                | $O(\log N)$        |

# Theoretical Computer Science Cheat Sheet

| Definitions                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                       | Series                                                                                                                                                                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $f(n) = O(g(n))$                                                                                                                                                                                                                                                       | iff $\exists$ positive $c, n_0$ such that $0 \leq f(n) \leq cg(n) \forall n \geq n_0$ .                                                                                                                                                               | $\sum_{i=1}^n i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}.$                                                                                                                                                                |
| $f(n) = \Omega(g(n))$                                                                                                                                                                                                                                                  | iff $\exists$ positive $c, n_0$ such that $f(n) \geq cg(n) \geq 0 \forall n \geq n_0$ .                                                                                                                                                               | In general:                                                                                                                                                                                                                                                                                         |
| $f(n) = \Theta(g(n))$                                                                                                                                                                                                                                                  | iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ .                                                                                                                                                                                                      | $\sum_{i=1}^n i^m = \frac{1}{m+1} \left[ (n+1)^{m+1} - 1 - \sum_{i=1}^n ((i+1)^{m+1} - i^{m+1} - (m+1)i^m) \right]$                                                                                                                                                                                 |
| $f(n) = o(g(n))$                                                                                                                                                                                                                                                       | iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$ .                                                                                                                                                                                                     | $\sum_{i=1}^{n-1} i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k n^{m+1-k}.$                                                                                                                                                                                                                   |
| $\lim_{n \rightarrow \infty} a_n = a$                                                                                                                                                                                                                                  | iff $\forall \epsilon > 0, \exists n_0$ such that $ a_n - a  < \epsilon, \forall n \geq n_0$ .                                                                                                                                                        | Geometric series:                                                                                                                                                                                                                                                                                   |
| $\sup S$                                                                                                                                                                                                                                                               | least $b \in \mathbb{R}$ such that $b \geq s, \forall s \in S$ .                                                                                                                                                                                      | $\sum_{i=0}^n c^i = \frac{c^{n+1} - 1}{c - 1}, \quad c \neq 1, \quad \sum_{i=0}^{\infty} c^i = \frac{1}{1 - c}, \quad \sum_{i=1}^{\infty} c^i = \frac{c}{1 - c}, \quad  c  < 1,$                                                                                                                    |
| $\inf S$                                                                                                                                                                                                                                                               | greatest $b \in \mathbb{R}$ such that $b \leq s, \forall s \in S$ .                                                                                                                                                                                   | $\sum_{i=0}^n ic^i = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}, \quad c \neq 1, \quad \sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2}, \quad  c  < 1.$                                                                                                                                               |
| $\liminf_{n \rightarrow \infty} a_n$                                                                                                                                                                                                                                   | $\lim_{n \rightarrow \infty} \inf \{a_i \mid i \geq n, i \in \mathbb{N}\}.$                                                                                                                                                                           | Harmonic series:                                                                                                                                                                                                                                                                                    |
| $\limsup_{n \rightarrow \infty} a_n$                                                                                                                                                                                                                                   | $\lim_{n \rightarrow \infty} \sup \{a_i \mid i \geq n, i \in \mathbb{N}\}.$                                                                                                                                                                           | $H_n = \sum_{i=1}^n \frac{1}{i}, \quad \sum_{i=1}^n iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}.$                                                                                                                                                                                                 |
| $\binom{n}{k}$                                                                                                                                                                                                                                                         | Combinations: Size $k$ sub-sets of a size $n$ set.                                                                                                                                                                                                    | $\sum_{i=1}^n H_i = (n+1)H_n - n, \quad \sum_{i=1}^n \binom{i}{m} H_i = \binom{n+1}{m+1} \left( H_{n+1} - \frac{1}{m+1} \right).$                                                                                                                                                                   |
| $[n]$                                                                                                                                                                                                                                                                  | Stirling numbers (1st kind): Arrangements of an $n$ element set into $k$ cycles.                                                                                                                                                                      | 1. $\binom{n}{k} = \frac{n!}{(n-k)!k!}, \quad 2. \sum_{k=0}^n \binom{n}{k} = 2^n, \quad 3. \binom{n}{k} = \binom{n}{n-k},$                                                                                                                                                                          |
| $\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$                                                                                                                                                                                                        | Stirling numbers (2nd kind): Partitions of an $n$ element set into $k$ non-empty sets.                                                                                                                                                                | 4. $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}, \quad 5. \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1},$                                                                                                                                                                                        |
| $\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle$                                                                                                                                                                                                         | 1st order Eulerian numbers: Permutations $\pi_1 \pi_2 \dots \pi_n$ on $\{1, 2, \dots, n\}$ with $k$ ascents.                                                                                                                                          | 6. $\binom{n}{m} \binom{m}{k} = \binom{n}{k} \binom{n-k}{m-k}, \quad 7. \sum_{k=0}^n \binom{r+k}{k} = \binom{r+n+1}{n},$                                                                                                                                                                            |
| $\langle \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \rangle$                                                                                                                                                                                         | 2nd order Eulerian numbers.                                                                                                                                                                                                                           | 8. $\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1}, \quad 9. \sum_{k=0}^n \binom{r}{k} \binom{s}{n-k} = \binom{r+s}{n},$                                                                                                                                                                              |
| $C_n$                                                                                                                                                                                                                                                                  | Catalan Numbers: Binary trees with $n+1$ vertices.                                                                                                                                                                                                    | 10. $\binom{n}{k} = (-1)^k \binom{k-n-1}{k}, \quad 11. \left\{ \begin{smallmatrix} n \\ 1 \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n \\ n \end{smallmatrix} \right\} = 1,$                                                                                                          |
| 14. $\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)!,$                                                                                                                                                                                                                   | 15. $\begin{bmatrix} n \\ 2 \end{bmatrix} = (n-1)!H_{n-1},$                                                                                                                                                                                           | 12. $\left\{ \begin{smallmatrix} n \\ 2 \end{smallmatrix} \right\} = 2^{n-1} - 1, \quad 13. \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} = k \left\{ \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\} + \left\{ \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\},$ |
| 16. $\begin{bmatrix} n \\ n \end{bmatrix} = 1,$                                                                                                                                                                                                                        | 17. $\begin{bmatrix} n \\ k \end{bmatrix} \geq \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\},$                                                                                                                                        |                                                                                                                                                                                                                                                                                                     |
| 18. $\begin{bmatrix} n \\ k \end{bmatrix} = (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix},$                                                                                                                                  | 19. $\left\{ \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \right\} = \begin{bmatrix} n \\ n-1 \end{bmatrix} = \binom{n}{2},$                                                                                                                        | 20. $\sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} = n!, \quad 21. C_n = \frac{1}{n+1} \binom{2n}{n},$                                                                                                                                                                                          |
| 22. $\langle \begin{smallmatrix} n \\ 0 \end{smallmatrix} \rangle = \langle \begin{smallmatrix} n \\ n-1 \end{smallmatrix} \rangle = 1,$                                                                                                                               | 23. $\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle = \langle \begin{smallmatrix} n \\ n-1-k \end{smallmatrix} \rangle,$                                                                                                                | 24. $\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle = (k+1) \langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \rangle + (n-k) \langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \rangle,$                                                                                 |
| 25. $\langle \begin{smallmatrix} 0 \\ k \end{smallmatrix} \rangle = \begin{cases} 1 & \text{if } k=0, \\ 0 & \text{otherwise} \end{cases}$                                                                                                                             | 26. $\langle \begin{smallmatrix} n \\ 1 \end{smallmatrix} \rangle = 2^n - n - 1,$                                                                                                                                                                     | 27. $\langle \begin{smallmatrix} n \\ 2 \end{smallmatrix} \rangle = 3^n - (n+1)2^n + \binom{n+1}{2},$                                                                                                                                                                                               |
| 28. $x^n = \sum_{k=0}^n \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \binom{x+k}{n},$                                                                                                                                                                  | 29. $\langle \begin{smallmatrix} n \\ m \end{smallmatrix} \rangle = \sum_{k=0}^m \binom{n+1}{k} (m+1-k)^n (-1)^k,$                                                                                                                                    | 30. $m! \left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\} = \sum_{k=0}^n \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \binom{k}{n-m},$                                                                                                                                  |
| 31. $\langle \begin{smallmatrix} n \\ m \end{smallmatrix} \rangle = \sum_{k=0}^n \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\} \binom{n-k}{m} (-1)^{n-k-m} k!,$                                                                                        | 32. $\langle \langle \begin{smallmatrix} n \\ 0 \end{smallmatrix} \rangle \rangle = 1,$                                                                                                                                                               | 33. $\langle \langle \begin{smallmatrix} n \\ n \end{smallmatrix} \rangle \rangle = 0 \text{ for } n \neq 0,$                                                                                                                                                                                       |
| 34. $\langle \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \rangle = (k+1) \langle \langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \rangle \rangle + (2n-1-k) \langle \langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \rangle \rangle,$ | 35. $\sum_{k=0}^n \langle \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \rangle = \frac{(2n)^n}{2^n},$                                                                                                                                 |                                                                                                                                                                                                                                                                                                     |
| 36. $\left\{ \begin{smallmatrix} x \\ x-n \end{smallmatrix} \right\} = \sum_{k=0}^n \langle \langle \begin{smallmatrix} n \\ k \end{smallmatrix} \rangle \rangle \binom{x+n-1-k}{2n},$                                                                                 | 37. $\left\{ \begin{smallmatrix} n+1 \\ m+1 \end{smallmatrix} \right\} = \sum_k \binom{n}{k} \left\{ \begin{smallmatrix} k \\ m \end{smallmatrix} \right\} = \sum_{k=0}^n \left\{ \begin{smallmatrix} k \\ m \end{smallmatrix} \right\} (m+1)^{n-k},$ |                                                                                                                                                                                                                                                                                                     |

# Theoretical Computer Science Cheat Sheet

## Identities Cont.

$$\begin{aligned}
 38. \quad \binom{n+1}{m+1} &= \sum_k \binom{n}{k} \binom{k}{m} = \sum_{k=0}^n \binom{k}{m} n^{n-k} = n! \sum_{k=0}^n \frac{1}{k!} \binom{k}{m}, & 39. \quad \begin{bmatrix} x \\ x-n \end{bmatrix} &= \sum_{k=0}^n \left\langle \begin{matrix} n \\ k \end{matrix} \right\rangle \binom{x+k}{2n}, \\
 40. \quad \left\{ \begin{matrix} n \\ m \end{matrix} \right\} &= \sum_k \binom{n}{k} \left\{ \begin{matrix} k+1 \\ m+1 \end{matrix} \right\} (-1)^{n-k}, & 41. \quad \begin{bmatrix} n \\ m \end{bmatrix} &= \sum_k \begin{bmatrix} n+1 \\ k+1 \end{bmatrix} \binom{k}{m} (-1)^{m-k}, \\
 42. \quad \left\{ \begin{matrix} m+n+1 \\ m \end{matrix} \right\} &= \sum_{k=0}^m k \left\{ \begin{matrix} n+k \\ k \end{matrix} \right\}, & 43. \quad \begin{bmatrix} m+n+1 \\ m \end{bmatrix} &= \sum_{k=0}^m k(n+k) \begin{bmatrix} n+k \\ k \end{bmatrix}, \\
 44. \quad \binom{n}{m} &= \sum_k \binom{n+1}{k+1} \binom{k}{m} (-1)^{m-k}, & 45. \quad (n-m)! \binom{n}{m} &= \sum_k \begin{bmatrix} n+1 \\ k+1 \end{bmatrix} \left\{ \begin{matrix} k \\ m \end{matrix} \right\} (-1)^{m-k}, \quad \text{for } n \geq m, \\
 46. \quad \left\{ \begin{matrix} n \\ n-m \end{matrix} \right\} &= \sum_k \binom{m-n}{m+k} \binom{m+n}{n+k} \begin{bmatrix} m+k \\ k \end{bmatrix}, & 47. \quad \begin{bmatrix} n \\ n-m \end{bmatrix} &= \sum_k \binom{m-n}{m+k} \binom{m+n}{n+k} \left\{ \begin{matrix} m+k \\ k \end{matrix} \right\}, \\
 48. \quad \left\{ \begin{matrix} n \\ \ell+m \end{matrix} \right\} \binom{\ell+m}{\ell} &= \sum_k \left\{ \begin{matrix} k \\ \ell \end{matrix} \right\} \left\{ \begin{matrix} n-k \\ m \end{matrix} \right\} \binom{n}{k}, & 49. \quad \begin{bmatrix} n \\ \ell+m \end{bmatrix} \binom{\ell+m}{\ell} &= \sum_k \begin{bmatrix} k \\ \ell \end{bmatrix} \begin{bmatrix} n-k \\ m \end{bmatrix} \binom{n}{k}.
 \end{aligned}$$

## Trees

Every tree with  $n$  vertices has  $n-1$  edges.

Kraft inequality: If the depths of the leaves of a binary tree are  $d_1, \dots, d_n$ :

$$\sum_{i=1}^n 2^{-d_i} \leq 1,$$

and equality holds only if every internal node has 2 sons.

## Recurrences

Master method:

$$T(n) = aT(n/b) + f(n), \quad a \geq 1, b > 1$$

If  $\exists \epsilon > 0$  such that  $f(n) = O(n^{\log_b a - \epsilon})$  then

$$T(n) = \Theta(n^{\log_b a}).$$

If  $f(n) = \Theta(n^{\log_b a})$  then

$$T(n) = \Theta(n^{\log_b a} \log_2 n).$$

If  $\exists \epsilon > 0$  such that  $f(n) = \Omega(n^{\log_b a + \epsilon})$ , and  $\exists c < 1$  such that  $af(n/b) \leq cf(n)$  for large  $n$ , then

$$T(n) = \Theta(f(n)).$$

Substitution (example): Consider the following recurrence

$$T_{i+1} = 2^{2^i} \cdot T_i^2, \quad T_1 = 2.$$

Note that  $T_i$  is always a power of two.

Let  $t_i = \log_2 T_i$ . Then we have

$$t_{i+1} = 2^i + 2t_i, \quad t_1 = 1.$$

Let  $u_i = t_i/2^i$ . Dividing both sides of the previous equation by  $2^{i+1}$  we get

$$\frac{t_{i+1}}{2^{i+1}} = \frac{2^i}{2^{i+1}} + \frac{t_i}{2^i}.$$

Substituting we find

$$u_{i+1} = \frac{1}{2} + u_i, \quad u_1 = \frac{1}{2},$$

which is simply  $u_i = i/2$ . So we find that  $T_i$  has the closed form  $T_i = 2^{i2^{i-1}}$ .

Summing factors (example): Consider the following recurrence

$$T(n) = 3T(n/2) + n, \quad T(1) = 1.$$

Rewrite so that all terms involving  $T$  are on the left side

$$T(n) - 3T(n/2) = n.$$

Now expand the recurrence, and choose a factor which makes the left side “telescope”

$$\begin{aligned}
 1(T(n) - 3T(n/2)) &= n \\
 3(T(n/2) - 3T(n/4)) &= n/2 \\
 &\vdots \\
 3^{\log_2 n - 1}(T(2) - 3T(1)) &= 2
 \end{aligned}$$

Let  $m = \log_2 n$ . Summing the left side we get  $T(n) - 3^m T(1) = T(n) - 3^m = T(n) - n^k$  where  $k = \log_2 3 \approx 1.58496$ . Summing the right side we get

$$\sum_{i=0}^{m-1} \frac{n}{2^i} 3^i = n \sum_{i=0}^{m-1} \left(\frac{3}{2}\right)^i.$$

Let  $c = \frac{3}{2}$ . Then we have

$$\begin{aligned}
 n \sum_{i=0}^{m-1} c^i &= n \left( \frac{c^m - 1}{c - 1} \right) \\
 &= 2n(c^{\log_2 n} - 1) \\
 &= 2n(c^{(k-1)\log_2 n} - 1) \\
 &= 2n^k - 2n,
 \end{aligned}$$

and so  $T(n) = 3n^k - 2n$ . Full history recurrences can often be changed to limited history ones (example): Consider

$$T_i = 1 + \sum_{j=0}^{i-1} T_j, \quad T_0 = 1.$$

Note that

$$T_{i+1} = 1 + \sum_{j=0}^i T_j.$$

Subtracting we find

$$\begin{aligned}
 T_{i+1} - T_i &= 1 + \sum_{j=0}^i T_j - 1 - \sum_{j=0}^{i-1} T_j \\
 &= T_i.
 \end{aligned}$$

And so  $T_{i+1} = 2T_i = 2^{i+1}$ .

Generating functions:

1. Multiply both sides of the equation by  $x^i$ .
2. Sum both sides over all  $i$  for which the equation is valid.
3. Choose a generating function  $G(x)$ . Usually  $G(x) = \sum_{i=0}^{\infty} x^i g_i$ .
3. Rewrite the equation in terms of the generating function  $G(x)$ .
4. Solve for  $G(x)$ .
5. The coefficient of  $x^i$  in  $G(x)$  is  $g_i$ .

Example:

$$g_{i+1} = 2g_i + 1, \quad g_0 = 0.$$

Multiply and sum:

$$\sum_{i \geq 0} g_{i+1} x^i = \sum_{i \geq 0} 2g_i x^i + \sum_{i \geq 0} x^i.$$

We choose  $G(x) = \sum_{i \geq 0} x^i g_i$ . Rewrite in terms of  $G(x)$ :

$$\frac{G(x) - g_0}{x} = 2G(x) + \sum_{i \geq 0} x^i.$$

Simplify:

$$\frac{G(x)}{x} = 2G(x) + \frac{1}{1-x}.$$

Solve for  $G(x)$ :

$$G(x) = \frac{x}{(1-x)(1-2x)}.$$

Expand this using partial fractions:

$$\begin{aligned}
 G(x) &= x \left( \frac{2}{1-2x} - \frac{1}{1-x} \right) \\
 &= x \left( 2 \sum_{i \geq 0} 2^i x^i - \sum_{i \geq 0} x^i \right) \\
 &= \sum_{i \geq 0} (2^{i+1} - 1) x^{i+1}.
 \end{aligned}$$

So  $g_i = 2^i - 1$ .

# Theoretical Computer Science Cheat Sheet

$\pi \approx 3.14159,$

$e \approx 2.71828,$

$\gamma \approx 0.57721,$

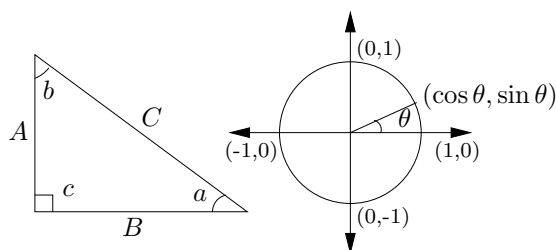
$\phi = \frac{1+\sqrt{5}}{2} \approx 1.61803,$

$\hat{\phi} = \frac{1-\sqrt{5}}{2} \approx -.61803$

| $i$                                 | $2^i$         | $p_i$ | General                                                                                                                                                                                                                        | Probability                                                                                   |
|-------------------------------------|---------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1                                   | 2             | 2     | Bernoulli Numbers ( $B_i = 0$ , odd $i \neq 1$ ):                                                                                                                                                                              | Continuous distributions: If                                                                  |
| 2                                   | 4             | 3     | $B_0 = 1, B_1 = -\frac{1}{2}, B_2 = \frac{1}{6}, B_4 = -\frac{1}{30},$                                                                                                                                                         | $\Pr[a < X < b] = \int_a^b p(x) dx,$                                                          |
| 3                                   | 8             | 5     | $B_6 = \frac{1}{42}, B_8 = -\frac{1}{30}, B_{10} = \frac{5}{66}.$                                                                                                                                                              | then $p$ is the probability density function of $X$ . If                                      |
| 4                                   | 16            | 7     | Change of base, quadratic formula:                                                                                                                                                                                             | $\Pr[X < a] = P(a),$                                                                          |
| 5                                   | 32            | 11    | $\log_b x = \frac{\log_a x}{\log_a b}, \quad \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$                                                                                                                                              | then $P$ is the distribution function of $X$ . If $P$ and $p$ both exist then                 |
| 6                                   | 64            | 13    | Euler's number $e$ :                                                                                                                                                                                                           | $P(a) = \int_{-\infty}^a p(x) dx.$                                                            |
| 7                                   | 128           | 17    | $e = 1 + \frac{1}{2} + \frac{1}{6} + \frac{1}{24} + \frac{1}{120} + \dots$                                                                                                                                                     | Expectation: If $X$ is discrete                                                               |
| 8                                   | 256           | 19    | $\lim_{n \rightarrow \infty} \left(1 + \frac{x}{n}\right)^n = e^x.$                                                                                                                                                            | $E[g(X)] = \sum_x g(x) \Pr[X = x].$                                                           |
| 9                                   | 512           | 23    | $\left(1 + \frac{1}{n}\right)^n < e < \left(1 + \frac{1}{n}\right)^{n+1}.$                                                                                                                                                     | If $X$ continuous then                                                                        |
| 10                                  | 1,024         | 29    | $\left(1 + \frac{1}{n}\right)^n = e - \frac{e}{2n} + \frac{11e}{24n^2} - O\left(\frac{1}{n^3}\right).$                                                                                                                         | $E[g(X)] = \int_{-\infty}^{\infty} g(x)p(x) dx = \int_{-\infty}^{\infty} g(x) dP(x).$         |
| 11                                  | 2,048         | 31    | Harmonic numbers:                                                                                                                                                                                                              | Variance, standard deviation:                                                                 |
| 12                                  | 4,096         | 37    | $1, \frac{3}{2}, \frac{11}{6}, \frac{25}{12}, \frac{137}{60}, \frac{49}{20}, \frac{363}{140}, \frac{761}{280}, \frac{7129}{2520}, \dots$                                                                                       | $\text{VAR}[X] = E[X^2] - E[X]^2,$                                                            |
| 13                                  | 8,192         | 41    | $\ln n < H_n < \ln n + 1,$                                                                                                                                                                                                     | $\sigma = \sqrt{\text{VAR}[X]}.$                                                              |
| 14                                  | 16,384        | 43    | $H_n = \ln n + \gamma + O\left(\frac{1}{n}\right).$                                                                                                                                                                            | For events $A$ and $B$ :                                                                      |
| 15                                  | 32,768        | 47    | Factorial, Stirling's approximation:                                                                                                                                                                                           | $\Pr[A \vee B] = \Pr[A] + \Pr[B] - \Pr[A \wedge B]$                                           |
| 16                                  | 65,536        | 53    | $1, 2, 6, 24, 120, 720, 5040, 40320, 362880, \dots$                                                                                                                                                                            | $\Pr[A \wedge B] = \Pr[A] \cdot \Pr[B],$                                                      |
| 17                                  | 131,072       | 59    | $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$                                                                                                                               | iff $A$ and $B$ are independent.                                                              |
| 18                                  | 262,144       | 61    | Ackermann's function and inverse:                                                                                                                                                                                              | $\Pr[A B] = \frac{\Pr[A \wedge B]}{\Pr[B]}$                                                   |
| 19                                  | 524,288       | 67    | $a(i, j) = \begin{cases} 2^j & i = 1 \\ a(i-1, 2) & j = 1 \\ a(i-1, a(i, j-1)) & i, j \geq 2 \end{cases}$                                                                                                                      | For random variables $X$ and $Y$ :                                                            |
| 20                                  | 1,048,576     | 71    | $\alpha(i) = \min\{j \mid a(j, j) \geq i\}.$                                                                                                                                                                                   | $E[X \cdot Y] = E[X] \cdot E[Y],$                                                             |
| 21                                  | 2,097,152     | 73    |                                                                                                                                                                                                                                | if $X$ and $Y$ are independent.                                                               |
| 22                                  | 4,194,304     | 79    |                                                                                                                                                                                                                                | $E[X + Y] = E[X] + E[Y],$                                                                     |
| 23                                  | 8,388,608     | 83    |                                                                                                                                                                                                                                | $E[cX] = cE[X].$                                                                              |
| 24                                  | 16,777,216    | 89    | Binomial distribution:                                                                                                                                                                                                         | Bayes' theorem:                                                                               |
| 25                                  | 33,554,432    | 97    | $\Pr[X = k] = \binom{n}{k} p^k q^{n-k}, \quad q = 1 - p,$                                                                                                                                                                      | $\Pr[A_i B] = \frac{\Pr[B A_i] \Pr[A_i]}{\sum_{j=1}^n \Pr[A_j] \Pr[B A_j]}.$                  |
| 26                                  | 67,108,864    | 101   | $E[X] = \sum_{k=1}^n k \binom{n}{k} p^k q^{n-k} = np.$                                                                                                                                                                         | Inclusion-exclusion:                                                                          |
| 27                                  | 134,217,728   | 103   | Poisson distribution:                                                                                                                                                                                                          | $\Pr\left[\bigvee_{i=1}^n X_i\right] = \sum_{i=1}^n \Pr[X_i] +$                               |
| 28                                  | 268,435,456   | 107   | $\Pr[X = k] = \frac{e^{-\lambda} \lambda^k}{k!}, \quad E[X] = \lambda.$                                                                                                                                                        | $\sum_{k=2}^n (-1)^{k+1} \sum_{i_1 < \dots < i_k} \Pr\left[\bigwedge_{j=1}^k X_{i_j}\right].$ |
| 29                                  | 536,870,912   | 109   | Normal (Gaussian) distribution:                                                                                                                                                                                                | Moment inequalities:                                                                          |
| 30                                  | 1,073,741,824 | 113   | $p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}, \quad E[X] = \mu.$                                                                                                                                               | $\Pr[ X  \geq \lambda E[X]] \leq \frac{1}{\lambda},$                                          |
| 31                                  | 2,147,483,648 | 127   | The "coupon collector": We are given a random coupon each day, and there are $n$ different types of coupons. The distribution of coupons is uniform. The expected number of days to pass before we to collect all $n$ types is | $\Pr[ X - E[X]  \geq \lambda \cdot \sigma] \leq \frac{1}{\lambda^2}.$                         |
| 32                                  | 4,294,967,296 | 131   | $nH_n.$                                                                                                                                                                                                                        | Geometric distribution:                                                                       |
| Pascal's Triangle                   |               |       |                                                                                                                                                                                                                                | $\Pr[X = k] = pq^{k-1}, \quad q = 1 - p,$                                                     |
| 1                                   |               |       |                                                                                                                                                                                                                                | $E[X] = \sum_{k=1}^{\infty} kpq^{k-1} = \frac{1}{p}.$                                         |
| 1 1                                 |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 2 1                               |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 3 3 1                             |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 4 6 4 1                           |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 5 10 10 5 1                       |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 6 15 20 15 6 1                    |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 7 21 35 35 21 7 1                 |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 8 28 56 70 56 28 8 1              |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 9 36 84 126 126 84 36 9 1         |               |       |                                                                                                                                                                                                                                |                                                                                               |
| 1 10 45 120 210 252 210 120 45 10 1 |               |       |                                                                                                                                                                                                                                |                                                                                               |

# Theoretical Computer Science Cheat Sheet

## Trigonometry



Pythagorean theorem:

$$C^2 = A^2 + B^2.$$

Definitions:

$$\sin a = A/C, \quad \cos a = B/C,$$

$$\csc a = C/A, \quad \sec a = C/B,$$

$$\tan a = \frac{\sin a}{\cos a} = \frac{A}{B}, \quad \cot a = \frac{\cos a}{\sin a} = \frac{B}{A}.$$

Area, radius of inscribed circle:

$$\frac{1}{2}AB, \quad \frac{AB}{A+B+C}.$$

Identities:

$$\sin x = \frac{1}{\csc x}, \quad \cos x = \frac{1}{\sec x},$$

$$\tan x = \frac{1}{\cot x}, \quad \sin^2 x + \cos^2 x = 1,$$

$$1 + \tan^2 x = \sec^2 x, \quad 1 + \cot^2 x = \csc^2 x,$$

$$\sin x = \cos\left(\frac{\pi}{2} - x\right), \quad \sin x = \sin(\pi - x),$$

$$\cos x = -\cos(\pi - x), \quad \tan x = \cot\left(\frac{\pi}{2} - x\right),$$

$$\cot x = -\cot(\pi - x), \quad \csc x = \cot \frac{x}{2} - \cot x,$$

$$\sin(x \pm y) = \sin x \cos y \pm \cos x \sin y,$$

$$\cos(x \pm y) = \cos x \cos y \mp \sin x \sin y,$$

$$\tan(x \pm y) = \frac{\tan x \pm \tan y}{1 \mp \tan x \tan y},$$

$$\cot(x \pm y) = \frac{\cot x \cot y \mp 1}{\cot x \pm \cot y},$$

$$\sin 2x = 2 \sin x \cos x, \quad \sin 2x = \frac{2 \tan x}{1 + \tan^2 x},$$

$$\cos 2x = \cos^2 x - \sin^2 x, \quad \cos 2x = 2 \cos^2 x - 1,$$

$$\cos 2x = 1 - 2 \sin^2 x, \quad \cos 2x = \frac{1 - \tan^2 x}{1 + \tan^2 x},$$

$$\tan 2x = \frac{2 \tan x}{1 - \tan^2 x}, \quad \cot 2x = \frac{\cot^2 x - 1}{2 \cot x},$$

$$\sin(x + y) \sin(x - y) = \sin^2 x - \sin^2 y,$$

$$\cos(x + y) \cos(x - y) = \cos^2 x - \sin^2 y.$$

Euler's equation:

$$e^{ix} = \cos x + i \sin x, \quad e^{i\pi} = -1.$$

## Matrices

Multiplication:

$$C = A \cdot B, \quad c_{i,j} = \sum_{k=1}^n a_{i,k} b_{k,j}.$$

Determinants:  $\det A \neq 0$  iff  $A$  is non-singular.

$$\det A \cdot B = \det A \cdot \det B,$$

$$\det A = \sum_{\pi} \prod_{i=1}^n \text{sign}(\pi) a_{i,\pi(i)}.$$

$2 \times 2$  and  $3 \times 3$  determinant:

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc,$$

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = g \begin{vmatrix} b & c \\ e & f \end{vmatrix} - h \begin{vmatrix} a & c \\ d & f \end{vmatrix} + i \begin{vmatrix} a & b \\ d & e \end{vmatrix}$$

$$= aei + bfg + cdh - ceg - fha - ibd.$$

Permanents:

$$\text{perm } A = \sum_{\pi} \prod_{i=1}^n a_{i,\pi(i)}.$$

## Hyperbolic Functions

Definitions:

$$\sinh x = \frac{e^x - e^{-x}}{2}, \quad \cosh x = \frac{e^x + e^{-x}}{2},$$

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \text{csch } x = \frac{1}{\sinh x},$$

$$\text{sech } x = \frac{1}{\cosh x}, \quad \coth x = \frac{1}{\tanh x}.$$

Identities:

$$\cosh^2 x - \sinh^2 x = 1, \quad \tanh^2 x + \text{sech}^2 x = 1,$$

$$\coth^2 x - \text{csch}^2 x = 1, \quad \sinh(-x) = -\sinh x,$$

$$\cosh(-x) = \cosh x, \quad \tanh(-x) = -\tanh x,$$

$$\sinh(x + y) = \sinh x \cosh y + \cosh x \sinh y,$$

$$\cosh(x + y) = \cosh x \cosh y + \sinh x \sinh y,$$

$$\sinh 2x = 2 \sinh x \cosh x,$$

$$\cosh 2x = \cosh^2 x + \sinh^2 x,$$

$$\cosh x + \sinh x = e^x, \quad \cosh x - \sinh x = e^{-x},$$

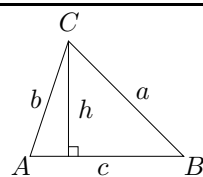
$$(\cosh x + \sinh x)^n = \cosh nx + \sinh nx, \quad n \in \mathbb{Z},$$

$$2 \sinh^2 \frac{x}{2} = \cosh x - 1, \quad 2 \cosh^2 \frac{x}{2} = \cosh x + 1.$$

| $\theta$        | $\sin \theta$        | $\cos \theta$        | $\tan \theta$        |
|-----------------|----------------------|----------------------|----------------------|
| 0               | 0                    | 1                    | 0                    |
| $\frac{\pi}{6}$ | $\frac{1}{2}$        | $\frac{\sqrt{3}}{2}$ | $\frac{\sqrt{3}}{3}$ |
| $\frac{\pi}{4}$ | $\frac{\sqrt{2}}{2}$ | $\frac{\sqrt{2}}{2}$ | 1                    |
| $\frac{\pi}{3}$ | $\frac{\sqrt{3}}{2}$ | $\frac{1}{2}$        | $\sqrt{3}$           |
| $\frac{\pi}{2}$ | 1                    | 0                    | $\infty$             |

... in mathematics you don't understand things, you just get used to them.  
- J. von Neumann

## More Trig.



Law of cosines:

$$c^2 = a^2 + b^2 - 2ab \cos C.$$

Area:

$$A = \frac{1}{2}hc,$$

$$= \frac{1}{2}ab \sin C,$$

$$= \frac{c^2 \sin A \sin B}{2 \sin C}.$$

Heron's formula:

$$A = \sqrt{s \cdot s_a \cdot s_b \cdot s_c},$$

$$s = \frac{1}{2}(a + b + c),$$

$$s_a = s - a,$$

$$s_b = s - b,$$

$$s_c = s - c.$$

More identities:

$$\sin \frac{x}{2} = \sqrt{\frac{1 - \cos x}{2}},$$

$$\cos \frac{x}{2} = \sqrt{\frac{1 + \cos x}{2}},$$

$$\tan \frac{x}{2} = \sqrt{\frac{1 - \cos x}{1 + \cos x}},$$

$$= \frac{1 - \cos x}{\sin x},$$

$$= \frac{\sin x}{1 + \cos x},$$

$$\cot \frac{x}{2} = \sqrt{\frac{1 + \cos x}{1 - \cos x}},$$

$$= \frac{1 + \cos x}{\sin x},$$

$$= \frac{\sin x}{1 - \cos x},$$

$$\sin x = \frac{e^{ix} - e^{-ix}}{2i},$$

$$\cos x = \frac{e^{ix} + e^{-ix}}{2},$$

$$\tan x = -i \frac{e^{ix} - e^{-ix}}{e^{ix} + e^{-ix}},$$

$$= -i \frac{e^{2ix} - 1}{e^{2ix} + 1},$$

$$\sin x = \frac{\sinh ix}{i},$$

$$\cos x = \cosh ix,$$

$$\tan x = \frac{\tanh ix}{i}.$$

# Theoretical Computer Science Cheat Sheet

## Number Theory

The Chinese remainder theorem: There exists a number  $C$  such that:

$$C \equiv r_1 \pmod{m_1}$$

$$\vdots \quad \vdots \quad \vdots$$

$$C \equiv r_n \pmod{m_n}$$

if  $m_i$  and  $m_j$  are relatively prime for  $i \neq j$ .

Euler's function:  $\phi(x)$  is the number of positive integers less than  $x$  relatively prime to  $x$ . If  $\prod_{i=1}^n p_i^{e_i}$  is the prime factorization of  $x$  then

$$\phi(x) = \prod_{i=1}^n p_i^{e_i-1} (p_i - 1).$$

Euler's theorem: If  $a$  and  $b$  are relatively prime then

$$1 \equiv a^{\phi(b)} \pmod{b}.$$

Fermat's theorem:

$$1 \equiv a^{p-1} \pmod{p}.$$

The Euclidean algorithm: if  $a > b$  are integers then

$$\gcd(a, b) = \gcd(a \bmod b, b).$$

If  $\prod_{i=1}^n p_i^{e_i}$  is the prime factorization of  $x$  then

$$S(x) = \sum_{d|x} d = \prod_{i=1}^n \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

Perfect Numbers:  $x$  is an even perfect number iff  $x = 2^{n-1}(2^n - 1)$  and  $2^n - 1$  is prime.

Wilson's theorem:  $n$  is a prime iff

$$(n-1)! \equiv -1 \pmod{n}.$$

Möbius inversion:

$$\mu(i) = \begin{cases} 1 & \text{if } i = 1. \\ 0 & \text{if } i \text{ is not square-free.} \\ (-1)^r & \text{if } i \text{ is the product of } r \text{ distinct primes.} \end{cases}$$

If

$$G(a) = \sum_{d|a} F(d),$$

then

$$F(a) = \sum_{d|a} \mu(d) G\left(\frac{a}{d}\right).$$

Prime numbers:

$$p_n = n \ln n + n \ln \ln n - n + n \frac{\ln \ln n}{\ln n}$$

$$+ O\left(\frac{n}{\ln n}\right),$$

$$\pi(n) = \frac{n}{\ln n} + \frac{n}{(\ln n)^2} + \frac{2!n}{(\ln n)^3}$$

$$+ O\left(\frac{n}{(\ln n)^4}\right).$$

## Graph Theory

### Definitions:

*Loop* An edge connecting a vertex to itself.

*Directed* Each edge has a direction.

*Simple* Graph with no loops or multi-edges.

*Walk* A sequence  $v_0 e_1 v_1 \dots e_\ell v_\ell$ .

*Trail* A walk with distinct edges.

*Path* A trail with distinct vertices.

*Connected* A graph where there exists a path between any two vertices.

*Component* A maximal connected subgraph.

*Tree* A connected acyclic graph.

*Free tree* A tree with no root.

*DAG* Directed acyclic graph.

*Eulerian* Graph with a trail visiting each edge exactly once.

*Hamiltonian* Graph with a cycle visiting each vertex exactly once.

*Cut* A set of edges whose removal increases the number of components.

*Cut-set* A minimal cut.

*Cut edge* A size 1 cut.

*k-Connected* A graph connected with the removal of any  $k-1$  vertices.

*k-Tough*  $\forall S \subseteq V, S \neq \emptyset$  we have  $k \cdot c(G-S) \leq |S|$ .

*k-Regular* A graph where all vertices have degree  $k$ .

*k-Factor* A  $k$ -regular spanning subgraph.

*Matching* A set of edges, no two of which are adjacent.

*Clique* A set of vertices, all of which are adjacent.

*Ind. set* A set of vertices, none of which are adjacent.

*Vertex cover* A set of vertices which cover all edges.

*Planar graph* A graph which can be embedded in the plane.

*Plane graph* An embedding of a planar graph.

$$\sum_{v \in V} \deg(v) = 2m.$$

If  $G$  is planar then  $n - m + f = 2$ , so

$$f \leq 2n - 4, \quad m \leq 3n - 6.$$

Any planar graph has a vertex with degree  $\leq 5$ .

### Notation:

$E(G)$  Edge set

$V(G)$  Vertex set

$c(G)$  Number of components

$G[S]$  Induced subgraph

$\deg(v)$  Degree of  $v$

$\Delta(G)$  Maximum degree

$\delta(G)$  Minimum degree

$\chi(G)$  Chromatic number

$\chi_E(G)$  Edge chromatic number

$G^c$  Complement graph

$K_n$  Complete graph

$K_{n_1, n_2}$  Complete bipartite graph

$r(k, \ell)$  Ramsey number

### Geometry

Projective coordinates: triples  $(x, y, z)$ , not all  $x, y$  and  $z$  zero.

$$(x, y, z) = (cx, cy, cz) \quad \forall c \neq 0.$$

Cartesian Projective

$$(x, y) \quad (x, y, 1)$$

$$y = mx + b \quad (m, -1, b)$$

$$x = c \quad (1, 0, -c)$$

Distance formula,  $L_p$  and  $L_\infty$  metric:

$$\sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2},$$

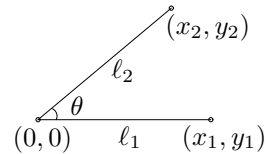
$$[|x_1 - x_0|^p + |y_1 - y_0|^p]^{1/p},$$

$$\lim_{p \rightarrow \infty} [|x_1 - x_0|^p + |y_1 - y_0|^p]^{1/p}.$$

Area of triangle  $(x_0, y_0)$ ,  $(x_1, y_1)$  and  $(x_2, y_2)$ :

$$\frac{1}{2} \text{abs} \begin{vmatrix} x_1 - x_0 & y_1 - y_0 \\ x_2 - x_0 & y_2 - y_0 \end{vmatrix}.$$

Angle formed by three points:



$$\cos \theta = \frac{(x_1, y_1) \cdot (x_2, y_2)}{\ell_1 \ell_2}.$$

Line through two points  $(x_0, y_0)$  and  $(x_1, y_1)$ :

$$\begin{vmatrix} x & y & 1 \\ x_0 & y_0 & 1 \\ x_1 & y_1 & 1 \end{vmatrix} = 0.$$

Area of circle, volume of sphere:

$$A = \pi r^2, \quad V = \frac{4}{3} \pi r^3.$$

If I have seen farther than others, it is because I have stood on the shoulders of giants.

– Issac Newton

# Theoretical Computer Science Cheat Sheet

$\pi$

Wallis' identity:

$$\pi = 2 \cdot \frac{2 \cdot 2 \cdot 4 \cdot 4 \cdot 6 \cdot 6 \cdots}{1 \cdot 3 \cdot 3 \cdot 5 \cdot 5 \cdot 7 \cdots}$$

Brouncker's continued fraction expansion:

$$\frac{\pi}{4} = 1 + \frac{1^2}{2 + \frac{3^2}{2 + \frac{5^2}{2 + \frac{7^2}{2 + \cdots}}}}$$

Gregory's series:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots$$

Newton's series:

$$\frac{\pi}{6} = \frac{1}{2} + \frac{1}{2 \cdot 3 \cdot 2^3} + \frac{1 \cdot 3}{2 \cdot 4 \cdot 5 \cdot 2^5} + \cdots$$

Sharp's series:

$$\frac{\pi}{6} = \frac{1}{\sqrt{3}} \left( 1 - \frac{1}{3^1 \cdot 3} + \frac{1}{3^2 \cdot 5} - \frac{1}{3^3 \cdot 7} + \cdots \right)$$

Euler's series:

$$\frac{\pi^2}{6} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \frac{1}{5^2} + \cdots$$

$$\frac{\pi^2}{8} = \frac{1}{1^2} + \frac{1}{3^2} + \frac{1}{5^2} + \frac{1}{7^2} + \frac{1}{9^2} + \cdots$$

$$\frac{\pi^2}{12} = \frac{1}{1^2} - \frac{1}{2^2} + \frac{1}{3^2} - \frac{1}{4^2} + \frac{1}{5^2} - \cdots$$

## Partial Fractions

Let  $N(x)$  and  $D(x)$  be polynomial functions of  $x$ . We can break down  $N(x)/D(x)$  using partial fraction expansion. First, if the degree of  $N$  is greater than or equal to the degree of  $D$ , divide  $N$  by  $D$ , obtaining

$$\frac{N(x)}{D(x)} = Q(x) + \frac{N'(x)}{D(x)},$$

where the degree of  $N'$  is less than that of  $D$ . Second, factor  $D(x)$ . Use the following rules: For a non-repeated factor:

$$\frac{N(x)}{(x-a)D(x)} = \frac{A}{x-a} + \frac{N'(x)}{D(x)},$$

where

$$A = \left[ \frac{N(x)}{D(x)} \right]_{x=a}.$$

For a repeated factor:

$$\frac{N(x)}{(x-a)^m D(x)} = \sum_{k=0}^{m-1} \frac{A_k}{(x-a)^{m-k}} + \frac{N'(x)}{D(x)},$$

where

$$A_k = \frac{1}{k!} \left[ \frac{d^k}{dx^k} \left( \frac{N(x)}{D(x)} \right) \right]_{x=a}.$$

The reasonable man adapts himself to the world; the unreasonable persists in trying to adapt the world to himself. Therefore all progress depends on the unreasonable.  
– George Bernard Shaw

## Calculus

Derivatives:

$$1. \frac{d(cu)}{dx} = c \frac{du}{dx}, \quad 2. \frac{d(u+v)}{dx} = \frac{du}{dx} + \frac{dv}{dx}, \quad 3. \frac{d(uv)}{dx} = u \frac{dv}{dx} + v \frac{du}{dx},$$

$$4. \frac{d(u^n)}{dx} = nu^{n-1} \frac{du}{dx}, \quad 5. \frac{d(u/v)}{dx} = \frac{v \left( \frac{du}{dx} \right) - u \left( \frac{dv}{dx} \right)}{v^2}, \quad 6. \frac{d(e^{cu})}{dx} = ce^{cu} \frac{du}{dx},$$

$$7. \frac{d(c^u)}{dx} = (\ln c) c^u \frac{du}{dx}, \quad 8. \frac{d(\ln u)}{dx} = \frac{1}{u} \frac{du}{dx},$$

$$9. \frac{d(\sin u)}{dx} = \cos u \frac{du}{dx}, \quad 10. \frac{d(\cos u)}{dx} = -\sin u \frac{du}{dx},$$

$$11. \frac{d(\tan u)}{dx} = \sec^2 u \frac{du}{dx}, \quad 12. \frac{d(\cot u)}{dx} = \csc^2 u \frac{du}{dx},$$

$$13. \frac{d(\sec u)}{dx} = \tan u \sec u \frac{du}{dx}, \quad 14. \frac{d(\csc u)}{dx} = -\cot u \csc u \frac{du}{dx},$$

$$15. \frac{d(\arcsin u)}{dx} = \frac{1}{\sqrt{1-u^2}} \frac{du}{dx}, \quad 16. \frac{d(\arccos u)}{dx} = \frac{-1}{\sqrt{1-u^2}} \frac{du}{dx},$$

$$17. \frac{d(\arctan u)}{dx} = \frac{1}{1+u^2} \frac{du}{dx}, \quad 18. \frac{d(\operatorname{arccot} u)}{dx} = \frac{-1}{1+u^2} \frac{du}{dx},$$

$$19. \frac{d(\operatorname{arcsec} u)}{dx} = \frac{1}{u\sqrt{1-u^2}} \frac{du}{dx}, \quad 20. \frac{d(\operatorname{arccsc} u)}{dx} = \frac{-1}{u\sqrt{1-u^2}} \frac{du}{dx},$$

$$21. \frac{d(\sinh u)}{dx} = \cosh u \frac{du}{dx}, \quad 22. \frac{d(\cosh u)}{dx} = \sinh u \frac{du}{dx},$$

$$23. \frac{d(\tanh u)}{dx} = \operatorname{sech}^2 u \frac{du}{dx}, \quad 24. \frac{d(\coth u)}{dx} = -\operatorname{csch}^2 u \frac{du}{dx},$$

$$25. \frac{d(\operatorname{sech} u)}{dx} = -\operatorname{sech} u \tanh u \frac{du}{dx}, \quad 26. \frac{d(\operatorname{csch} u)}{dx} = -\operatorname{csch} u \coth u \frac{du}{dx},$$

$$27. \frac{d(\operatorname{arcsinh} u)}{dx} = \frac{1}{\sqrt{1+u^2}} \frac{du}{dx}, \quad 28. \frac{d(\operatorname{arccosh} u)}{dx} = \frac{1}{\sqrt{u^2-1}} \frac{du}{dx},$$

$$29. \frac{d(\operatorname{arctanh} u)}{dx} = \frac{1}{1-u^2} \frac{du}{dx}, \quad 30. \frac{d(\operatorname{arcoth} u)}{dx} = \frac{1}{u^2-1} \frac{du}{dx},$$

$$31. \frac{d(\operatorname{arcsech} u)}{dx} = \frac{-1}{u\sqrt{1-u^2}} \frac{du}{dx}, \quad 32. \frac{d(\operatorname{arccsch} u)}{dx} = \frac{-1}{|u|\sqrt{1+u^2}} \frac{du}{dx}.$$

Integrals:

$$1. \int cu \, dx = c \int u \, dx, \quad 2. \int (u+v) \, dx = \int u \, dx + \int v \, dx,$$

$$3. \int x^n \, dx = \frac{1}{n+1} x^{n+1}, \quad n \neq -1, \quad 4. \int \frac{1}{x} \, dx = \ln x, \quad 5. \int e^x \, dx = e^x,$$

$$6. \int \frac{dx}{1+x^2} = \arctan x, \quad 7. \int u \frac{dv}{dx} \, dx = uv - \int v \frac{du}{dx} \, dx,$$

$$8. \int \sin x \, dx = -\cos x, \quad 9. \int \cos x \, dx = \sin x,$$

$$10. \int \tan x \, dx = -\ln |\cos x|, \quad 11. \int \cot x \, dx = \ln |\cos x|,$$

$$12. \int \sec x \, dx = \ln |\sec x + \tan x|, \quad 13. \int \csc x \, dx = \ln |\csc x + \cot x|,$$

$$14. \int \arcsin \frac{x}{a} \, dx = \arcsin \frac{x}{a} + \sqrt{a^2 - x^2}, \quad a > 0,$$

# Theoretical Computer Science Cheat Sheet

## Calculus Cont.

15.  $\int \arccos \frac{x}{a} dx = \arccos \frac{x}{a} - \sqrt{a^2 - x^2}, \quad a > 0,$
16.  $\int \arctan \frac{x}{a} dx = x \arctan \frac{x}{a} - \frac{a}{2} \ln(a^2 + x^2), \quad a > 0,$
17.  $\int \sin^2(ax) dx = \frac{1}{2a} (ax - \sin(ax) \cos(ax)),$
18.  $\int \cos^2(ax) dx = \frac{1}{2a} (ax + \sin(ax) \cos(ax)),$
19.  $\int \sec^2 x dx = \tan x,$
20.  $\int \csc^2 x dx = -\cot x,$
21.  $\int \sin^n x dx = -\frac{\sin^{n-1} x \cos x}{n} + \frac{n-1}{n} \int \sin^{n-2} x dx,$
22.  $\int \cos^n x dx = \frac{\cos^{n-1} x \sin x}{n} + \frac{n-1}{n} \int \cos^{n-2} x dx,$
23.  $\int \tan^n x dx = \frac{\tan^{n-1} x}{n-1} - \int \tan^{n-2} x dx, \quad n \neq 1,$
24.  $\int \cot^n x dx = -\frac{\cot^{n-1} x}{n-1} - \int \cot^{n-2} x dx, \quad n \neq 1,$
25.  $\int \sec^n x dx = \frac{\tan x \sec^{n-1} x}{n-1} + \frac{n-2}{n-1} \int \sec^{n-2} x dx, \quad n \neq 1,$
26.  $\int \csc^n x dx = -\frac{\cot x \csc^{n-1} x}{n-1} + \frac{n-2}{n-1} \int \csc^{n-2} x dx, \quad n \neq 1,$
27.  $\int \sinh x dx = \cosh x,$
28.  $\int \cosh x dx = \sinh x,$
29.  $\int \tanh x dx = \ln |\cosh x|,$
30.  $\int \coth x dx = \ln |\sinh x|,$
31.  $\int \operatorname{sech} x dx = \arctan \sinh x,$
32.  $\int \operatorname{csch} x dx = \ln \left| \tanh \frac{x}{2} \right|,$
33.  $\int \sinh^2 x dx = \frac{1}{4} \sinh(2x) - \frac{1}{2} x,$
34.  $\int \cosh^2 x dx = \frac{1}{4} \sinh(2x) + \frac{1}{2} x,$
35.  $\int \operatorname{sech}^2 x dx = \tanh x,$
36.  $\int \operatorname{arcsinh} \frac{x}{a} dx = x \operatorname{arcsinh} \frac{x}{a} - \sqrt{x^2 + a^2}, \quad a > 0,$
37.  $\int \operatorname{arctanh} \frac{x}{a} dx = x \operatorname{arctanh} \frac{x}{a} + \frac{a}{2} \ln |a^2 - x^2|,$
38.  $\int \operatorname{arccosh} \frac{x}{a} dx = \begin{cases} x \operatorname{arccosh} \frac{x}{a} - \sqrt{x^2 + a^2}, & \text{if } \operatorname{arccosh} \frac{x}{a} > 0 \text{ and } a > 0, \\ x \operatorname{arccosh} \frac{x}{a} + \sqrt{x^2 + a^2}, & \text{if } \operatorname{arccosh} \frac{x}{a} < 0 \text{ and } a > 0, \end{cases}$
39.  $\int \frac{dx}{\sqrt{a^2 + x^2}} = \ln \left( x + \sqrt{a^2 + x^2} \right), \quad a > 0,$
40.  $\int \frac{dx}{a^2 + x^2} = \frac{1}{a} \arctan \frac{x}{a}, \quad a > 0,$
41.  $\int \sqrt{a^2 - x^2} dx = \frac{x}{2} \sqrt{a^2 - x^2} + \frac{a^2}{2} \arcsin \frac{x}{a}, \quad a > 0,$
42.  $\int (a^2 - x^2)^{3/2} dx = \frac{x}{8} (5a^2 - 2x^2) \sqrt{a^2 - x^2} + \frac{3a^4}{8} \arcsin \frac{x}{a}, \quad a > 0,$
43.  $\int \frac{dx}{\sqrt{a^2 - x^2}} = \arcsin \frac{x}{a}, \quad a > 0,$
44.  $\int \frac{dx}{a^2 - x^2} = \frac{1}{2a} \ln \left| \frac{a+x}{a-x} \right|,$
45.  $\int \frac{dx}{(a^2 - x^2)^{3/2}} = \frac{x}{a^2 \sqrt{a^2 - x^2}},$
46.  $\int \sqrt{a^2 \pm x^2} dx = \frac{x}{2} \sqrt{a^2 \pm x^2} \pm \frac{a^2}{2} \ln \left| x + \sqrt{a^2 \pm x^2} \right|,$
47.  $\int \frac{dx}{\sqrt{x^2 - a^2}} = \ln \left| x + \sqrt{x^2 - a^2} \right|, \quad a > 0,$
48.  $\int \frac{dx}{ax^2 + bx} = \frac{1}{a} \ln \left| \frac{x}{a+bx} \right|,$
49.  $\int x \sqrt{a+bx} dx = \frac{2(3bx-2a)(a+bx)^{3/2}}{15b^2},$
50.  $\int \frac{\sqrt{a+bx}}{x} dx = 2\sqrt{a+bx} + a \int \frac{1}{x\sqrt{a+bx}} dx,$
51.  $\int \frac{x}{\sqrt{a+bx}} dx = \frac{1}{\sqrt{2}} \ln \left| \frac{\sqrt{a+bx} - \sqrt{a}}{\sqrt{a+bx} + \sqrt{a}} \right|, \quad a > 0,$
52.  $\int \frac{\sqrt{a^2 - x^2}}{x} dx = \sqrt{a^2 - x^2} - a \ln \left| \frac{a + \sqrt{a^2 - x^2}}{x} \right|,$
53.  $\int x \sqrt{a^2 - x^2} dx = -\frac{1}{3} (a^2 - x^2)^{3/2},$
54.  $\int x^2 \sqrt{a^2 - x^2} dx = \frac{x}{8} (2x^2 - a^2) \sqrt{a^2 - x^2} + \frac{a^4}{8} \arcsin \frac{x}{a}, \quad a > 0,$
55.  $\int \frac{dx}{\sqrt{a^2 - x^2}} = -\frac{1}{a} \ln \left| \frac{a + \sqrt{a^2 - x^2}}{x} \right|,$
56.  $\int \frac{x dx}{\sqrt{a^2 - x^2}} = -\sqrt{a^2 - x^2},$
57.  $\int \frac{x^2 dx}{\sqrt{a^2 - x^2}} = -\frac{x}{2} \sqrt{a^2 - x^2} + \frac{a^2}{2} \arcsin \frac{x}{a}, \quad a > 0,$
58.  $\int \frac{\sqrt{a^2 + x^2}}{x} dx = \sqrt{a^2 + x^2} - a \ln \left| \frac{a + \sqrt{a^2 + x^2}}{x} \right|,$
59.  $\int \frac{\sqrt{x^2 - a^2}}{x} dx = \sqrt{x^2 - a^2} - a \arccos \frac{a}{|x|}, \quad a > 0,$
60.  $\int x \sqrt{x^2 \pm a^2} dx = \frac{1}{3} (x^2 \pm a^2)^{3/2},$
61.  $\int \frac{dx}{x \sqrt{x^2 + a^2}} = \frac{1}{a} \ln \left| \frac{x}{a + \sqrt{a^2 + x^2}} \right|,$

# Theoretical Computer Science Cheat Sheet

## Calculus Cont.

$$\begin{aligned}
 62. \int \frac{dx}{x\sqrt{x^2 - a^2}} &= \frac{1}{a} \arccos \frac{a}{|x|}, \quad a > 0, & 63. \int \frac{dx}{x^2\sqrt{x^2 \pm a^2}} &= \mp \frac{\sqrt{x^2 \pm a^2}}{a^2 x}, \\
 64. \int \frac{x dx}{\sqrt{x^2 \pm a^2}} &= \sqrt{x^2 \pm a^2}, & 65. \int \frac{\sqrt{x^2 \pm a^2}}{x^4} dx &= \mp \frac{(x^2 + a^2)^{3/2}}{3a^2 x^3}, \\
 66. \int \frac{dx}{ax^2 + bx + c} &= \begin{cases} \frac{1}{\sqrt{b^2 - 4ac}} \ln \left| \frac{2ax + b - \sqrt{b^2 - 4ac}}{2ax + b + \sqrt{b^2 - 4ac}} \right|, & \text{if } b^2 > 4ac, \\ \frac{2}{\sqrt{4ac - b^2}} \arctan \frac{2ax + b}{\sqrt{4ac - b^2}}, & \text{if } b^2 < 4ac, \end{cases} \\
 67. \int \frac{dx}{\sqrt{ax^2 + bx + c}} &= \begin{cases} \frac{1}{\sqrt{a}} \ln \left| 2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c} \right|, & \text{if } a > 0, \\ \frac{1}{\sqrt{-a}} \arcsin \frac{-2ax - b}{\sqrt{b^2 - 4ac}}, & \text{if } a < 0, \end{cases} \\
 68. \int \sqrt{ax^2 + bx + c} dx &= \frac{2ax + b}{4a} \sqrt{ax^2 + bx + c} + \frac{4ac - b^2}{8a} \int \frac{dx}{\sqrt{ax^2 + bx + c}}, \\
 69. \int \frac{x dx}{\sqrt{ax^2 + bx + c}} &= \frac{\sqrt{ax^2 + bx + c}}{a} - \frac{b}{2a} \int \frac{dx}{\sqrt{ax^2 + bx + c}}, \\
 70. \int \frac{dx}{x\sqrt{ax^2 + bx + c}} &= \begin{cases} \frac{-1}{\sqrt{c}} \ln \left| \frac{2\sqrt{c}\sqrt{ax^2 + bx + c} + bx + 2c}{x} \right|, & \text{if } c > 0, \\ \frac{1}{\sqrt{-c}} \arcsin \frac{bx + 2c}{|x|\sqrt{b^2 - 4ac}}, & \text{if } c < 0, \end{cases} \\
 71. \int x^3 \sqrt{x^2 + a^2} dx &= \left(\frac{1}{3}x^2 - \frac{2}{15}a^2\right)(x^2 + a^2)^{3/2}, \\
 72. \int x^n \sin(ax) dx &= -\frac{1}{a}x^n \cos(ax) + \frac{n}{a} \int x^{n-1} \cos(ax) dx, \\
 73. \int x^n \cos(ax) dx &= \frac{1}{a}x^n \sin(ax) - \frac{n}{a} \int x^{n-1} \sin(ax) dx, \\
 74. \int x^n e^{ax} dx &= \frac{x^n e^{ax}}{a} - \frac{n}{a} \int x^{n-1} e^{ax} dx, \\
 75. \int x^n \ln(ax) dx &= x^{n+1} \left( \frac{\ln(ax)}{n+1} - \frac{1}{(n+1)^2} \right), \\
 76. \int x^n (\ln ax)^m dx &= \frac{x^{n+1}}{n+1} (\ln ax)^m - \frac{m}{n+1} \int x^n (\ln ax)^{m-1} dx.
 \end{aligned}$$

## Finite Calculus

Difference, shift operators:

$$\Delta f(x) = f(x+1) - f(x),$$

$$\mathbf{E} f(x) = f(x+1).$$

Fundamental Theorem:

$$f(x) = \Delta F(x) \Leftrightarrow \sum f(x) \delta x = F(x) + C.$$

$$\sum_a^b f(x) \delta x = \sum_{i=a}^{b-1} f(i).$$

Differences:

$$\Delta(cu) = c\Delta u, \quad \Delta(u+v) = \Delta u + \Delta v,$$

$$\Delta(uv) = u\Delta v + \mathbf{E} v \Delta u,$$

$$\Delta(x^n) = nx^{n-1},$$

$$\Delta(H_x) = x^{-1}, \quad \Delta(2^x) = 2^x,$$

$$\Delta(c^x) = (c-1)c^x, \quad \Delta\binom{x}{m} = \binom{x}{m-1}.$$

Sums:

$$\sum cu \delta x = c \sum u \delta x,$$

$$\sum(u+v) \delta x = \sum u \delta x + \sum v \delta x,$$

$$\sum u \Delta v \delta x = uv - \sum \mathbf{E} v \Delta u \delta x,$$

$$\sum x^n \delta x = \frac{x^{n+1}}{n+1}, \quad \sum x^{-1} \delta x = H_x,$$

$$\sum c^x \delta x = \frac{c^x}{c-1}, \quad \sum \binom{x}{m} \delta x = \binom{x}{m+1}.$$

Falling Factorial Powers:

$$x^{\underline{n}} = x(x-1) \cdots (x-n+1), \quad n > 0,$$

$$x^{\underline{0}} = 1,$$

$$x^{\underline{n}} = \frac{1}{(x+1) \cdots (x+|n|)}, \quad n < 0,$$

$$x^{\underline{n+m}} = x^{\underline{n}}(x-m)^{\underline{n}}.$$

Rising Factorial Powers:

$$x^{\overline{n}} = x(x+1) \cdots (x+n-1), \quad n > 0,$$

$$x^{\overline{0}} = 1,$$

$$x^{\overline{n}} = \frac{1}{(x-1) \cdots (x-|n|)}, \quad n < 0,$$

$$x^{\overline{n+m}} = x^{\overline{n}}(x+m)^{\overline{n}}.$$

Conversion:

$$x^{\underline{n}} = (-1)^n (-x)^{\overline{n}} = (x-n+1)^{\overline{n}}$$

$$= 1/(x+1)^{\overline{-n}},$$

$$x^{\overline{n}} = (-1)^n (-x)^{\underline{n}} = (x+n-1)^{\underline{n}}$$

$$= 1/(x-1)^{\underline{-n}},$$

$$x^n = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^{\underline{k}} = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} (-1)^{n-k} x^{\overline{k}},$$

$$x^{\underline{n}} = \sum_{k=1}^n \left[ \begin{matrix} n \\ k \end{matrix} \right] (-1)^{n-k} x^k,$$

$$x^{\overline{n}} = \sum_{k=1}^n \left[ \begin{matrix} n \\ k \end{matrix} \right] x^k.$$

|                      |                                                                                                           |                       |                                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------------------------|-----------------------|------------------------------------------------------------------------------------------------------|
| $x^1 =$              | $x^{\underline{1}}$                                                                                       | $=$                   | $x^{\overline{1}}$                                                                                   |
| $x^2 =$              | $x^{\underline{2}} + x^{\underline{1}}$                                                                   | $=$                   | $x^{\overline{2}} - x^{\overline{1}}$                                                                |
| $x^3 =$              | $x^{\underline{3}} + 3x^{\underline{2}} + x^{\underline{1}}$                                              | $=$                   | $x^{\overline{3}} - 3x^{\overline{2}} + x^{\overline{1}}$                                            |
| $x^4 =$              | $x^{\underline{4}} + 6x^{\underline{3}} + 7x^{\underline{2}} + x^{\underline{1}}$                         | $=$                   | $x^{\overline{4}} - 6x^{\overline{3}} + 7x^{\overline{2}} - x^{\overline{1}}$                        |
| $x^5 =$              | $x^{\underline{5}} + 15x^{\underline{4}} + 25x^{\underline{3}} + 10x^{\underline{2}} + x^{\underline{1}}$ | $=$                   | $x^{\overline{5}} - 15x^{\overline{4}} + 25x^{\overline{3}} - 10x^{\overline{2}} + x^{\overline{1}}$ |
| $x^{\overline{1}} =$ | $x^1$                                                                                                     | $x^{\underline{1}} =$ | $x^1$                                                                                                |
| $x^{\overline{2}} =$ | $x^2 + x^1$                                                                                               | $x^{\underline{2}} =$ | $x^2 - x^1$                                                                                          |
| $x^{\overline{3}} =$ | $x^3 + 3x^2 + 2x^1$                                                                                       | $x^{\underline{3}} =$ | $x^3 - 3x^2 + 2x^1$                                                                                  |
| $x^{\overline{4}} =$ | $x^4 + 6x^3 + 11x^2 + 6x^1$                                                                               | $x^{\underline{4}} =$ | $x^4 - 6x^3 + 11x^2 - 6x^1$                                                                          |
| $x^{\overline{5}} =$ | $x^5 + 10x^4 + 35x^3 + 50x^2 + 24x^1$                                                                     | $x^{\underline{5}} =$ | $x^5 - 10x^4 + 35x^3 - 50x^2 + 24x^1$                                                                |

# Theoretical Computer Science Cheat Sheet

## Series

Taylor's series:

$$f(x) = f(a) + (x-a)f'(a) + \frac{(x-a)^2}{2}f''(a) + \dots = \sum_{i=0}^{\infty} \frac{(x-a)^i}{i!} f^{(i)}(a).$$

Expansions:

$$\begin{aligned} \frac{1}{1-x} &= 1 + x + x^2 + x^3 + x^4 + \dots = \sum_{i=0}^{\infty} x^i, \\ \frac{1}{1-cx} &= 1 + cx + c^2x^2 + c^3x^3 + \dots = \sum_{i=0}^{\infty} c^i x^i, \\ \frac{1}{1-x^n} &= 1 + x^n + x^{2n} + x^{3n} + \dots = \sum_{i=0}^{\infty} x^{ni}, \\ \frac{x}{(1-x)^2} &= x + 2x^2 + 3x^3 + 4x^4 + \dots = \sum_{i=0}^{\infty} ix^i, \\ \sum_{k=0}^n \binom{n}{k} \frac{k!z^k}{(1-z)^{k+1}} &= x + 2^n x^2 + 3^n x^3 + 4^n x^4 + \dots = \sum_{i=0}^{\infty} i^n x^i, \\ e^x &= 1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3 + \dots = \sum_{i=0}^{\infty} \frac{x^i}{i!}, \\ \ln(1+x) &= x - \frac{1}{2}x^2 + \frac{1}{3}x^3 - \frac{1}{4}x^4 + \dots = \sum_{i=1}^{\infty} (-1)^{i+1} \frac{x^i}{i}, \\ \ln \frac{1}{1-x} &= x + \frac{1}{2}x^2 + \frac{1}{3}x^3 + \frac{1}{4}x^4 + \dots = \sum_{i=1}^{\infty} \frac{x^i}{i}, \\ \sin x &= x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 - \frac{1}{7!}x^7 + \dots = \sum_{i=0}^{\infty} (-1)^i \frac{x^{2i+1}}{(2i+1)!}, \\ \cos x &= 1 - \frac{1}{2!}x^2 + \frac{1}{4!}x^4 - \frac{1}{6!}x^6 + \dots = \sum_{i=0}^{\infty} (-1)^i \frac{x^{2i}}{(2i)!}, \\ \tan^{-1} x &= x - \frac{1}{3}x^3 + \frac{1}{5}x^5 - \frac{1}{7}x^7 + \dots = \sum_{i=0}^{\infty} (-1)^i \frac{x^{2i+1}}{(2i+1)}, \\ (1+x)^n &= 1 + nx + \frac{n(n-1)}{2}x^2 + \dots = \sum_{i=0}^{\infty} \binom{n}{i} x^i, \\ \frac{1}{(1-x)^{n+1}} &= 1 + (n+1)x + \binom{n+2}{2}x^2 + \dots = \sum_{i=0}^{\infty} \binom{i+n}{i} x^i, \\ \frac{x}{e^x - 1} &= 1 - \frac{1}{2}x + \frac{1}{12}x^2 - \frac{1}{720}x^4 + \dots = \sum_{i=0}^{\infty} \frac{B_i x^i}{i!}, \\ \frac{1}{2x}(1 - \sqrt{1-4x}) &= 1 + x + 2x^2 + 5x^3 + \dots = \sum_{i=0}^{\infty} \frac{1}{i+1} \binom{2i}{i} x^i, \\ \frac{1}{\sqrt{1-4x}} &= 1 + 2x + 6x^2 + 20x^3 + \dots = \sum_{i=0}^{\infty} \binom{2i}{i} x^i, \\ \frac{1}{\sqrt{1-4x}} \left( \frac{1 - \sqrt{1-4x}}{2x} \right)^n &= 1 + (2+n)x + \binom{4+n}{2}x^2 + \dots = \sum_{i=0}^{\infty} \binom{2i+n}{i} x^i, \\ \frac{1}{1-x} \ln \frac{1}{1-x} &= x + \frac{3}{2}x^2 + \frac{11}{6}x^3 + \frac{25}{12}x^4 + \dots = \sum_{i=1}^{\infty} H_i x^i, \\ \frac{1}{2} \left( \ln \frac{1}{1-x} \right)^2 &= \frac{1}{2}x^2 + \frac{3}{4}x^3 + \frac{11}{24}x^4 + \dots = \sum_{i=2}^{\infty} \frac{H_{i-1} x^i}{i}, \\ \frac{x}{1-x-x^2} &= x + x^2 + 2x^3 + 3x^4 + \dots = \sum_{i=0}^{\infty} F_i x^i, \\ \frac{F_n x}{1 - (F_{n-1} + F_{n+1})x - (-1)^n x^2} &= F_n x + F_{2n} x^2 + F_{3n} x^3 + \dots = \sum_{i=0}^{\infty} F_{ni} x^i. \end{aligned}$$

Ordinary power series:

$$A(x) = \sum_{i=0}^{\infty} a_i x^i.$$

Exponential power series:

$$A(x) = \sum_{i=0}^{\infty} a_i \frac{x^i}{i!}.$$

Dirichlet power series:

$$A(x) = \sum_{i=1}^{\infty} \frac{a_i}{i^x}.$$

Binomial theorem:

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k.$$

Difference of like powers:

$$x^n - y^n = (x-y) \sum_{k=0}^{n-1} x^{n-1-k} y^k.$$

For ordinary power series:

$$\alpha A(x) + \beta B(x) = \sum_{i=0}^{\infty} (\alpha a_i + \beta b_i) x^i,$$

$$x^k A(x) = \sum_{i=k}^{\infty} a_{i-k} x^i,$$

$$\frac{A(x) - \sum_{i=0}^{k-1} a_i x^i}{x^k} = \sum_{i=0}^{\infty} a_{i+k} x^i,$$

$$A(cx) = \sum_{i=0}^{\infty} c^i a_i x^i,$$

$$A'(x) = \sum_{i=0}^{\infty} (i+1) a_{i+1} x^i,$$

$$x A'(x) = \sum_{i=1}^{\infty} i a_i x^i,$$

$$\int A(x) dx = \sum_{i=1}^{\infty} \frac{a_{i-1}}{i} x^i,$$

$$\frac{A(x) + A(-x)}{2} = \sum_{i=0}^{\infty} a_{2i} x^{2i},$$

$$\frac{A(x) - A(-x)}{2} = \sum_{i=0}^{\infty} a_{2i+1} x^{2i+1}.$$

Summation: If  $b_i = \sum_{j=0}^i a_j$  then

$$B(x) = \frac{1}{1-x} A(x).$$

Convolution:

$$A(x)B(x) = \sum_{i=0}^{\infty} \left( \sum_{j=0}^i a_j b_{i-j} \right) x^i.$$

God made the natural numbers;  
all the rest is the work of man.  
– Leopold Kronecker

# Theoretical Computer Science Cheat Sheet

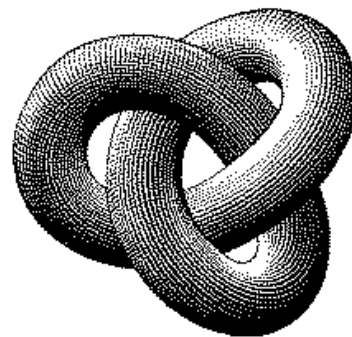
## Series

Expansions:

$$\begin{aligned} \frac{1}{(1-x)^{n+1}} \ln \frac{1}{1-x} &= \sum_{i=0}^{\infty} (H_{n+i} - H_n) \binom{n+i}{i} x^i, \\ x^{\overline{n}} &= \sum_{i=0}^{\infty} \left[ \begin{matrix} n \\ i \end{matrix} \right] x^i, \\ \left( \ln \frac{1}{1-x} \right)^n &= \sum_{i=0}^{\infty} \left[ \begin{matrix} i \\ n \end{matrix} \right] \frac{n! x^i}{i!}, \\ \tan x &= \sum_{i=1}^{\infty} (-1)^{i-1} \frac{2^{2i} (2^{2i} - 1) B_{2i} x^{2i-1}}{(2i)!}, \\ \frac{1}{\zeta(x)} &= \sum_{i=1}^{\infty} \frac{\mu(i)}{i^x}, \\ \zeta(x) &= \prod_p \frac{1}{1 - p^{-x}}, \\ \zeta^2(x) &= \sum_{i=1}^{\infty} \frac{d(i)}{x^i} \quad \text{where } d(n) = \sum_{d|n} 1, \\ \zeta(x) \zeta(x-1) &= \sum_{i=1}^{\infty} \frac{S(i)}{x^i} \quad \text{where } S(n) = \sum_{d|n} d, \\ \zeta(2n) &= \frac{2^{2n-1} |B_{2n}|}{(2n)!} \pi^{2n}, \quad n \in \mathbb{N}, \\ \frac{x}{\sin x} &= \sum_{i=0}^{\infty} (-1)^{i-1} \frac{(4^i - 2) B_{2i} x^{2i}}{(2i)!}, \\ \left( \frac{1 - \sqrt{1-4x}}{2x} \right)^n &= \sum_{i=0}^{\infty} \frac{n(2i+n-1)!}{i!(n+i)!} x^i, \\ e^x \sin x &= \sum_{i=1}^{\infty} \frac{2^{i/2} \sin \frac{i\pi}{4}}{i!} x^i, \\ \sqrt{\frac{1 - \sqrt{1-x}}{x}} &= \sum_{i=0}^{\infty} \frac{(4i)!}{16^i \sqrt{2} (2i)!(2i+1)!} x^i, \\ \left( \frac{\arcsin x}{x} \right)^2 &= \sum_{i=0}^{\infty} \frac{4^i i!^2}{(i+1)(2i+1)!} x^{2i}. \end{aligned}$$

$$\begin{aligned} \left( \frac{1}{x} \right)^{\overline{-n}} &= \sum_{i=0}^{\infty} \left\{ \begin{matrix} i \\ n \end{matrix} \right\} x^i, \\ (e^x - 1)^n &= \sum_{i=0}^{\infty} \left\{ \begin{matrix} i \\ n \end{matrix} \right\} \frac{n! x^i}{i!}, \\ x \cot x &= \sum_{i=0}^{\infty} \frac{(-4)^i B_{2i} x^{2i}}{(2i)!}, \\ \zeta(x) &= \sum_{i=1}^{\infty} \frac{1}{i^x}, \\ \frac{\zeta(x-1)}{\zeta(x)} &= \sum_{i=1}^{\infty} \frac{\phi(i)}{i^x}, \end{aligned}$$

## Escher's Knot



## Stieltjes Integration

If  $G$  is continuous in the interval  $[a, b]$  and  $F$  is nondecreasing then

$$\int_a^b G(x) dF(x)$$

exists. If  $a \leq b \leq c$  then

$$\int_a^c G(x) dF(x) = \int_a^b G(x) dF(x) + \int_b^c G(x) dF(x).$$

If the integrals involved exist

$$\int_a^b (G(x) + H(x)) dF(x) = \int_a^b G(x) dF(x) + \int_a^b H(x) dF(x),$$

$$\int_a^b G(x) d(F(x) + H(x)) = \int_a^b G(x) dF(x) + \int_a^b G(x) dH(x),$$

$$\int_a^b c \cdot G(x) dF(x) = \int_a^b G(x) d(c \cdot F(x)) = c \int_a^b G(x) dF(x),$$

$$\int_a^b G(x) dF(x) = G(b)F(b) - G(a)F(a) - \int_a^b F(x) dG(x).$$

If the integrals involved exist, and  $F$  possesses a derivative  $F'$  at every point in  $[a, b]$  then

$$\int_a^b G(x) dF(x) = \int_a^b G(x) F'(x) dx.$$

## Cramer's Rule

If we have equations:

$$a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n = b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n = b_2$$

$$\vdots \quad \quad \quad \vdots$$

$$a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,n}x_n = b_n$$

Let  $A = (a_{i,j})$  and  $B$  be the column matrix  $(b_i)$ . Then there is a unique solution iff  $\det A \neq 0$ . Let  $A_i$  be  $A$  with column  $i$  replaced by  $B$ . Then

$$x_i = \frac{\det A_i}{\det A}.$$

Improvement makes strait roads, but the crooked roads without Improvement, are roads of Genius.  
– William Blake (The Marriage of Heaven and Hell)

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 00 | 47 | 18 | 76 | 29 | 93 | 85 | 34 | 61 | 52 |
| 86 | 11 | 57 | 28 | 70 | 39 | 94 | 45 | 02 | 63 |
| 95 | 80 | 22 | 67 | 38 | 71 | 49 | 56 | 13 | 04 |
| 59 | 96 | 81 | 33 | 07 | 48 | 72 | 60 | 24 | 15 |
| 73 | 69 | 90 | 82 | 44 | 17 | 58 | 01 | 35 | 26 |
| 68 | 74 | 09 | 91 | 83 | 55 | 27 | 12 | 46 | 30 |
| 37 | 08 | 75 | 19 | 92 | 84 | 66 | 23 | 50 | 41 |
| 14 | 25 | 36 | 40 | 51 | 62 | 03 | 77 | 88 | 99 |
| 21 | 32 | 43 | 54 | 65 | 06 | 10 | 89 | 97 | 78 |
| 42 | 53 | 64 | 05 | 16 | 20 | 31 | 98 | 79 | 87 |

The Fibonacci number system:  
Every integer  $n$  has a unique representation

$$n = F_{k_1} + F_{k_2} + \cdots + F_{k_m},$$

where  $k_i \geq k_{i+1} + 2$  for all  $i$ ,  
 $1 \leq i < m$  and  $k_m \geq 2$ .

## Fibonacci Numbers

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

Definitions:

$$F_i = F_{i-1} + F_{i-2}, \quad F_0 = F_1 = 1,$$

$$F_{-i} = (-1)^{i-1} F_i,$$

$$F_i = \frac{1}{\sqrt{5}} \left( \phi^i - \hat{\phi}^i \right),$$

Cassini's identity: for  $i > 0$ :

$$F_{i+1}F_{i-1} - F_i^2 = (-1)^i.$$

Additive rule:

$$F_{n+k} = F_k F_{n+1} + F_{k-1} F_n,$$

$$F_{2n} = F_n F_{n+1} + F_{n-1} F_n.$$

Calculation by matrices:

$$\begin{pmatrix} F_{n-2} & F_{n-1} \\ F_{n-1} & F_n \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}^n.$$

# Contest Snippets — Index

|                                                   |      |                                           |       |
|---------------------------------------------------|------|-------------------------------------------|-------|
| 1. dijkstra                                       | p.23 | 51. Dynamic_Segment_Tree                  | p.66  |
| 2. include                                        | p.23 | 52. small_segment_tree                    | p.67  |
| 3. sieve_algo                                     | p.24 | 53. Segment_Tree_Merging                  | p.68  |
| 4. fast_power_mod                                 | p.25 | 54. xor_segment_tree                      | p.71  |
| 5. fast_power                                     | p.25 | 55. xor_segment_tree_Non-commutative_case | p.72  |
| 6. mod_op                                         | p.25 | 56. segment_tree_beats_min_add            | p.73  |
| 7. nck_with_mod                                   | p.26 | 57. segment_tree_beats_max_add            | p.76  |
| 8. nck_without_mod                                | p.26 | 58. segment_tree_beats_mod,div,sqrt       | p.78  |
| 9. prefix_sum_2d                                  | p.27 | 59. Fenwick_Range_Update                  | p.81  |
| 10. findSubarrayCount                             | p.27 | 60. Persistent_Trie                       | p.82  |
| 11. DSU_                                          | p.28 | 61. Persistent_Binary_Trie                | p.84  |
| 12. mint_mod                                      | p.28 | 62. Persistent_Binary_Trie_ver            | p.86  |
| 13. ordered_set_call                              | p.30 | 63. DSU_with_rollback                     | p.89  |
| 14. prime_div_in)map                              | p.30 | 64. Kruskal_Tree                          | p.90  |
| 15. prefix_and_or_xor                             | p.30 | 65. Persistent_DSU                        | p.92  |
| 16. Hashing                                       | p.31 | 66. Weighted_DSU                          | p.93  |
| 17. Trie                                          | p.32 | 67. DSU_range_merge                       | p.94  |
| 18. Binary_Trie                                   | p.33 | 68. mo_with_rollback                      | p.95  |
| 19. Z_Function                                    | p.34 | 69. mo_with_update                        | p.97  |
| 20. number_of_prime_factors_of_a_number           | p.34 | 70. Suffix_Array                          | p.99  |
| 21. xor_from_1_to_n                               | p.35 | 71. Suffix_Automaton                      | p.101 |
| 22. sparse_table                                  | p.35 |                                           |       |
| 23. mo_in_array                                   | p.36 |                                           |       |
| 24. segment_tree                                  | p.37 |                                           |       |
| 25. sqrt_deco                                     | p.38 |                                           |       |
| 26. lazy_segment_(q,p)                            | p.38 |                                           |       |
| 27. lazy_segment_(q,q)                            | p.39 |                                           |       |
| 28. XOR_Basis                                     | p.40 |                                           |       |
| 29. segment_tree_2d                               | p.42 |                                           |       |
| 30. Li_Chao_Tree                                  | p.44 |                                           |       |
| 31. LCA                                           | p.45 |                                           |       |
| 32. NTT                                           | p.46 |                                           |       |
| 33. Matrix_def                                    | p.47 |                                           |       |
| 34. Fenwick_tree                                  | p.48 |                                           |       |
| 35. Lucas_Theorem                                 | p.49 |                                           |       |
| 36. Aho_Corasick                                  | p.49 |                                           |       |
| 37. KMP and go state                              | p.52 |                                           |       |
| 38. Merge_Sort_Tree                               | p.52 |                                           |       |
| 39. mobius                                        | p.53 |                                           |       |
| 40. Median_query                                  | p.54 |                                           |       |
| 41. Median_Sliding                                | p.55 |                                           |       |
| 42. 2-sat sheet                                   | p.56 |                                           |       |
| 43. Tarjan for SCC                                | p.57 |                                           |       |
| 44. Tarjan for Bridges                            | p.57 |                                           |       |
| 45. Tarjan for Articulation Points                | p.58 |                                           |       |
| 46. Centroid_Decomposition_eg                     | p.59 |                                           |       |
| 47. Heavy_Light_Decomposition_eg                  | p.60 |                                           |       |
| 48. stirling_2d_type                              | p.61 |                                           |       |
| 49. persistent_segment_tree                       | p.63 |                                           |       |
| 50. Persistent_Segment_Tree_with_Lazy_Propagation | p.64 |                                           |       |

## 1. dijkstra

```
ll MAXSS = 2e5;
vector<bool> vis(MAXSS+50);
vector<vector<pair<ll, ll>>> node(MAXSS+50);
vlll dis(MAXSS+50);
void dij(ll st) {
    priority_queue < pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> q;
    q.push({ 0, st });
    while (!q.empty()) {
        ll cost = q.top().first;
        ll nod = q.top().second;
        q.pop();
        if (vis[nod]) continue;
        vis[nod] = true;
        dis[nod] = cost;
        for (auto [a, b] : node[nod]) {
            if (!vis[a]) {
                q.push({ cost + b, a });
            }
        }
    }
}
```

---

## 2. include

```
#include <bits/stdc++.h>
using namespace std;
#define FOR(i,z,x) for(ll i = z ; i<x ; i++)
#define FOR1(i,z,x) for(ll i = z ; i<=x ; i++)
#define cin(v) for (auto& i : v) cin >> i
#define cout(v) for (auto i : v) cout << i << " "
#define endl '\n'
#define ll long long
#define vi vector<int>
#define vll vector<ll>
#define pb push_back
#define all(v1) v1.begin(), v1.end()
#define rall(v1) v1.rbegin(), v1.rend()

//-----deb-----
#ifdef ONLINE_JUDGE
#define deb(x) cerr<<#x<<" ";_print(x);cerr<<endl;
#else
#define deb(x)
#endif
void _print(ll t){cerr<<t;}
void _print(int t){cerr<<t;}
void _print(string t){cerr<<t;}
void _print(char t){cerr<<t;}
void _print(long double t){cerr<<t;}
void _print(double t){cerr<<t;}
void _print(unsigned long long t){cerr<<t;}
template<class T,class V>void _print(pair<T,V> p);
template<class T>void _print(vector<T> v);
template<class T>void _print(set<T> v);
template<class T,class V>void _print(map<T,V> v);
template<class T>void _print(multiset<T> v);
```

```

template<class T>void _print(stack<T> s);
template<class T>void _print(queue<T> q);
template<class T>void _print(deque<T> dq);
template<class T, class V>void _print(pair<T, V> p){cerr<<"{";_print(p.first);cerr<<" ";_print(p.second);cerr<<"}";}
template<class T>void _print(vector<T> v){cerr<<"[";for(T i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T>void _print(set<T> v){cerr<<"[";for(T i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T>void _print(multiset<T> v){cerr<<"[";for(T i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T, class V>void _print(map<T, V> v){cerr<<"[";for(auto i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T>void _print(stack<T> s){vector<T> v;while(!s.empty()){v.push_back(s.top());s.pop();}for(T i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T>void _print(queue<T> q){vector<T> v;while(!q.empty()){v.push_back(q.front());q.pop();}for(T i:v){_print(i);cerr<<" ";}cerr<<"]";}
template<class T>void _print(deque<T> dq){cerr<<"[";for(T i:dq){_print(i);cerr<<" ";}cerr<<"]";}

//-----

void Hello_World()
{

}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    // #ifndef ONLINE_JUDGE
    // freopen("input.txt", "r", stdin);
    // freopen("output.txt", "w", stdout);
    // #endif

    //////////////////////////////////////
    ll t = 1;
    cin >> t;
    FOR(I, 0, t)
        Hello_World();
    //////////////////////////////////////
    return 0;
}

```

---

### 3. sieve\_algo

```

ll nn = ...;
vill v(nn,0);
vill pri;
void sieve(){
    for(ll i = 2 ; i * i <= nn ; i++){
        if(!v[i]){
            for(ll j = i*i; j<=nn ; j+=i){
                v[j] = i;
            }
            FOR1(i,2,nn) if(!v[i]) pri.pb(i);
        }
    }
}

```

---

## 4. fast\_power\_mod

```
ll mod = 1e9+7;
ll pow(ll num, ll po){
    ll fres = 1;
    while(po > 0){
        if(po & 1)
            fres = fres * num % mod;
        po >>= 1;
        num = num * num % mod;
    }
    return fres;
}
```

---

## 5. fast\_power

```
ll fastPower(int a, int b) {
    if(b==0) return 1;
    ll ans = fastPower(a,b/2);
    ans=ans*ans;
    if(b&1)return a*ans;
    return ans;
}
```

---

## 6. mod\_op

```
ll mod = 1e9+7;
ll expo(ll a, ll b, ll m = mod)
{
    ll res = 1;
    while (b > 0)
    {
        if (b & 1)
            res = (res * a) % m;
        a = (a * a) % m;
        b = b >> 1;
    }
    return res;
}

ll modinv(ll a, ll m = mod) {
    return expo(a, m - 2, m);
}

ll add(ll a, ll b, ll m = mod)
{
    a = a % m;
    b = b % m;
    return (((a + b) % m) + m) % m;
}

ll mul(ll a, ll b, ll m = mod)
```

```

{
    a = a % m;
    b = b % m;
    return (((a * b) % m) + m) % m;
}

ll sub(ll a, ll b, ll m = mod)
{
    a = a % m;
    b = b % m;
    return (((a - b) % m) + m) % m;
}

ll div(ll a, ll b, ll m = mod)
{
    a = a % m;
    b = b % m;
    return (mul(a, modinv(b, m), m) + m) % m;
}

```

---

## 7. nck\_with\_mod

```

ll mod = 1e9+7;
ll pw(ll num, ll po){
    ll fres = 1;
    while(po > 0){
        if(po & 1)
            fres = fres * num % mod;
        po >>= 1;
        num = num * num % mod;
    }
    return fres;
}

ll fact[N], fact_inv[N];
inline ll nck(ll n, ll k){
    return fact[n] * (fact_inv[k] * fact_inv[n - k] % mod) % mod;
}

void cf(){
    fact[0] = 1;
    for(ll i = 1; i < N; i++) fact[i] = i * fact[i - 1] % mod;
    fact_inv[N - 1] = pw(fact[N - 1], mod - 2);
    for(ll i = N - 2; i >= 0; i--) fact_inv[i] = fact_inv[i + 1] * (i + 1) % mod;
}

```

---

## 8. nck\_without\_mod

```

ll nck(ll n, ll k){
    ll ans = 1;
    FOR(i, 0, k){
        ans *= n - i;
        ans /= i + 1;
    }
}

```

```

    }■
    return ans;■
}■

```

---

## 9. prefix\_sum\_2d

```

class presum2d {■
public:■
    ll n_, m_;■
    vector<vll> v_;■
    vector<vll> pre;■
    presum2d(ll a, ll b, vector<vll> c) {■
        n_ = a; m_ = b; v_ = c;■
        pre.resize(a + 1, vll(b + 1));■
        makeit();■
    }■
    void makeit() {■
        FOR1(i, 1, n_) {■
            FOR1(j, 1, m_) {■
                pre[i][j] = pre[i - 1][j] + pre[i][j - 1] + v_[i - 1][j - 1] - pre[i - 1][j - 1];■
            }■
        }■
    }■
    ll sums(ll x, ll y, ll a, ll b) {■
        return pre[a][b] - pre[a][y-1] - pre[x-1][b] + pre[x-1][y-1];■
    }■
};

```

---

## 10. findSubarrayCount

```

ll fsac(vll &arr, int n, int k)■
{■
    ll st = 0, en = 0, p = 1, co = 0, res = 0;■
    while (en < n) {■
        p *= (arr[en]);■
        if(p > k) {■
            while(st <= en && p > k) {■
                p /= arr[st];■
                st++;■
            }■
        }■
        if(p == k) {■
            co = 0;■
            while(en + 1 < n && arr[en + 1] == 1) {■
                co++;■
                en++;■
            }■
            res += (co + 1);■
            while(st <= en && arr[st] == 1 && k!=1) {■
                res += (co + 1);■
                st++;■
            }■
            p /= arr[st];■
        }■
    }■
}■

```

```

        st++;
    }
    en++;
}
return res;
}

```

---

## 11. DSU\_

```

struct DSU{
    ll sz;
    vll parent, group;
}
DSU(ll n){
    sz = n+1;
    parent.resize(sz+1);
    group.resize(sz+1, 1);
    FOR1(i, 1, n) parent[i] = i;
}
ll find(ll node){
    if(parent[node]==node) return node;
    return find(parent[node]);
}
void merge(ll u, ll v){
    ll le1 = find(u);
    ll le2 = find(v);
    if(le1==le2) return;
    if(group[le1]>group[le2]) swap(le1, le2);
    parent[le1] = le2;
    group[le2] += group[le1];
}
bool samegroup(ll v, ll u){
    return find(v)==find(u);
}
ll getsize(ll v){
    return group[find(v)];
}
};

```

---

## 12. mint\_mod

```

const ll MOD = 1e9+7;
template <int32_t MOD, int32_t RT>
struct modular{
{
    static const int32_t mod = MOD;
    static constexpr modular rt() { return RT; }
    int32_t v;
    explicit operator int32_t() const{
        return v;
    }
    modular() : v(0) {}
}

```

```

modular(long long _v)■
{■
    v = int32_t((-MOD < _v && _v < MOD) ? _v : _v % MOD);■
    if (v < 0)■
        v += MOD;■
}■
friend bool operator!=(const modular &a, const modular &b) { return !(a == b); }■
bool operator==(const modular &o) const { return v == o.v; }■
friend bool operator<(const modular &a, const modular &b) { return a.v < b.v; }■
friend bool operator<=(const modular &a, const modular &b) { return a.v <= b.v; }■
friend bool operator>(const modular &a, const modular &b) { return a.v > b.v; }■
friend bool operator>=(const modular &a, const modular &b) { return a.v >= b.v; }■
■
friend ostream &operator<<(ostream &os, modular a)■
{■
    os << int32_t(a);■
    return os;■
}■
■
friend istream &operator>>(istream &is, modular &a)■
{■
    long long x;■
    is >> x;■
    a = modular(x);■
    return is;■
}■
modular &operator*=(const modular &o)■
{■
    v = int32_t((long long)v * o.v % MOD);■
    return *this;■
}■
modular &operator+=(const modular &o)■
{■
    if ((v += o.v) >= MOD)■
        v -= MOD;■
    return *this;■
}■
modular &operator-=(const modular &o)■
{■
    if ((v -= o.v) < 0)■
        v += MOD;■
    return *this;■
}■
modular &operator/=(const modular &o) { return (*this) *= inv(o); }■
friend modular pow(modular a, long long p)■
{■
    modular result = 1;■
    assert(p >= 0);■
    for (; p; p /= 2, a *= a)■
        if (p & 1)■
            result *= a;■
    return result;■
}■
friend modular inv(const modular &a)■
{■
    assert(a.v != 0);■
    return pow(a, MOD - 2);■
}■
modular operator-() const { return modular(-v); }■
■
modular &operator--() { return *this -= 1; }■
modular &operator++() { return *this += 1; }■
friend modular operator-(modular a, const modular &b) { return a -= b; }■
friend modular operator+(modular a, const modular &b) { return a += b; }■
friend modular operator/(modular a, const modular &b) { return a /= b; }■

```

```

        friend modular operator*(modular a, const modular &b) { return a *= b; }
    };
    using mint = modular<MOD, 5>;

```

---

### 13. ordered\_set\_call

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<int, null_type, less<int>, rb_tree_tag, tree_order_statistics_node_update>

```

---

### 14. prime\_div\_in)map

```

const ll N = 2e5+50;
vector<map<ll,ll>> d(N);
void di(){
    d[1][1]++;
    FOR1(y,2,N-20){
        d[y][1]++;
        ll j = y;
        for (ll i = 2; i * i <= j; i++) {
            while (j % i == 0) {
                d[y][i]++;
                j /= i;
            }
        }
        if (j > 1) d[y][j]++;
    }
}

```

---

### 15. prefix\_and\_or\_xor

```

struct prebit{
    ll n;
    vector<vector<ll>>prefix_bits;
    vector<ll> prexor;
    prebit(ll nn, vector<ll> a){
        prexor.resize(nn+1);
        prefix_bits.resize(32,vector<ll>(nn+5));
        n = nn;
        for(int i = 0; i < n; i++){
            for(int j = 0; j < 30; j++){
                prefix_bits[j][i+1] = prefix_bits[j][i] + (a[i]&(1ll<<j)?1:0);
            }
        }
    }
}

```

```

        for(ll i = 1; i<=n; i++){
            prexor[i]=prexor[i-1]^a[i-1];
        }
    }

    ll por(ll l, ll r){
        int ans = 0;
        for(int j = 0; j < 30; j++){
            if(prefix_bits[j][r] - prefix_bits[j][l-1]){
                ans = ans | (1ll << j);
            }
        }
        return ans;
    }

    ll pand(ll l, ll r){
        int ans = 0;
        for(int j = 0; j < 30; j++){
            if(prefix_bits[j][r] - prefix_bits[j][l-1] == r - l + 1){
                ans = ans | (1ll << j);
            }
        }
        return ans;
    }

    ll pxor(ll l, ll r){
        return prexor[r] ^ prexor[l-1];
    }
};

```

---

## 16. Hashing

```

const int N = 1e6 + 5;
const ll MOD1 = 1e9 + 7, P1 = 31;
const ll MOD2 = 999999937, P2 = 37;
ll pow1[N], inv_pow1[N], pow2[N], inv_pow2[N];
ll power(ll base, ll exp, ll mod) {
    ll res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}
ll modinv(ll n, ll mod) {
    return power(n, mod - 2, mod);
}

struct StringHasher {
    int n;
    ll pre1[N], pre2[N], suf1[N], suf2[N];
    static void init() {
        pow1[0] = pow2[0] = 1;
        for (int i = 1; i < N; i++) {
            pow1[i] = (pow1[i - 1] * P1) % MOD1;
            pow2[i] = (pow2[i - 1] * P2) % MOD2;
        }
    }
};

```

```

    ll invp1 = modinv(P1, MOD1);
    ll invp2 = modinv(P2, MOD2);
    inv_pow1[0] = 1;
    inv_pow2[0] = 1;
    for (int i = 1; i < N; i++) {
        inv_pow1[i] = (inv_pow1[i - 1] * invp1) % MOD1;
        inv_pow2[i] = (inv_pow2[i - 1] * invp2) % MOD2;
    }
}

void build(const string& s) {
    n = (int)s.length();
    if (n == 0) return;
    pre1[0] = ((ll)(s[0] - 'a' + 1) * pow1[0]) % MOD1;
    pre2[0] = ((ll)(s[0] - 'a' + 1) * pow2[0]) % MOD2;
    for (int i = 1; i < n; i++) {
        pre1[i] = (pre1[i - 1] + ((s[i] - 'a' + 1) * pow1[i]) % MOD1) % MOD1;
        pre2[i] = (pre2[i - 1] + ((s[i] - 'a' + 1) * pow2[i]) % MOD2) % MOD2;
    }
    string rs = s;
    reverse(rs.begin(), rs.end());
    suf1[0] = ((ll)(rs[0] - 'a' + 1) * pow1[0]) % MOD1;
    suf2[0] = ((ll)(rs[0] - 'a' + 1) * pow2[0]) % MOD2;
    for (int i = 1; i < n; i++) {
        suf1[i] = (suf1[i - 1] + ((rs[i] - 'a' + 1) * pow1[i]) % MOD1) % MOD1;
        suf2[i] = (suf2[i - 1] + ((rs[i] - 'a' + 1) * pow2[i]) % MOD2) % MOD2;
    }
}

pair<ll, ll> prehash(int l, int r) {
    if (l < 0 || r >= n || l > r) return { 0, 0 };
    ll h1 = (pre1[r] - (l > 0 ? pre1[l - 1] : 0) + MOD1) % MOD1;
    ll h2 = (pre2[r] - (l > 0 ? pre2[l - 1] : 0) + MOD2) % MOD2;

    h1 = (h1 * inv_pow1[l]) % MOD1;
    h2 = (h2 * inv_pow2[l]) % MOD2;
    return { h1, h2 };
}

pair<ll, ll> sufhash(int l, int r) {
    if (l < 0 || r >= n || l > r) return { 0, 0 };
    int L = n - 1 - r;
    int R = n - 1 - l;
    ll h1 = (suf1[R] - (L > 0 ? suf1[L - 1] : 0) + MOD1) % MOD1;
    ll h2 = (suf2[R] - (L > 0 ? suf2[L - 1] : 0) + MOD2) % MOD2;
    h1 = (h1 * inv_pow1[L]) % MOD1;
    h2 = (h2 * inv_pow2[L]) % MOD2;
    return { h1, h2 };
}

bool is_palindrome(int l, int r) {
    if (l > r) return true;
    return prehash(l, r) == sufhash(l, r);
}
};

```

---

## 17. Trie

```

struct trie{
    struct node{
        node*ch[26];
        ll freq,end;
        node(){

```

```

        freq=end=0;
        memset(ch,0,sizeof ch);
    }
};
node*root = new node();
void insert(string s){
    node*cur = root;
    for(auto it:s){
        int idx = it-'a'; // for string
        if(cur->ch[idx]==0) {
            cur->ch[idx]=new node();
        }
        cur=cur->ch[idx];
        cur->freq++;
    }
    cur->end++;
}
ll count(string s){
    node*cur = root;
    for(auto it:s){
        int idx = it-'a'; // for string
        if(cur->ch[idx]==0) return 0;
        cur = cur->ch[idx];
    }
    return cur->freq; // freq or full
}
};
};

```

---

## 18. Binary\_Trie

```

struct binary_trie {
    struct node {
        node* ch[2];
        ll freq[2];
        node() {
            ch[0] = ch[1] = 0;
            freq[0] = freq[1] = 0;
        }
    };
};
node* root = new node();
void insert(ll n) {
    node* cur = root;
    for (ll i = 23; i >= 0; i--) {
        bool idx = (n >> i) & 1;
        if (cur->ch[idx] == 0) {
            cur->ch[idx] = new node();
        }
        cur->freq[idx]++;
        cur = cur->ch[idx];
    }
}
void del(ll n, ll i, node* cur) {
    if (i == -1) return;
    bool idx = (1 & (n >> i));
    //deb(i);
    del(n, i - 1, cur->ch[idx]);
    cur->freq[idx]--;
    if (cur->freq[idx] == 0) {
        delete cur->ch[idx];
    }
}

```

```

        cur->ch[idx] = 0;
    }
}
ll mx_xor(ll n) {
    node* cur = root;
    ll ret = 0;
    for (ll i = 23; i >= 0; i--) {
        bool idx = (1 & (n >> i));
        if (cur->freq[idx ^ 1]) {
            cur = cur->ch[idx ^ 1];
            ret += (1ll << i);
        }
        else {
            cur = cur->ch[idx];
        }
    }
    return ret;
}
ll mn_xor(ll n) {
    node* cur = root;
    ll ret = 0;
    for (ll i = 23; i >= 0; i--) {
        bool idx = (1 & (n >> i));
        if (cur->freq[idx]) {
            cur = cur->ch[idx];
        }
        else {
            ret += (1ll << i);
            cur = cur->ch[idx^1];
        }
    }
    return ret;
}
};

```

---

## 19. Z\_Function

```

vector<int> zfun(string s) {
    int n = s.size();
    vector<int> z(n);
    int x = 0, y = 0;
    for (int i = 1; i < n; i++) {
        z[i] = max(0, min(z[i-x], y-i+1));
        while (i+z[i] < n && s[z[i]] == s[i+z[i]]) {
            x = i; y = i+z[i]; z[i]++;
        }
    }
    return z;
}

```

---

## 20. number\_of\_prime\_factors\_of\_a\_number

```

const ll N = 1e5;

```

```

vector<vll> pr(N+50);
vll num(N+50);
void fun(){
    FOR(i,2,N+1){
        if(pr[i].empty()) for(ll j = i; j<=N; j+=i){
            pr[j].pb(i);
        }
    }
    FOR(i,2,N+1){
        ll x = i;
        ll sum = 0;
        for(auto j : pr[i]){
            while(x%j==0){
                sum++;
                x/=j;
            }
        }
        num[i]+=sum;
    }
}

```

---

## 21. xor\_from\_1\_to\_n

```

auto xor1_n = [&](ll y){
    ll x = y%4;
    if(x==1) return 1ll;
    if(x==2) return y+1;
    if(x==3) return 0ll;
    return y;
};

```

---

## 22. sparse\_table

```

vll v;
vector<vll> st;
vll logg;
ll merge(ll a, ll b){
    return min(a,b);
}
void build(ll n){
    FOR(i,0,n) st[i][0] = v[i];
    for(ll pw = 1; (1ll<<pw)<=n; pw++){
        for(ll i = 0; i+(1ll<<pw)<=n; i++){
            st[i][pw] = merge(st[i][pw-1], st[i+(1ll<<(pw-1))][pw-1]);
        }
    }
}
ll query(ll l, ll r) { // O(log(n)) for sum/mul/xor and all query2
    ll sz = r-l+1;
    ll ret = 1e18;
    for(ll i = 30; i>=0; i--){
        if((sz>>i)&1){

```

```

        ret = merge(ret, st[l][i]);
        l+=(1ll<i);
    }
}
return ret;
}
ll query2(ll l, ll r){ //O(1) gcd/lcm/mx/mn
    ll sz = r-l+1;
    ll pw = logg[sz];
    return merge(st[l][pw], st[r-(1ll<pw)+1][pw]);
}
void blg(ll n){
    logg.resize(n+3);
    logg[1] = 0;
    FOR(i,2,n+1) logg[i] = logg[i/2]+1;
}

```

---

## 23. mo\_in\_array

```

ll sq;
struct query{
    ll l,r,blk,idx;
    query(ll _l, ll rr, ll i){
        l = _l; r = rr; idx = i; blk = _l/sq;
    }
    bool operator < (const query &q) const{
        if(blk!=q.blk) return blk<q.blk;
        return r<q.r;
    }
};
const ll N = 2e5+5;
ll v[N],ans[N],vis[N];
ll n,q,ret = 0;
vector<query> qu;
void add(ll l){
    if(!vis[v[l]]) ret++;
    vis[v[l]]++;
}
void remove(ll l){
    vis[v[l]]--;
    if(!vis[v[l]]) ret--;
}
void mo_build(){
    sort(all(qu));
    ll l = 1, r = 0;
    FOR(i,0,q){
        while(l<qu[i].l) remove(l++);
        while(l>qu[i].l) add(--l);
        while(r<qu[i].r) add(++r);
        while(r>qu[i].r) remove(r--);
        ans[qu[i].idx] = ret;
    }
}

```

---

## 24. segment\_tree

```
struct segment_tree{
private:
#define mid (l+r>>1)
#define L 2*node+1
#define R 2*node+2
    ll sz; vll seg;
    ll merge(ll i, ll j){
        return (i+j);
    }
    void build(ll l, ll r, ll node, vll & arr){
        if(l==r){
            if(l<arr.size()){
                seg[node] = arr[l];
            }
            return;
        }
        build(l,mid,L,arr);
        build(mid+1,r,R,arr);
        seg[node] = merge(seg[L],seg[R]);
    }
    void update(ll l, ll r, ll node, ll idx, ll val){
        if(l==r){
            seg[node] = val;
            return;
        }
        if(idx>mid){
            update(mid+1,r,R,idx,val);
        }
        else {
            update(l,mid,L,idx,val);
        }
        seg[node] = merge(seg[L],seg[R]);
    }
    ll query(ll l, ll r, ll node, ll lq, ll rq){
        if(lq>r||rq<l){
            return 0;
        }
        if(l>=lq&&r<=rq){
            return seg[node];
        }
        return (merge(query(l,mid,L,lq,rq),query(mid+1,r,R,lq,rq)));
    }
public:
    segment_tree(vll arr){
        sz = 1;
        ll n = arr.size();
        while(sz<n) sz*=2;
        seg.resize(2*sz);
        build(0,sz-1,0,arr);
    }
    void update(ll idx, ll val) {
        update(0,sz-1,0,idx,val);
    }
    ll query(ll l, ll r) {
        return query(0,sz-1,0,l,r);
    }
#undef L
#undef R
#undef mid
};
```

---

## 25. sqrt\_deco

```
vll v, blk;
vector<vll> b;
ll sq;
void build(ll n){
    FOR(i, 0, n){
        b[i/sq].pb(v[i]);
        blk[i/sq] += v[i];
    }
}
void update(ll id, ll val){
    ll blk_idx = id/sq;
    ll id_in_blk = id%sq;
    blk[blk_idx] -= v[id];
    blk[blk_idx] += val;
    v[id] = val;
    b[blk_idx][id_in_blk] = val;
}
ll query(ll l, ll r){
    ll ret = 0;
    while(l <= r){
        if(l%sq == 0 && l + sq <= r){
            ret += blk[l/sq];
            l += sq;
        }
        else{
            ret += v[l++];
        }
    }
    return ret;
}
```

---

## 26. lazy\_segment\_(q,p)

```
struct segment_tree{
private:
#define mid (l+r>>1)
#define L 2*node+1
#define R 2*node+2
    ll sz; vll seg;
    ll merge(ll i, ll j){
        return (i+j);
    }
    void build(ll l, ll r, ll node, vll & arr){
        if(l==r){
            if(l<arr.size()){
                seg[node] = arr[l];
            }
            return;
        }
        build(l, mid, L, arr);
        build(mid+1, r, R, arr);
```

```

    }
    void update(ll l, ll r, ll node, ll lq, ll rq, ll val){
        if(l>rq||r<lq) return;
        if(l>=lq&&r<=rq){
            seg[node] = merge(seg[node],val);
            return;
        }
        update(l,mid,L,lq,rq,val);
        update(mid+1,r,R,lq,rq,val);
    }
    ll query(ll l, ll r, ll node,ll idx){
        if(l==r){
            return seg[node];
        }
        if(idx<=mid){
            return merge(query(l,mid,L,idx),seg[node]);
        }
        return merge(query(mid+1,r,R,idx),seg[node]);
    }
public:
    segment_tree(vll arr){
        sz = 1;
        ll n = arr.size();
        while(sz<n) sz*=2;
        seg.resize(2*sz);
        build(0,sz-1,0,arr);
    }
    void update(ll l,ll r, ll val) {
        update(0,sz-1,0,l,r,val);
    }
    ll query(ll idx) {
        return query(0,sz-1,0,idx);
    }
    #undef L
    #undef R
    #undef mid
};

```

---

## 27. lazy\_segment\_(q,q)

```

struct segment_tree{
private:
#define mid (l+r>>1)
#define L 2*node+1
#define R 2*node+2
    ll sz; vll seg,laz;
    ll merge(ll i, ll j){
        return (i+j);
    }
    void prop(ll l, ll r, ll node){
        if(laz[node]==0) {
            return;
        }
        if(l!=r){
            laz[L] += laz[node]/2;
            laz[R] += laz[node]/2;
        }
    }

```

```

    }
    seg[node] += laz[node];
    laz[node] = 0;
}

void build(ll l, ll r, ll node, vll & arr){
    if(l==r){
        if(l<arr.size()){
            seg[node] = arr[l];
        }
        return;
    }
    build(l,mid,L,arr);
    build(mid+1,r,R,arr);
    seg[node] = merge(seg[L],seg[R]);
}

void update(ll l, ll r, ll node, ll lq,ll rq, ll val){
    prop(l,r,node);
    if(l>rq||lq>r) return;
    if(l>=lq&&r<=rq){
        laz[node] += val*(r-l+1);
        prop(l,r,node);
        return;
    }
    update(l,mid,L,lq,rq,val);
    update(mid+1,r,R,lq,rq,val);
    seg[node] = merge(seg[L],seg[R]);
}

ll query(ll l, ll r, ll node, ll lq, ll rq){
    prop(l,r,node);
    if(lq>r||rq<l){
        return 0;
    }
    if(l>=lq&&r<=rq){
        return seg[node];
    }
    return (merge(query(l,mid,L,lq,rq),query(mid+1,r,R,lq,rq)));
}

public:
    segment_tree(vll arr){
        sz = 1;
        ll n = arr.size();
        while(sz<n) sz*=2;
        seg.resize(2*sz);
        laz.resize(2*sz);
        build(0,sz-1,0,arr);
    }
    void update(ll l,ll r, ll val) {
        update(0,sz-1,0,l,r,val);
    }
    ll query(ll l, ll r) {
        return query(0,sz-1,0,l,r);
    }
}

#undef L
#undef R
#undef mid
};

```

---

## 28. XOR\_Basis

```

struct XorBasis■
{■
    static const int MAX_BITS = 64;■
    uint64_t basis[MAX_BITS];■
■
    XorBasis()■
    {■
        for (int i = 0; i < MAX_BITS; ++i)■
            basis[i] = 0;■
    }■
■
    bool insert(uint64_t x)■
    {■
        for (int i = MAX_BITS - 1; i >= 0; --i)■
        {■
            if ((x >> i) & 1ULL)■
            {■
                if (!basis[i])■
                {■
                    basis[i] = x;■
                    return true;■
                }■
                x ^= basis[i];■
            }■
        }■
        return false;■
    }■
■
    uint64_t maxXor() const■
    {■
        uint64_t res = 0;■
        for (int i = MAX_BITS - 1; i >= 0; --i)■
        {■
            if (basis[i] && (res ^ basis[i]) > res)■
                res ^= basis[i];■
        }■
        return res;■
    }■
■
    int size() const■
    {■
        int cnt = 0;■
        for (int i = 0; i < MAX_BITS; ++i)■
            if (basis[i])■
                ++cnt;■
        return cnt;■
    }■
■
    bool canRepresent(uint64_t x) const■
    {■
        for (int i = MAX_BITS - 1; i >= 0; --i)■
        {■
            if ((x >> i) & 1ULL)■
            {■
                if (!basis[i])■
                    return false;■
                x ^= basis[i];■
            }■
        }■
        return x == 0;■
    }■
■
    vector<uint64_t> enumerate() const■
    {■

```

```

vector<uint64_t> subsets = {0};
for (int i = 0; i < MAX_BITS; ++i)
{
    if (basis[i])
    {
        size_t n = subsets.size();
        for (size_t j = 0; j < n; ++j)
        {
            subsets.push_back(subsets[j] ^ basis[i]);
        }
    }
}
return subsets;
};

```

---

## 29. segment\_tree\_2d

```

struct segment_tree_2d {
private:
    ll n, m;
    vector<vector<ll>> seg;

    ll merge(ll i, ll j) {
        return (i + j);
    }

    void build_y(ll lx, ll rx, ll node_x, ll ly, ll ry, ll node_y, vector<vector<ll>> &arr) {
        if (ly == ry) {
            if (lx == rx) {
                seg[node_x][node_y] = arr[lx][ly];
            } else {
                seg[node_x][node_y] = merge(seg[2 * node_x + 1][node_y], seg[2 * node_x + 2][node_y]);
            }
            return;
        }

        ll mid_y = (ly + ry) / 2;
        build_y(lx, rx, node_x, ly, mid_y, 2 * node_y + 1, arr);
        build_y(lx, rx, node_x, mid_y + 1, ry, 2 * node_y + 2, arr);

        seg[node_x][node_y] = merge(seg[node_x][2 * node_y + 1], seg[node_x][2 * node_y + 2]);
    }

    void build_x(ll lx, ll rx, ll node_x, vector<vector<ll>> &arr) {
        if (lx == rx) {
            build_y(lx, rx, node_x, 0, m - 1, 0, arr);
            return;
        }

        ll mid_x = (lx + rx) / 2;
        build_x(lx, mid_x, 2 * node_x + 1, arr);
        build_x(mid_x + 1, rx, 2 * node_x + 2, arr);

        seg[node_x].resize(4 * m);
        build_y(lx, rx, node_x, 0, m - 1, 0, arr);
    }

    void update_y(ll lx, ll rx, ll node_x, ll ly, ll ry, ll node_y, ll x, ll y, ll val) {

```

```

        if (ly == ry) {
            if (lx == rx) {
                seg[node_x][node_y] = val;
            } else {
                seg[node_x][node_y] = merge(seg[2 * node_x + 1][node_y], seg[2 * node_x + 2][node_y]);
            }
            return;
        }

        ll mid_y = (ly + ry) / 2;
        if (y <= mid_y) {
            update_y(lx, rx, node_x, ly, mid_y, 2 * node_y + 1, x, y, val);
        } else {
            update_y(lx, rx, node_x, mid_y + 1, ry, 2 * node_y + 2, x, y, val);
        }

        seg[node_x][node_y] = merge(seg[node_x][2 * node_y + 1], seg[node_x][2 * node_y + 2]);
    }

    void update_x(ll lx, ll rx, ll node_x, ll x, ll y, ll val) {
        if (lx == rx) {
            update_y(lx, rx, node_x, 0, m - 1, 0, x, y, val);
            return;
        }

        ll mid_x = (lx + rx) / 2;
        if (x <= mid_x) {
            update_x(lx, mid_x, 2 * node_x + 1, x, y, val);
        } else {
            update_x(mid_x + 1, rx, 2 * node_x + 2, x, y, val);
        }

        update_y(lx, rx, node_x, 0, m - 1, 0, x, y, val);
    }

    ll query_y(ll lx, ll rx, ll node_x, ll ly, ll ry, ll node_y, ll lqy, ll rqy) {
        if (lqy > ry || rqy < ly) return 0;
        if (ly >= lqy && ry <= rqy) return seg[node_x][node_y];

        ll mid_y = (ly + ry) / 2;
        return merge(query_y(lx, rx, node_x, ly, mid_y, 2 * node_y + 1, lqy, rqy),
                    query_y(lx, rx, node_x, mid_y + 1, ry, 2 * node_y + 2, lqy, rqy));
    }

    ll query_x(ll lx, ll rx, ll node_x, ll lqx, ll rqx, ll lqy, ll rqy) {
        if (lqx > rx || rqx < lx) return 0;
        if (lx >= lqx && rx <= rqx) return query_y(lx, rx, node_x, 0, m - 1, 0, lqy, rqy);

        ll mid_x = (lx + rx) / 2;
        return merge(query_x(lx, mid_x, 2 * node_x + 1, lqx, rqx, lqy, rqy),
                    query_x(mid_x + 1, rx, 2 * node_x + 2, lqx, rqx, lqy, rqy));
    }

public:
    segment_tree_2d(vector<vector<ll>> &arr) {
        n = arr.size();
        m = arr[0].size();
        seg.resize(4 * n, vector<ll>(4 * m));
        build_x(0, n - 1, 0, arr);
    }

    void update(ll x, ll y, ll val) {
        update_x(0, n - 1, 0, x, y, val);
    }

```

```

    ll query(ll x1, ll y1, ll x2, ll y2) {
        return query_x(0, n - 1, 0, x1, x2, y1, y2);
    }
};

```

---

## 30. Li\_Chao\_Tree

```

struct Line {
    ll a, b;
    ll eval(ll x) const {
        return a * x + b;
    }
};

struct LiChaoNode {
    Line line;
    LiChaoNode* left;
    LiChaoNode* right;
    LiChaoNode(Line l) : line(l), left(nullptr), right(nullptr) {}
};

class LiChaoTree {
private:
    LiChaoNode* root;
    ll l, r;

    void insert(LiChaoNode*& node, ll l, ll r, Line newLine) {
        if (!node) {
            node = new LiChaoNode(newLine);
            return;
        }
        ll m = l + (r - l) / 2;
        bool leftBetter = newLine.eval(l) > node->line.eval(l);
        bool midBetter = newLine.eval(m) > node->line.eval(m);
        if (midBetter) {
            swap(node->line, newLine);
        }
        if (l == r) return;
        if (leftBetter != midBetter) {
            insert(node->left, l, m, newLine);
        }
        else {
            insert(node->right, m + 1, r, newLine);
        }
    }

    ll query(LiChaoNode* node, ll l, ll r, ll x) {
        if (!node) return LLONG_MIN;
        ll m = l + (r - l) / 2;
        ll curr = node->line.eval(x);
        if (x <= m) {
            return max(curr, query(node->left, l, m, x));
        }
        else {
            return max(curr, query(node->right, m + 1, r, x));
        }
    }

public:

```

```

    LiChaoTree(ll _l, ll _r) : l(_l), r(_r), root(nullptr) {}
    void add_line(Line line) {
        insert(root, l, r, line);
    }
    ll get_max(ll x) {
        return query(root, l, r, x);
    }
};

```

---

## 31. LCA

```

const ll N = 1e5, LOG = 23;
ll anc[N][LOG], lvl[N]; vector<vi> adj(N);
void buildanc(ll node, ll par) {
    anc[node][0] = par;
    lvl[node] = lvl[par] + 1;
    FOR(i, 1, LOG) {
        int p = anc[node][i - 1];
        anc[node][i] = anc[p][i - 1];
    }
    for (auto it : adj[node]) if (it != par) {
        buildanc(it, node);
    }
}
int kthanc(ll node, ll k) {
    for (ll i = LOG - 1; i >= 0; i--) {
        if ((k >> i) & 1) {
            node = anc[node][i];
        }
    }
    return node;
}
int LCA(ll u, ll v) {
    if (lvl[v] > lvl[u]) swap(u, v);
    u = kthanc(u, lvl[u] - lvl[v]);
    if (u == v) return v;
    for (int i = LOG - 1; i >= 0; i--) {
        if (anc[u][i] != anc[v][i]) {
            v = anc[v][i];
            u = anc[u][i];
        }
    }
    return anc[v][0];
}
int disbet(ll u, ll v) {
    int lca = LCA(u, v);
    return lvl[u] + lvl[v] - 2 * lvl[lca];
}
void clear(ll n) {
    FOR(i, 0, n + 5) {
        adj[i].clear();
        lvl[i] = 0;
        FOR(j, 0, LOG) {
            anc[i][j] = 0;
        }
    }
}

```

---

## 32. NTT

```
const int MOD = 998244353;
const int root = 3;

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % MOD, e /= 2)
        if (e & 1) ans = ans * b % MOD;
    return ans;
}

// Primitive Root of the MOD of form 2^a * b + 1
int generator () {
    vector<int> fact;
    int phi = MOD-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=MOD; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= modpow (res, phi / fact[i]) != 1;
        if (ok) return res;
    }
    return -1;
}

ll modexp (ll a, ll b, ll mod) {
    ll res = 1;
    while(b > 0){
        if(b & 1)
            res = (long long) res * a % mod;
        a = (long long) a * a % mod;
        b >>= 1;
    }
    return res;
}

void ntt(vector<ll>& a, bool invert) {
    ll n = a.size();
    for (ll i = 1, j = 0; i < n; i++) {
        ll bit = n >> 1;
        for(; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if(i < j)
            swap(a[i], a[j]);
    }

    for (ll len = 2; len <= n; len <= 1) {
        ll wlen = modexp(root, (MOD - 1) / len, MOD);
```

```

    if (invert)■
        wlen = modexp(wlen, MOD - 2, MOD); ■
    for (ll i = 0; i < n; i += len) {■
        ll w = 1;■
        for (ll j = 0; j < len/2; j++) {■
            ll u = a[i+j];■
            ll v = (ll)((long long) a[i+j+len/2] * w % MOD);■
            a[i+j] = u + v < MOD ? u + v : u + v - MOD;■
            a[i+j+len/2] = u - v >= 0 ? u - v : u - v + MOD;■
            w = (ll)((long long) w * wlen % MOD);■
        }■
    }■
}■
if(invert) {■
    ll n_inv = modexp(n, MOD - 2, MOD);■
    for (ll & x : a)■
        x = (ll)((long long) x * n_inv % MOD);■
}■
}■
}■
vector<ll> multiply(vector<ll> const& a, vector<ll> const& b) {■
    vector<ll> fa(a.begin(), a.end()), fb(b.begin(), b.end());■
    ll n = 1;■
    while(n < a.size() + b.size() - 1)■
        n <<= 1;■
    fa.resize(n);■
    fb.resize(n);■
    ntt(fa, false);■
    ntt(fb, false);■
    for (ll i = 0; i < n; i++)■
        fa[i] = (ll)((long long) fa[i] * fb[i] % MOD);■
    ntt(fa, true);■
    while(!fa.empty() && fa.back() == 0) fa.pop_back(); // WTF????????????????■
    // for(auto &x:fa) if(x) x = 1; ( )■
    return fa;■
}■
vector<ll> poly_mul(const vector<ll>& a, const vector<ll>& b) {■
    vector<ll> res = multiply(a, b);■
    return res;■
}■
vector<ll> poly_pow(vector<ll> base, ll exp, ll lim = 1e9) {■
    vector<ll> result = {1}; ■
    while(exp > 0) {■
        if(exp & 1) {■
            result = poly_mul(result, base);■
            if(result.size() > lim) result.resize(lim+1); ■
        }■
        base = poly_mul(base, base); ■
        if(base.size() > lim) base.resize(lim+1);■
        exp >>= 1;■
    }■
    return result;■
}

```

---

### 33. Matrix\_def

```

template <typename T>
struct Matrix{
    ll n;
    static const int MOD = 1000000007;

    vector<vector<T>> v;
    Matrix(ll i)
    {
        n = i;
        v.resize(n, vector<T>(n));
    }
    Matrix operator*(const Matrix &other){
        Matrix<T> c(this->n);
        for (int i = 0; i < n; i++){
            for (int j = 0; j < n; j++){
                for (int k = 0; k < n; k++){
                    c.v[i][k] += ((v[i][j]%MOD) * (other.v[j][k]%MOD))%MOD;
                    c.v[i][k] %= MOD;
                }
            }
        }
        return c;
    }
    void Exp_pow(ll pow){
        Matrix<T> tmp(this->n);
        Matrix<T> base(this->n);
        base.v = (this->v);

        for (int i = 0; i < n; i++){
            tmp.v[i][i] = 1;
        }
        while (pow)
        {
            if(pow%2)
            {
                tmp = tmp * base;
            }
            base = base * base;
            pow /= 2;
        }
        this->v = tmp.v;
    }
};

```

---

## 34. Fenwick\_tree

```

struct FenwickTree {
    int n;
    vector<ll> tree;

    FenwickTree(const vector<ll>& arr) {
        n = arr.size();
        tree.assign(n + 1, 0);
    }
};

```

```

        for (int i = 0; i < n; i++) {
            update(i, arr[i]);
        }
    }

    void update(int idx, ll delta) {
        idx++;
        while (idx <= n) {
            tree[idx] += delta;
            idx += idx & -idx;
        }
    }

    ll query(int idx) {
        ll result = 0;
        idx++;
        while (idx > 0) {
            result += tree[idx];
            idx -= idx & -idx;
        }
        return result;
    }

    ll query(int l, int r) {
        if(l) return (query(r) - query(l - 1));
        else return query(r);
    }
};

```

---

## 35. Lucas\_Theorem

```

const int mod = 2;
long long fact[mod], invfact[mod];
void init() {
    fact[0] = 1;
    for (int i = 1; i < mod; i++)
        fact[i] = fact[i-1] * i % mod;
    invfact[mod-1] = modinv(fact[mod-1], mod);
    for (int i = mod-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % mod;
}
long long lucas(long long n, long long k) {
    if (k == 0) return 1;
    int ni = n % mod, ki = k % mod;
    if (ki > ni) return 0;
    return lucas(n / mod, k / mod) * fact[ni] % mod * invfact[ki] % mod * invfact[ni-ki] % mod;
}

```

---

## 36. Aho\_Corasick

```

struct Aho {
    vector< array<int,26> > next;

```

```

vector<int> fail;
vector<int> out;
vector<int> endNode;
vector<int> depthOrder;

Aho(const vector<string>& patterns) {
    build(patterns);
}

void build(const vector<string>& patterns) {
    int n = patterns.size();
    next.clear();
    fail.clear();
    out.clear();
    endNode.assign(n, 0);
    next.push_back(array<int, 26>());
    fail.push_back(0);
    out.push_back(0);
    for(int c = 0; c < 26; c++){
        next[0][c] = -1;
    }
    for(int i = 0; i < n; i++) {
        const string &s = patterns[i];
        int v = 0;
        for(char ch : s) {
            int c = ch - 'a';
            if(next[v][c] == -1) {
                next[v][c] = (int)next.size();
                next.push_back(array<int, 26>());
                fail.push_back(0);
                out.push_back(0);
                for(int d = 0; d < 26; d++){
                    next.back()[d] = -1;
                }
            }
            v = next[v][c];
        }
        endNode[i] = v;
        out[v]++;
    }
    queue<int> q;
    for(int c = 0; c < 26; c++){
        if(next[0][c] != -1) {
            int u = next[0][c];
            fail[u] = 0;
            q.push(u);
        } else {
            next[0][c] = 0;
        }
    }
    while(!q.empty()) {
        int v = q.front();
        q.pop();
        for(int c = 0; c < 26; c++){
            int u = next[v][c];
            if(u != -1) {
                int f = fail[v];
                fail[u] = next[f][c];
                out[u] += out[ fail[u] ];
                q.push(u);
            } else {
                next[v][c] = next[ fail[v] ][c];
            }
        }
    }
}

```

```

        int V = next.size();
        vector<vector<int>> adjFail(V);
        for(int u = 1; u < V; u++){
            adjFail[ fail[u] ].push_back(u);
        }
        depthOrder.clear();
        depthOrder.reserve(V);
        function<void(int)> dfs = [&](int u) {
            for(int w : adjFail[u]) dfs(w);
            depthOrder.push_back(u);
        };
        dfs(0);
    }

    long long query(const string &s) const {
        long long ans = 0;
        int v = 0;
        for(char ch : s) {
            int c = ch - 'a';
            v = next[v][c];
            ans += out[v];
        }
        return ans;
    }

    vector<long long> countOccurrences(const string &text) const {
        int V = next.size();
        vector<long long> cntNode(V, 0);
        int v = 0;
        for(char ch : text) {
            int c = ch - 'a';
            v = next[v][c];
            cntNode[v]++;
        }
        for(int u : depthOrder) {
            if(u == 0) continue;
            cntNode[ fail[u] ] += cntNode[u];
        }
        int n = endNode.size();
        vector<long long> result(n, 0);
        for(int i = 0; i < n; i++) {
            result[i] = cntNode[ endNode[i] ];
        }
        return result;
    }
};

static const int MAXLG = 20;

struct DynamicAC {
    vector<string> bag[MAXLG];
    Aho *aho[MAXLG];
    DynamicAC() {
        for(int i = 0; i < MAXLG; i++) {
            aho[i] = nullptr;
        }
    }
    void insert(const string &s) {
        vector<string> cur_list(1);
        cur_list[0] = s;
        for(int i = 0; i < MAXLG; i++) {
            if(bag[i].empty()) {
                bag[i] = move(cur_list);
                aho[i] = new Aho(bag[i]);
                return;
            }

```

```

        } else {
            vector<string> merged;
            merged.reserve(bag[i].size() + cur_list.size());
            for(auto &x : bag[i]) merged.push_back(move(x));
            for(auto &x : cur_list) merged.push_back(move(x));
            bag[i].clear();
            delete aho[i];
            aho[i] = nullptr;
            cur_list = move(merged);
        }
    }
}

long long query(const string &s) const {
    long long ans = 0;
    for(int i = 0; i < MAXLG; i++) {
        if(aho[i] != nullptr) {
            ans += aho[i]->query(s);
        }
    }
    return ans;
}
};

```

---

## 37. KMP and go state

```

vector<int> pi(m);
for (int i = 1, j = 0; i < m; i++) {
    while (j > 0 && t[i] != t[j])
        j = pi[j-1];
    if (t[i] == t[j])
        ++j;
    pi[i] = j;
}

vector<array<int,26>> go(m+1);
for (int st = 0; st <= m; st++) {
    for (int c = 0; c < 26; c++) {
        if (st < m && t[st] == 'a'+c ) {
            go[st][c] = st + 1;
        } else if (st == 0) {
            go[st][c] = 0;
        } else {
            go[st][c] = go[ pi[st-1] ][c];
        }
    }
}
}

```

---

## 38. Merge\_Sort\_Tree

```

struct merge_sort_tree {
private:
#define L node*2+1
#define R node*2+2

```

```

#define mid ((r+l)>>1)
ll sz; vector<vll> tree;
void build(ll l, ll r, ll node, vll &arr){
    if(l==r){
        if(l<arr.size()) tree[node] = {arr[l]};
        return;
    }
    build(l,mid,L,arr);
    build(mid+1,r,R,arr);
    tree[node].resize(r-l+1);
    ll i = 0, j = 0, k = 0;
    ll szl = tree[L].size();
    ll szr = tree[R].size();
    while(i<tree[L].size()&&j<tree[R].size()){
        if(tree[L][i]<=tree[R][j]){
            tree[node][k] = tree[L][i];
            i++; k++;
        }
        else{
            tree[node][k] = tree[R][j];
            j++; k++;
        }
    }
    while(i<tree[L].size()){
        tree[node][k] = tree[L][i];
        i++; k++;
    }
    while(j<tree[R].size()){
        tree[node][k] = tree[R][j];
        j++; k++;
    }
}
ll query(ll l, ll r, ll node, ll lq, ll rq, ll x){
    if(lq>r||rq<l){
        return 0;
    }
    if(l>=lq&&r<=rq){
        return upper_bound(all(tree[node]),x)-tree[node].begin();
    }
    return(query(l,mid,L,lq,rq,x)+query(mid+1,r,R,lq,rq,x));
}

public:
    merge_sort_tree(vll &arr){
        sz = 1;
        ll n = arr.size();
        while(sz<n) sz*=2;
        tree.resize(sz*2);
        build(0,sz-1,0,arr);
    }
    ll query(ll l, ll r, ll x){
        return query(0,sz-1,0,l,r,x);
    }
};

```

---

## 39. mobius

```

void sieve_mobius() {
    vector<int> primes;

```

```

vector<bool> is_comp(MAXW, false);
mu[1] = 1;
for (int i = 2; i < MAXW; ++i) {
    if (!is_comp[i]) {
        primes.push_back(i);
        mu[i] = -1;
    }
    for (int p : primes) {
        int x = i * p;
        if (x >= MAXW) break;
        is_comp[x] = true;
        if (i % p == 0) {
            mu[x] = 0;
            break;
        } else {
            mu[x] = -mu[i];
        }
    }
}
}

```

---

## 40. Median\_query

```

struct MedianQuery {
    struct Node {
        int cnt;
        Node *l, *r;
        Node(int _cnt = 0, Node* _l = nullptr, Node* _r = nullptr)
            : cnt(_cnt), l(_l), r(_r) {}
    };

    int n, M;
    vector<ll> vals;
    vector<Node*> root;

    Node* build(int L, int R) {
        if (L == R) return new Node();
        int mid = (L + R) >> 1;
        return new Node(0, build(L, mid), build(mid + 1, R));
    }

    Node* update(Node* prev, int L, int R, int pos) {
        if (L == R) return new Node(prev->cnt + 1);
        int mid = (L + R) >> 1;
        if (pos <= mid) {
            Node* nl = update(prev->l, L, mid, pos);
            return new Node(prev->cnt + 1, nl, prev->r);
        } else {
            Node* nr = update(prev->r, mid + 1, R, pos);
            return new Node(prev->cnt + 1, prev->l, nr);
        }
    }

    int kth(Node* u, Node* v, int L, int R, int k) const {
        if (L == R) return L;
        int cntLeft = v->l->cnt - u->l->cnt;
        int mid = (L + R) >> 1;
        if (k <= cntLeft)
            return kth(u->l, v->l, L, mid, k);
    }
}

```

```

        return kth(u->r, v->r, mid + 1, R, k - cntLeft);
    }
}

MedianQuery(const vector<ll>& a) {
    n = a.size();
    vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    M = vals.size();

    root.resize(n + 1);
    root[0] = build(1, M);
    FOR1(i, 1, n) {
        int id = int(lower_bound(vals.begin(), vals.end(), a[i-1]) - vals.begin()) + 1;
        root[i] = update(root[i-1], 1, M, id);
    }
}

// (1-based indices)
ll query(int l, int r) const {
    int len = r - l + 1;
    int p = (len + 2) / 2;
    int idx = kth(root[l-1], root[r], 1, M, p);
    return vals[idx-1];
}
};

```

---

## 41. Median\_Sliding

```

struct SlidingMedian {
    multiset<ll> lower, upper;
    int size() const { return lower.size() + upper.size(); }
    void rebalance() {
        if (lower.size() > upper.size() + 1) {
            upper.insert(*prev(lower.end()));
            lower.erase(prev(lower.end()));
        }
        else if (upper.size() > lower.size()) {
            lower.insert(*upper.begin());
            upper.erase(upper.begin());
        }
    }
    void insert(ll x) {
        if (lower.empty() || x <= *prev(lower.end()))
            lower.insert(x);
        else
            upper.insert(x);
        rebalance();
    }
    void erase(ll x) {
        auto it = lower.find(x);
        if (it != lower.end())
            lower.erase(it);
        else {
            it = upper.find(x);
            if (it != upper.end())
                upper.erase(it);
        }
        rebalance();
    }
};

```

```

}
ll get_median() const {
    return *prev(lower.end());
}
};

```

## 42. 2-sat sheet

```

/*
=====
## 2-SAT Implication Graph Cheat Sheet ##
=====

Core Idea: Each (A or B) clause is equivalent to two implications.
An implication A -> B means "if A is true, then B must be true."

|-----|
| Clause          | Equivalent Implications |
|-----|
| (a OR b)        | ~a -> b , ~b -> a      |
| (~a OR b)       | a -> b , ~b -> ~a      |
| (~a OR ~b)      | a -> ~b , b -> ~a      |
|-----|
| Special Cases   |
|-----|
| a -> b (Implies) | Same as (~a OR b)      |
|                  | a -> b , ~b -> ~a      |
|-----|
| Force a = TRUE  | (a OR a) => ~a -> a     |
| Force a = FALSE | (~a OR ~a) => a -> ~a   |
|-----|
| a OR b = FALSE  | Force both to be FALSE |
|                  | (a -> ~a) AND (b -> ~b) |
|-----|
| a XOR b = TRUE   | Requires 4 implications: |
|                  | (~a->b, ~b->a) AND (a->~b, b->~a) |
|-----|
| a XOR b = FALSE  | Same as a IFF b / XNOR  |
| (a equals b)     | (a->b, ~b->~a) AND (b->a, ~a->~b) |
|-----|

**How to Solve:**
1. Convert every clause into its two implications to build a graph.
2. Find the Strongly Connected Components (SCCs) of the graph.
3. If any variable 'x' and its negation '~x' are in the same SCC,
   the problem is UNSATISFIABLE.
4. Otherwise, it is SATISFIABLE.
5. **Finding the Assignment:** If satisfiable, assign values based on the
   topological order of the SCCs. A common implementation rule is:
   - if scc_id(x) < scc_id(~x) => x = TRUE
   - if scc_id(x) > scc_id(~x) => x = FALSE
   (This assumes SCCs found earlier by the algorithm get a lower ID).
*/

```

## 43. Tarjan for SCC

```
// tarjan for scc
ll n,m;
cin>>n>>m;
vector<vill> node(n+1);
FOR(i,0,m){
    ll a,b;
    cin>>a>>b;
    node[a].pb(b);
}
vill vis(n+1,-1), onst(n+1), low(n+1,1e18);
ll t = 0;
stack<ll> st;
vector<vill> com;
function<void(ll)> dfs = [&](ll u){
    vis[u] = low[u] = t++;
    st.push(u);
    onst[u] = 1;
    for(auto v:node[u]){
        if(vis[v]==-1){
            dfs(v);
            low[u] = min(low[v], low[u]);
        }
        else if(onst[v]){
            low[u] = min(low[u], vis[v]);
        }
    }
    if(low[u]==vis[u]){
        vill p;
        while(1){
            ll b = st.top();
            st.pop();
            p.pb(b);
            onst[b] = 0;
            if(b==u) break;
        }
        com.pb(p);
    }
};
FOR(i,1,n+1) if(vis[i]==-1) dfs(i);
// return void(cout<<"(&)");
deb(com);
if(com.size()==1) cout<<"YES";
else{
    cout<<"NO"<<endl;
    ll x = com[0][0];
    ll y = com[1][0];
    cout<<x<<" "<<y<<endl;
}
```

---

## 44. Tarjan for Bridges

```
// tarjan for bridges
ll n,m;
cin>>n>>m;
vector<vill> node(n+1);
FOR(i,0,m){
```

```

    ll a,b;
    cin>>a>>b;
    node[a].pb(b);
    node[b].pb(a);
}
vill vis(n+1,-1), low(n+1,1e18);
ll t = 0;
vector<pair<ll,ll>> bridges;
function<void(ll,ll)> dfs = [&](ll u, ll f){
    vis[u] = low[u] = t++;
    for(auto v:node[u]) if(v!=f) {
        if(vis[v]==-1){
            dfs(v,u);
            low[u] = min(low[v],low[u]);
            if(low[v]>vis[u]){
                bridges.pb({u,v});
            }
        }
        else {
            low[u] = min(low[u],vis[v]);
        }
    }
};
FOR(i,1,n+1) if(vis[i]==-1) dfs(i,-1);

```

---

## 45. Tarjan for Articulation Points

```

// tarjan for articulation points
ll n,m;
cin>>n>>m;
vector<vill> node(n+1);
FOR(i,0,m){
    ll a,b;
    cin>>a>>b;
    node[a].pb(b);
    node[b].pb(a);
}
vill vis(n+1,-1), low(n+1,1e18);
ll t = 0;
set<ll> artpoints;
function<void(ll,ll)> dfs = [&](ll u, ll f){
    vis[u] = low[u] = t++;
    ll ch = 0;
    for(auto v:node[u]) if(v!=f) {
        if(vis[v]==-1){
            ch++;
            dfs(v,u);
            low[u] = min(low[v],low[u]);
            if(low[v]>=vis[u]&&f!=-1){
                artpoints.insert(u);
            }
        }
        else {
            low[u] = min(low[u],vis[v]);
        }
    }
    if (f==-1&&ch>1) {
        artpoints.insert(u);
    }
};

```

```
};
FOR(i,1,n+1) if(vis[i]==-1) dfs(i, -1);
```

---

## 46. Centroid\_Decomposition\_eg

```
void Hello_World()
{
    ll n,k; cin>>n>>k;
    vector<vill> node(n+1);
    FOR(i,0,n-1){
        ll a,b;
        cin>>a>>b;
        node[a].pb(b);
        node[b].pb(a);
    }
    vill dead(n+1);
    vill sz(n+1);
    ll ans = 0;
    function<void(ll)> fun = [&](ll u){
        function<void(ll,ll)> dfssz = [&](ll u, ll f){
            sz[u] = 1;
            for(auto v:node[u]) if(v!=f&&dead[v]==0){
                dfssz(v,u);
                sz[u]+=sz[v];
            }
        };
        dfssz(u, -1);
        ll d = sz[u];
        function<ll(ll,ll)> dfscen = [&](ll u, ll f){
            ll mx = -1, to = -1;
            for(auto v:node[u]) if(v!=f&&dead[v]==0){
                if(sz[v]>mx){
                    mx = sz[v];
                    to = v;
                }
            }
            if(mx<=d/2) return u;
            else return dfscen(to,u);
        };
        ll cen = dfscen(u, -1);
        vill dis;
        vill mp(d+4);
        mp[0] = 1;
        function<void(ll,ll,ll)> dfsdis = [&](ll u, ll f, ll ds){
            if(ds>k) return;
            dis.pb(ds);
            for(auto v:node[u]) if(v!=f&&dead[v]==0){
                dfsdis(v,u,ds+1);
            }
        };
        for(auto v:node[cen]) if(dead[v]==0){
            dis.clear();
            dfsdis(v,cen,1);
            for(auto x:dis){
                if(k-x>=0&&k-x<=d) ans+=mp[k-x];
            }
            for(auto x:dis){
                mp[x]++;
            }
        }
    }
}
```

```

    }■
    dead[cen] = 1;■
    for(auto v:node[cen]) if(dead[v]==0) fun(v);■
    ■
};■
fun(1);■
cout<<ans;■
}

```

---

## 47. Heavy\_Light\_Decomposition\_eg

```

void Hello_World()■
{■
    ll n,q;■
    cin>>n>>q;■
    ll v[n];■
    ll tt[n];■
    FOR(i,0,n) {cin>>v[i]; tt[i] = v[i];}■
    FOR(i,0,n-1){■
        ll a,b;■
        cin>>a>>b;■
        a--; b--;■
        adj[a].pb(b);■
        adj[b].pb(a);■
    }■
    buildanc(0,0);■
    ll id[n], sz[n], bigsz[n], par[n];■
    ll d = 0;■
    segment_tree seg(vill(n+3));■
    function<void(ll,ll)> dfssz = [&](ll u, ll f){■
        ll sum = 0;■
        par[u] = f;■
        ll mx = -1, mxu = -1;■
        for(auto v:adj[u]) if(v!=f){■
            dfssz(v,u);■
            if(sz[v]>mx){■
                mx = sz[v];■
                mxu = v;■
            }■
            sum+=sz[v];■
        }■
        sz[u] = sum+1;■
        bigsz[u] = mxu;■
    };■
    dfssz(0,-1);■
    function<void(ll,ll)> dfsid = [&](ll u, ll f){■
        id[u] = d++;■
        seg.update(id[u],v[u]);■
        tt[id[u]] = v[u];■
        if (bigsz[u] != -1) {■
            dfsid(bigsz[u], u);■
        }■
        for(auto v:adj[u]) if(v!=f&&v!=bigsz[u]){■
            dfsid(v,u);■
        }■
    };■
    dfsid(0,-1);■
    ll chain[n];■
    FOR(i,0,n) chain[i] = i;■
}

```

```

function<void(ll,ll)> dfsch = [&](ll u, ll f){
    if(bigsz[u]!=-1){
        chain[bigsz[u]] = chain[u];
    }
    for(auto v:adj[u]) if(v!=f){
        dfsch(v,u);
    }
};
dfsch(0,-1);
function<ll(ll,ll)> query_chain = [&](int v, int p) {
    int val = 0;
    while (lvl[p] < lvl[v]) {
        int top = chain[v];
        if (lvl[top] <= lvl[p]) {
            val = max(val, seg.query(id[p] + 1, id[v]));
            v = p;
        } else {
            val = max(val, seg.query(id[top], id[v]));
            v = par[top];
        }
    }
    return val;
};
function<ll(ll,ll)> query = [&](int u, int v) {
    int lc = LCA(u, v);
    int val = max(query_chain(u, lc) , query_chain(v, lc));
    val = max(val , tt[id[lc]]);
    return val;
};
while(q--){
    ll op; cin>>op;
    if(op==1){
        ll u,c;
        cin>>u>>c;
        u--;
        tt[id[u]] = c;
        seg.update(id[u],c);
    }
    else{
        ll u,v;
        cin>>u>>v;
        u--; v--;
        ll ans = query(u,v);
        cout<<ans<<" ";
    }
}

```

---

## 48. stirling\_2d\_type

```

//////////
// O(k*log(k))
ll stirling2(ll n, ll k) {
    if (k < 0 || n < 0 || k > n) {
        return 0;
    }
    if (k == 0 && n == 0) {

```

```

        return 1;■
    }■
    if (k == 0 && n > 0) {■
        return 0;■
    }■
    ll sum = 0;■
    for (ll i = 0; i <= k; ++i) {■
        ll term = (nck(k, i) * pw(i, n)) % mod;■
        if ((k - i) % 2 == 1) { // This handles the (-1)^(k-i) part■
            sum = (sum - term + mod) % mod;■
        } else {■
            sum = (sum + term) % mod;■
        }■
    }■
    return (sum * fact_inv[k]) % mod;■
}■
////////////////////////////////////■
const int MAX_N = 2005;■
const int MAX_K = 2005;■
ll stirring_dp[MAX_N][MAX_K];■
void precompute_stirling(int maxN, int maxK) {■
    stirring_dp[0][0] = 1;■
    for (int i = 1; i < maxN; ++i) {■
        stirring_dp[i][0] = 0; ■
    }■
    for (int j = 1; j < maxK; ++j) {■
        stirring_dp[0][j] = 0; ■
    }■
    for (int i = 1; i < maxN; ++i) {■
        for (int j = 1; j < maxK; ++j) {■
            if (j > i) {■
                stirring_dp[i][j] = 0;■
                continue;■
            }■
            // S(i-1, j-1)■
            ll term1 = stirring_dp[i - 1][j - 1];■
            // j * S(i-1, j)■
            ll term2 = ( (ll)j * stirring_dp[i - 1][j] ) % mod;■
            stirring_dp[i][j] = (term1 + term2) % mod;■
        }■
    }■
}■
ll stirring2d(int n, int k) {■
    // Boundary checks■
    if (k < 0 || n < 0 || k > n) {■
        return 0;■
    }■
    return stirring_dp[n][k];■
}■
////////////////////////////////////■
vector<ll> stirring_row_ntt(int n) {■
    if (n == 0) {■
        return {1};■
    }

```

```

}■
vector<ll> A(n + 1);■
for (int i = 0; i <= n; ++i) {■
    A[i] = (pw(i, n) * fact_inv[i]) % mod;■
}■
vector<ll> B(n + 1);■
for (int j = 0; j <= n; ++j) {■
    B[j] = fact_inv[j];■
    if (j % 2 == 1) {■
        B[j] = (mod - B[j]) % mod;■
    }■
}■
vector<ll> stirling_coeffs = poly_mul(A, B);■
stirling_coeffs.resize(n + 1);■
■
return stirling_coeffs;■
}

```

---

## 49. persistent\_segment\_tree

```

#define MID ((l + r) >> 1)■
■
struct node {■
    ll val;■
    node *l, *r;■
    node(): val(0), l(nullptr), r(nullptr) {}■
    node(ll x): val(x), l(nullptr), r(nullptr) {}■
    node(ll x, node* L, node* R): val(x), l(L), r(R) {}■
};■
■
const int MAXNODE = 8e6;■
const int MAXROOTS = 8e6;■
■
static node buffer[MAXNODE];■
static node* roots[MAXROOTS];■
static int nodecnt = 0;■
■
node* get_new_node() {■
    buffer[nodecnt] = node();■
    return &buffer[nodecnt++];■
}■
■
node* build(const vll &arr, int l, int r) {■
    node* v = get_new_node();■
    if (l == r) {■
        if (l >= 0 && l < (int)arr.size()) v->val = arr[l];■
        else v->val = 0;■
    } else {■
        v->l = build(arr, l, MID);■
        v->r = build(arr, MID + 1, r);■
        v->val = (v->l ? v->l->val : 0) + (v->r ? v->r->val : 0);■
    }■
    return v;■
}■
■
node* update(node* prev, int l, int r, int pos, int val) {■
    node* v = get_new_node();■
    if (prev) *v = *prev;■
    if (l == r) {■

```

```

        v->val = val;■
        v->l = v->r = nullptr;■
    } else {■
        if (pos <= MID) v->l = update(prev ? prev->l : nullptr, l, MID, pos, val);■
        else v->r = update(prev ? prev->r : nullptr, MID + 1, r, pos, val);■
        ll leftVal = v->l ? v->l->val : 0;■
        ll rightVal = v->r ? v->r->val : 0;■
        v->val = leftVal + rightVal;■
    }■
    return v;■
}■
■
ll query(node* v, int l, int r, int ql, int qr) {■
    if (!v || ql > r || qr < l || ql > qr) return 0;■
    if (ql <= l && r <= qr) return v->val;■
    return query(v->l, l, MID, ql, qr) + query(v->r, MID + 1, r, ql, qr);■
}■
■
int sz = 0;■
int ver = 0;■
■
void build(const vill &arr) {■
    ll n = arr.size();■
    sz = n;■
    nodecnt = 0;■
    roots[0] = build(arr, 0, sz - 1);■
    ver = 0;■
}■
■
int clone_version(int base_version) {■
    roots[++ver] = roots[base_version];■
    return ver;■
}■
■
void update(int idx, ll val) {■
    roots[++ver] = update(roots[ver - 1], 0, sz - 1, idx, val);■
}■
■
void update(int base_version, int idx, ll val) {■
    roots[++ver] = update(roots[base_version], 0, sz - 1, idx, val);■
}■
■
ll query(int l, int r) {■
    return query(roots[ver], 0, sz - 1, l, r);■
}■
■
ll query(int l, int r, int version) {■
    return query(roots[version], 0, sz - 1, l, r);■
}■
#undef MID■

```

---

## 50. Persistent\_Segment\_Tree\_with\_Lazy\_Propagation

```

const int MAXROOTS = 200000 + 5;■
const int MAXNODE = 8000000;■
■
struct node {■
    ll val;■
    ll lazy;■

```

```

    node *l, *r;
    node(): val(0), lazy(0), l(nullptr), r(nullptr) {}
};
static node buffer[MAXNODE];
static node* roots[MAXROOTS];
static int nodecnt = 0;

node* new_node(node* from = nullptr) {
    if (nodecnt >= MAXNODE) {
        cerr << "ERROR: node buffer overflow\n";
        exit(1);
    }
    if (from) {
        buffer[nodecnt] = *from;
    } else {
        buffer[nodecnt] = node();
    }
    return &buffer[nodecnt++];
}

node* build(const vector<ll>& arr, int l, int r) {
    node* v = new_node(nullptr);
    if (l == r) {
        v->val = arr[l];
        return v;
    }
    int mid = (l + r) >> 1;
    v->l = build(arr, l, mid);
    v->r = build(arr, mid + 1, r);
    v->val = (v->l ? v->l->val : 0) + (v->r ? v->r->val : 0);
    return v;
}

node* apply_add(node* v, int l, int r, ll add) {
    node* t = new_node(v);
    t->val += add * (r - l + 1);
    t->lazy += add;
    return t;
}

void push(node* v, int l, int r) {
    if (!v || v->lazy == 0 || l == r) return;
    int mid = (l + r) >> 1;
    v->l = apply_add(v->l, l, mid, v->lazy);
    v->r = apply_add(v->r, mid + 1, r, v->lazy);
    v->lazy = 0;
}

node* range_update(node* prev, int l, int r, int ql, int qr, ll val) {
    if (qr < l || r < ql) return prev ? new_node(prev) : new_node(nullptr);
    node* v = new_node(prev);
    if (ql <= l && r <= qr) {
        v->val += val * (r - l + 1);
        v->lazy += val;
        return v;
    }
    push(v, l, r);
    int mid = (l + r) >> 1;
    v->l = range_update(v->l, l, mid, ql, qr, val);
    v->r = range_update(v->r, mid + 1, r, ql, qr, val);
    v->val = (v->l ? v->l->val : 0) + (v->r ? v->r->val : 0);
    return v;
}

```

```

ll range_query(node* v, int l, int r, int ql, int qr) {
    if (!v || qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return v->val;
    push(v, l, r);
    int mid = (l + r) >> 1;
    return range_query(v->l, l, mid, ql, qr) + range_query(v->r, mid + 1, r, ql, qr);
}

int sz = 0;
int ver = 0;

void build(const vector<ll>& arr) {
    nodecnt = 0;
    sz = (int)arr.size();
    roots[0] = build(arr, 0, sz - 1);
    ver = 0;
}

int clone_version(int base_version) {
    ++ver;
    roots[ver] = roots[base_version];
    return ver;
}

int update_current(int l, int r, ll add) {
    ++ver;
    roots[ver] = range_update(roots[ver - 1], 0, sz - 1, l, r, add);
    return ver;
}

int update_from_version(int base_version, int l, int r, ll add) {
    ++ver;
    roots[ver] = range_update(roots[base_version], 0, sz - 1, l, r, add);
    return ver;
}

ll query_current(int l, int r) {
    return range_query(roots[ver], 0, sz - 1, l, r);
}

ll query_version(int version, int l, int r) {
    return range_query(roots[version], 0, sz - 1, l, r);
}

```

---

## 51. Dynamic\_Segment\_Tree

```

struct Node {
    int l = -1;
    int r = -1;
    ll sum = 0;
    Node() = default;
    Node(int L, int R, ll S) : l(L), r(R), sum(S) {}
};

struct DynamicSegTree {
    int root;
    int minV, maxV;
    vector<Node> pool;
}

```

```

DynamicSegTree(int lo, int hi, size_t reserve_nodes = 1<<24) {
    minV = lo; maxV = hi;
    pool.reserve(reserve_nodes);
    root = new_node();
}

inline int new_node() {
    pool.emplace_back();
    return (int)pool.size() - 1;
}

void add(int pos, ll val) {
    if (pos < minV || pos > maxV) return;
    add_rec(root, minV, maxV, pos, val);
}

ll query(int l, int r) {
    if (r < l) return 0;
    if (r < minV || l > maxV) return 0;
    l = max(l, minV);
    r = min(r, maxV);
    return sum_rec(root, minV, maxV, l, r);
}

private:
void add_rec(int idx, int L, int R, int pos, ll val) {
    pool[idx].sum += val;
    if (L == R) return;
    int mid = L + ((R - L) >> 1);
    if (pos <= mid) {
        if (pool[idx].l == -1) pool[idx].l = new_node();
        add_rec(pool[idx].l, L, mid, pos, val);
    } else {
        if (pool[idx].r == -1) pool[idx].r = new_node();
        add_rec(pool[idx].r, mid+1, R, pos, val);
    }
}

ll sum_rec(int idx, int L, int R, int ql, int qr) {
    if (idx == -1) return 0;
    if (ql <= L && R <= qr) return pool[idx].sum;
    if (qr < L || ql > R) return 0;
    int mid = L + ((R - L) >> 1);
    ll s = 0;
    if (ql <= mid && pool[idx].l != -1) s += sum_rec(pool[idx].l, L, mid, ql, qr);
    if (qr > mid && pool[idx].r != -1) s += sum_rec(pool[idx].r, mid+1, R, ql, qr);
    return s;
}
};

```

---

## 52. small\_segment\_tree

```

const ll N = 2e6; // limit for array size
ll n; // array size
ll t[2 * N];

//// FOR(i,0,n) t[i+n] = v[i]; in main before build
void build() { // build the tree
    for (ll i = n - 1; i > 0; --i) t[i] = t[i<<1] + t[i<<1|1];
}

```

```

}■
■
void update(ll p, ll value) { // add value at position p or set if t[p+=n] = value;..■
    for (t[p += n] += value; p > 1; p >= 1) t[p>=1] = t[p] + t[p^1];■
}■
■
ll query(ll l, ll r) { // sum on interval [l, r]■
    r++;■
    ll res = 0;■
    for (l += n, r += n; l < r; l >= 1, r >= 1) {■
        if (l&1) res += t[l++];■
        if (r&1) res += t[--r];■
    }■
    return res;■
}

```

---

## 53. Segment\_Tree\_Merging

```

struct Node {■
    int l = -1, r = -1;■
    long long sum = 0;■
};■
■
struct DynamicSegTree {■
    int minV, maxV;■
    vector<Node> pool;■
■
    DynamicSegTree(int lo, int hi, size_t reserve_nodes = (1u << 20)) {■
        minV = lo; maxV = hi;■
        pool.reserve(reserve_nodes);■
    }■
■
    int make_root() { return new_node(); }■
■
    // point add■
    void add_at(int root, int pos, long long val) {■
        if (root == -1 || pos < minV || pos > maxV || val == 0) return;■
        add_rec(root, minV, maxV, pos, val);■
    }■
■
    // range sum■
    long long query_at(int root, int l, int r) const {■
        if (root == -1 || r < l) return 0;■
        l = max(l, minV); r = min(r, maxV);■
        if (r < minV || l > maxV) return 0;■
        return sum_rec(root, minV, maxV, l, r);■
    }■
■
    // destructive merge: returns 'a' and consumes 'b'■
    int merge_roots(int a, int b) {■
        return merge_rec(a, b, minV, maxV);■
    }■
■
    // ----- NEW: SPLITS -----■
    // 1) Split by key: left contains keys <= key, right contains keys > key■
    pair<int,int> split_by_key(int root, int key) {■
        return split_key_rec(root, minV, maxV, key);■
    }■
}

```

```

// 2) Split by prefix sum: left has first k units of sum (in order), right the rest
// If sums are counts, this is a "split by kth".
pair<int,int> split_by_sum(int root, long long k) {
    return split_sum_rec(root, minV, maxV, k);
}
int kth_smallest(int root, long long k) const {
    return kth_rec(root, minV, maxV, k);
}
// -----

private:
    int new_node() { pool.emplace_back(); return (int)pool.size() - 1; }
    inline long long node_sum(int u) const { return (u == -1 ? 0LL : pool[u].sum); }

    void add_rec(int idx, int L, int R, int pos, long long val) {
        pool[idx].sum += val;
        if (L == R) return;
        int mid = L + ((R - L) >> 1);
        if (pos <= mid) {
            if (pool[idx].l == -1) pool[idx].l = new_node();
            add_rec(pool[idx].l, L, mid, pos, val);
        } else {
            if (pool[idx].r == -1) pool[idx].r = new_node();
            add_rec(pool[idx].r, mid + 1, R, pos, val);
        }
    }

    long long sum_rec(int idx, int L, int R, int ql, int qr) const {
        if (idx == -1) return 0;
        if (ql <= L && R <= qr) return pool[idx].sum;
        if (qr < L || ql > R) return 0;
        int mid = L + ((R - L) >> 1);
        long long s = 0;
        if (pool[idx].l != -1 && ql <= mid) s += sum_rec(pool[idx].l, L, mid, ql, qr);
        if (pool[idx].r != -1 && qr > mid) s += sum_rec(pool[idx].r, mid + 1, R, ql, qr);
        return s;
    }

    int merge_rec(int a, int b, int L, int R) {
        if (a == -1) return b;
        if (b == -1) return a;
        if (L == R) {
            pool[a].sum += pool[b].sum;
            return a;
        }
        int mid = L + ((R - L) >> 1);
        pool[a].l = merge_rec(pool[a].l, pool[b].l, L, mid);
        pool[a].r = merge_rec(pool[a].r, pool[b].r, mid + 1, R);
        pool[a].sum = node_sum(pool[a].l) + node_sum(pool[a].r);
        return a;
    }

    // ---- Implementation: split by key ----
    pair<int,int> split_key_rec(int idx, int L, int R, int key) {
        if (idx == -1) return {-1, -1};
        if (key < L) return {-1, idx}; // whole segment goes to right
        if (key >= R) return {idx, -1}; // whole segment goes to left

        int mid = L + ((R - L) >> 1);

        // Well keep 'idx' as the left node and create a fresh node for right
        int leftRoot = idx;
        int rightRoot = new_node();

        auto [lL, lR] = split_key_rec(pool[idx].l, L, mid, key);

```

```

    auto [rL, rR] = split_key_rec(pool[idx].r, mid + 1, R, key);

    pool[leftRoot].l = lL;
    pool[leftRoot].r = rL;
    pool[rightRoot].l = lR;
    pool[rightRoot].r = rR;

    pool[leftRoot].sum = node_sum(pool[leftRoot].l) + node_sum(pool[leftRoot].r);
    pool[rightRoot].sum = node_sum(pool[rightRoot].l) + node_sum(pool[rightRoot].r);

    if (pool[leftRoot].sum == 0) leftRoot = -1; // drop empty roots
    if (pool[rightRoot].sum == 0) rightRoot = -1;
    return {leftRoot, rightRoot};
}

// ---- Implementation: split by prefix sum k ----
pair<int,int> split_sum_rec(int idx, int L, int R, long long k) {
    if (idx == -1) return {-1, -1};
    if (k <= 0) return {-1, idx}; // nothing to left
    if (pool[idx].sum <= k) return {idx, -1}; // everything to left

    int mid = L + ((R - L) >> 1);
    int leftRoot = idx;
    int rightRoot = new_node();

    long long leftSum = node_sum(pool[idx].l);

    if (k <= leftSum) {
        // take k from left child; right child entirely to right
        auto [lL, lR] = split_sum_rec(pool[idx].l, L, mid, k);
        pool[leftRoot].l = lL;
        pool[leftRoot].r = -1;

        pool[rightRoot].l = lR;
        pool[rightRoot].r = pool[idx].r;
    } else {
        // take entire left child; take (k - leftSum) from right child
        auto [rL, rR] = split_sum_rec(pool[idx].r, mid + 1, R, k - leftSum);
        pool[leftRoot].l = pool[idx].l;
        pool[leftRoot].r = rL;

        pool[rightRoot].l = -1;
        pool[rightRoot].r = rR;
    }

    pool[leftRoot].sum = node_sum(pool[leftRoot].l) + node_sum(pool[leftRoot].r);
    pool[rightRoot].sum = node_sum(pool[rightRoot].l) + node_sum(pool[rightRoot].r);

    if (pool[leftRoot].sum == 0) leftRoot = -1;
    if (pool[rightRoot].sum == 0) rightRoot = -1;
    return {leftRoot, rightRoot};
}

int kth_rec(int idx, int L, int R, long long k) const {
    if (idx == -1 || k <= 0 || pool[idx].sum < k) return -1; // Not found
    if (L == R) return L; // Found it

    int mid = L + ((R - L) >> 1);
    long long leftSum = node_sum(pool[idx].l);

    if (k <= leftSum) {
        return kth_rec(pool[idx].l, L, mid, k);
    } else {
        return kth_rec(pool[idx].r, mid + 1, R, k - leftSum);
    }
}

```

```

    }■
};■

```

---

## 54. xor\_segment\_tree

```

struct xor_segment_tree {■
private:■
#define mid (l+r>>1)■
#define L 2*node+1■
#define R 2*node+2■
    ll sz, mxlvl;■
    vll seg;■
    ll merge(ll i, ll j) {■
        return (i + j);■
    }■
■
    void build(ll l, ll r, ll node, vll& arr) {■
        if (l == r) {■
            if (l < arr.size()) {■
                seg[node] = arr[l];■
            }■
            return;■
        }■
        build(l, mid, L, arr);■
        build(mid + 1, r, R, arr);■
        seg[node] = merge(seg[L], seg[R]);■
    }■
■
    void update(ll l, ll r, ll node, ll idx, ll val) {■
        if (l == r) {■
            seg[node] = val;■
            return;■
        }■
        if (idx > mid) {■
            update(mid + 1, r, R, idx, val);■
        }■
        else {■
            update(l, mid, L, idx, val);■
        }■
        seg[node] = merge(seg[L], seg[R]);■
    }■
■
    ll query(ll l, ll r, ll node, ll lq, ll rq, ll x, ll lvl) {■
        if (lq > r || rq < l) {■
            return 0;■
        }■
        if (l >= lq && r <= rq) {■
            return seg[node];■
        }■
        ll cur = (lvl == 0) ? 0 : (1LL << (lvl - 1));■
        ll md = (r - l + 1) / 2;■
        if (x & cur) {■
            ll a = max(lq, mid + 1) - md;■
            ll b = min(rq, r) - md;■
            ll c = max(lq, l) + md;■
            ll d = min(rq, mid) + md;■
            return merge(query(l, mid, L, a, b, x, lvl - 1),■
                query(mid + 1, r, R, c, d, x, lvl - 1));■
        }■
    }■

```

```

        else {
            return (merge(query(l, mid, L, lq, rq, x, lvl - 1),
                          query(mid + 1, r, R, lq, rq, x, lvl - 1)));
        }
    }

public:
    xor_segment_tree(vll arr) {
        sz = 1;
        ll n = arr.size();
        while (sz < n) sz *= 2;
        mxlvl = 0;
        if (sz > 0) {
            while ((1LL <= mxlvl) < sz) {
                mxlvl++;
            }
        } seg.resize(2 * sz);
        build(0, sz - 1, 0, arr);
    }
    void update(ll idx, ll val) {
        update(0, sz - 1, 0, idx, val);
    }
    ll query(ll l, ll r, ll x) {
        return query(0, sz - 1, 0, l, r, x, mxlvl);
    }
};

#undef L
#undef R
#undef mid
};

```

---

## 55. xor\_segment\_tree\_Non-commutative\_case

```

struct Node {
    ll sum, pref, suff, ans;
    Node(ll val = 0) {
        sum = val;
        pref = max(0LL, val);
        suff = max(0LL, val);
        ans = max(0LL, val);
    }
};

Node merge(const Node& L, const Node& R) {
    Node res;
    res.sum = L.sum + R.sum;
    res.pref = max(L.pref, L.sum + R.pref);
    res.suff = max(R.suff, R.sum + L.suff);
    res.ans = max({L.ans, R.ans, L.suff + R.pref});
    return res;
}

struct xor_segment_tree {
private:
#define mid (l+r>>1)
#define L 2*node+1
#define R 2*node+2
    ll sz, mxlvl;
    vector<vector<Node>> seg;
    void build(ll l, ll r, ll node, ll lvl, vll& arr) {

```

```

        seg[node].resize(1LL << lvl);
        if (l == r) {
            if (l < arr.size()) {
                seg[node][0] = Node(arr[l]);
            } else {
                seg[node][0] = Node(0);
            }
            return;
        }
        build(l, mid, L, lvl - 1, arr);
        build(mid + 1, r, R, lvl - 1, arr);
        ll msk = (1LL << (lvl - 1));
        ll current_bit = msk;

        for (ll i = 0; i < msk; i++) {
            seg[node][i] = merge(seg[L][i], seg[R][i]);
            seg[node][i | current_bit] = merge(seg[R][i], seg[L][i]);
        }
    }

public:
    xor_segment_tree(vll arr, ll n_exponent) {
        mxlvl = n_exponent;
        sz = (1LL << mxlvl);
        seg.resize(4 * sz);
        build(0, sz - 1, 0, mxlvl, arr);
    }
    ll query(ll x_mask) {
        return seg[0][x_mask].ans;
    }
};

#undef L
#undef R
#undef mid
};

```

---

## 56. segment\_tree\_beats\_min\_add

```

const ll INF = 1e18;
const ll NINF = -1e18;
struct segment_tree {
private:
    #define mid (l + r >> 1)
    #define L 2 * node + 1
    #define R 2 * node + 2
    struct Node {
        ll sum;
        ll max; // For chmax: Rename to min, second_min, min_count
        ll second_max;
        int max_count;
        ll lazy_add;
        ll lazy_chmin; // For chmax: Rename to lazy_chmax
        Node() : sum(0),
                max(NINF), // For chmax: Change to INF
                second_max(NINF), // For chmax: Change to INF
                max_count(0),
                lazy_add(0),
                lazy_chmin(INF) // For chmax: Change to NINF
        {}
    }

```

```

};
ll sz;
vector<Node> seg;
Node merge(const Node& left, const Node& right) {
    Node res;
    res.sum = left.sum + right.sum;
    // For chmax: Invert all comparisons ('>' to '<')
    if (left.max > right.max) {
        res.max = left.max;
        res.max_count = left.max_count;
        // For chmax: Use min
        res.second_max = max(left.second_max, right.max);
    } else if (right.max > left.max) {
        res.max = right.max;
        res.max_count = right.max_count;
        // For chmax: Use min
        res.second_max = max(left.max, right.second_max);
    } else { // left.max == right.max
        res.max = left.max;
        res.max_count = left.max_count + right.max_count;
        // For chmax: Use min
        res.second_max = max(left.second_max, right.second_max);
    }
    return res;
}
// For chmax: Rename parameter chmin_val to chmax_val
void apply(ll l, ll r, ll node, ll add_val, ll chmin_val) {
    if (add_val != 0) {
        seg[node].sum += add_val * (r - l + 1);
        seg[node].max += add_val; // For chmax: seg[node].min += add_val
        // For chmax: Check against INF
        if (seg[node].second_max != NINF) {
            // For chmax: seg[node].second_min += add_val
            seg[node].second_max += add_val;
        }
        seg[node].lazy_add += add_val;
        // For chmax: Check against NINF
        if (seg[node].lazy_chmin != INF) {
            // For chmax: seg[node].lazy_chmax += add_val
            seg[node].lazy_chmin += add_val;
        }
    }
    // For chmax: Change check to (chmax_val != NINF)
    if (chmin_val != INF) {
        // For chmax: Skip condition becomes (chmax_val <= seg[node].min)
        if (chmin_val >= seg[node].max) return;
        // For chmax: This becomes an ADDITION
        // seg[node].sum += (chmax_val - seg[node].min) * seg[node].min_count;
        seg[node].sum -= (seg[node].max - chmin_val) * seg[node].max_count;
        // For chmax: seg[node].min = chmax_val;
        seg[node].max = chmin_val;
        // For chmax: seg[node].lazy_chmax = chmax_val;
        seg[node].lazy_chmin = chmin_val;
    }
}
void push(ll l, ll r, ll node) {
    if (l == r) return;
    apply(l, mid, L, seg[node].lazy_add, seg[node].lazy_chmin);
    apply(mid + 1, r, R, seg[node].lazy_add, seg[node].lazy_chmin);
    seg[node].lazy_add = 0;
    seg[node].lazy_chmin = INF; // For chmax: Reset lazy_chmax to NINF
}
void build(ll l, ll r, ll node, const vll& arr) {
    if (l == r) {
        if (l < arr.size()) {

```

```

        seg[node].sum = arr[l];
        // For chmax: .min = arr[l]
        seg[node].max = arr[l];
        // For chmax: .second_min = INF
        seg[node].second_max = NINF;
        // For chmax: .min_count = 1
        seg[node].max_count = 1;
    }
    return;
}
build(l, mid, L, arr);
build(mid + 1, r, R, arr);
seg[node] = merge(seg[L], seg[R]);
}
void update_add(ll l, ll r, ll node, ll lq, ll rq, ll val) {
    push(l, r, node);
    if (l > rq || r < lq) return;
    if (l >= lq && r <= rq) {
        // For chmax: apply(l, r, node, val, NINF);
        apply(l, r, node, val, INF);
        return;
    }
    update_add(l, mid, L, lq, rq, val);
    update_add(mid + 1, r, R, lq, rq, val);
    seg[node] = merge(seg[L], seg[R]);
}
// For chmax: Rename function to update_max
void update_min(ll l, ll r, ll node, ll lq, ll rq, ll val) {
    push(l, r, node);
    if (l > rq || r < lq) return;
    // [Skip Condition]
    // For chmax: if (seg[node].min >= val) return;
    if (seg[node].max <= val) {
        return;
    }
    // [Full Update Condition]
    // For chmax: if (... && seg[node].second_min > val)
    if (l >= lq && r <= rq && seg[node].second_max < val) {
        // For chmax: apply(l, r, node, 0, val);
        apply(l, r, node, 0, val);
        return;
    }
    // [Recurse Condition]
    // For chmax: Call update_max
    update_min(l, mid, L, lq, rq, val);
    update_min(mid + 1, r, R, lq, rq, val);
    seg[node] = merge(seg[L], seg[R]);
}
ll query_sum(ll l, ll r, ll node, ll lq, ll rq) {
    push(l, r, node);
    if (l > rq || r < lq) {
        return 0;
    }
    if (l >= lq && r <= rq) {
        return seg[node].sum;
    }
    return query_sum(l, mid, L, lq, rq) + query_sum(mid + 1, r, R, lq, rq);
}
public:
    segment_tree(const vll& arr) {
        sz = 1;
        ll n = arr.size();
        while (sz < n) sz *= 2;
        seg.resize(2 * sz);
    }

```

```

        build(0, sz - 1, 0, arr);
    }
    void update_add(ll l, ll r, ll val) {
        update_add(0, sz - 1, 0, l, r, val);
    }
    // For chmax: Rename function to update_max
    void update_min(ll l, ll r, ll val) {
        update_min(0, sz - 1, 0, l, r, val);
    }
    ll query_sum(ll l, ll r) {
        return query_sum(0, sz - 1, 0, l, r);
    }
    #undef L
    #undef R
    #undef mid
};

```

---

## 57. segment\_tree\_beats\_max\_add

```

const ll INF = 1e18;
const ll NINF = -1e18;
struct segment_tree {
private:
    #define mid (l + r >> 1)
    #define L 2 * node + 1
    #define R 2 * node + 2
    struct Node {
        ll sum;
        ll min;
        ll second_min;
        int min_count;
        ll lazy_add;
        ll lazy_chmax;
        Node() : sum(0),
                min(INF),
                second_min(INF),
                min_count(0),
                lazy_add(0),
                lazy_chmax(NINF)
        {}
    };
    ll sz;
    vector<Node> seg;
    Node merge(const Node& left, const Node& right) {
        Node res;
        res.sum = left.sum + right.sum;
        if (left.min < right.min) {
            res.min = left.min;
            res.min_count = left.min_count;
            res.second_min = min(left.second_min, right.min);
        } else if (right.min < left.min) {
            res.min = right.min;
            res.min_count = right.min_count;
            res.second_min = min(left.min, right.second_min);
        } else { // left.min == right.min
            res.min = left.min;
            res.min_count = left.min_count + right.min_count;
            res.second_min = min(left.second_min, right.second_min);
        }
    }

```

```

        return res;
    }
    void apply(ll l, ll r, ll node, ll add_val, ll chmax_val) {
        if (add_val != 0) {
            seg[node].sum += add_val * (r - l + 1);
            seg[node].min += add_val;
            if (seg[node].second_min != INF) {
                seg[node].second_min += add_val;
            }
            seg[node].lazy_add += add_val;
            if (seg[node].lazy_chmax != NINF) {
                seg[node].lazy_chmax += add_val;
            }
        }
        if (chmax_val != NINF) {
            if (chmax_val <= seg[node].min) return;
            seg[node].sum += (chmax_val - seg[node].min) * seg[node].min_count;
            seg[node].min = chmax_val;
            seg[node].lazy_chmax = chmax_val;
        }
    }
    void push(ll l, ll r, ll node) {
        if (l == r) return;
        apply(l, mid, L, seg[node].lazy_add, seg[node].lazy_chmax);
        apply(mid + 1, r, R, seg[node].lazy_add, seg[node].lazy_chmax);
        seg[node].lazy_add = 0;
        seg[node].lazy_chmax = NINF;
    }
    void build(ll l, ll r, ll node, const vi& arr) {
        if (l == r) {
            if (l < arr.size()) {
                seg[node].sum = arr[l];
                seg[node].min = arr[l];
                seg[node].second_min = INF;
                seg[node].min_count = 1;
            }
            return;
        }
        build(l, mid, L, arr);
        build(mid + 1, r, R, arr);
        seg[node] = merge(seg[L], seg[R]);
    }
    void update_add(ll l, ll r, ll node, ll lq, ll rq, ll val) {
        push(l, r, node);
        if (l > rq || r < lq) return;
        if (l >= lq && r <= rq) {
            apply(l, r, node, val, NINF);
            return;
        }
        update_add(l, mid, L, lq, rq, val);
        update_add(mid + 1, r, R, lq, rq, val);
        seg[node] = merge(seg[L], seg[R]);
    }
    void update_max(ll l, ll r, ll node, ll lq, ll rq, ll val) {
        push(l, r, node);
        if (l > rq || r < lq) return;
        if (seg[node].min >= val) {
            return;
        }
        if (l >= lq && r <= rq && seg[node].second_min > val) {
            apply(l, r, node, 0, val);
            return;
        }
        update_max(l, mid, L, lq, rq, val);
        update_max(mid + 1, r, R, lq, rq, val);
    }

```

```

        seg[node] = merge(seg[L], seg[R]);
    }
    ll query_sum(ll l, ll r, ll node, ll lq, ll rq) {
        push(l, r, node);
        if (l > rq || r < lq) {
            return 0;
        }
        if (l >= lq && r <= rq) {
            return seg[node].sum;
        }
        return query_sum(l, mid, L, lq, rq) + query_sum(mid + 1, r, R, lq, rq);
    }
}

public:
    segment_tree(const vi& arr) {
        sz = 1;
        ll n = arr.size();
        while (sz < n) sz *= 2;
        seg.resize(2 * sz);
        build(0, sz - 1, 0, arr);
    }
    void update_add(ll l, ll r, ll val) {
        update_add(0, sz - 1, 0, l, r, val);
    }
    void update_max(ll l, ll r, ll val) {
        update_max(0, sz - 1, 0, l, r, val);
    }
    ll query_sum(ll l, ll r) {
        return query_sum(0, sz - 1, 0, l, r);
    }
}

#undef L
#undef R
#undef mid
};

```

---

## 58. segment\_tree\_beats\_mod,div,sqrt

```

const ll INF = (ll)1e18;
const ll NINF = (ll)-1e18;
struct segment_tree {
public:
    enum OpType { MOD, Sqrt, DIV };

private:
#define mid ((l + r) >> 1)
#define L (2 * node + 1)
#define R (2 * node + 2)

    struct Node {
        ll sum;
        ll mx;
        ll smax;
        int cnt;
        Node(): sum(0), mx(NINF), smax(NINF), cnt(0) {}
    };
    ll sz;
    vector<Node> seg;
    vector<ll> lazy_add;
    vector<ll> lazy_set;

```

```

static constexpr ll NO_SET = (1LL<<62);
OpType operation_type;
static Node merge(const Node& a, const Node& b) {
    Node c;
    c.sum = a.sum + b.sum;
    if (a.mx > b.mx) {
        c.mx = a.mx; c.cnt = a.cnt;
        c.smax = std::max(a.smax, b.mx);
    } else if (b.mx > a.mx) {
        c.mx = b.mx; c.cnt = b.cnt;
        c.smax = std::max(b.smax, a.mx);
    } else {
        c.mx = a.mx; c.cnt = a.cnt + b.cnt;
        c.smax = std::max(a.smax, b.smax);
    }
    return c;
}

inline void apply_set(ll node, ll l, ll r, ll x) {
    lazy_set[node] = x;
    lazy_add[node] = 0;
    seg[node].sum = x * (r - l + 1);
    seg[node].mx = x;
    seg[node].smax = NINF;
    seg[node].cnt = (int)(r - l + 1);
}

inline void apply_add(ll node, ll l, ll r, ll x) {
    if (lazy_set[node] != NO_SET) lazy_set[node] += x;
    else lazy_add[node] += x;
    seg[node].sum += x * (r - l + 1);
    seg[node].mx += x;
    if (seg[node].smax != NINF) seg[node].smax += x;
}

inline void chmin_on_max(ll node, ll new_mx) {
    ll dec = seg[node].mx - new_mx;
    if (dec <= 0) return;
    seg[node].sum -= dec * (ll)seg[node].cnt;
    seg[node].mx = new_mx;
}

inline void push(ll l, ll r, ll node) {
    if (l == r) { lazy_add[node] = 0; lazy_set[node] = NO_SET; return; }
    if (lazy_set[node] != NO_SET) {
        apply_set(L, l, mid, lazy_set[node]);
        apply_set(R, mid + 1, r, lazy_set[node]);
        lazy_set[node] = NO_SET;
    }
    if (lazy_add[node] != 0) {
        apply_add(L, l, mid, lazy_add[node]);
        apply_add(R, mid + 1, r, lazy_add[node]);
        lazy_add[node] = 0;
    }
    if (seg[L].mx > seg[node].mx)
        chmin_on_max(L, seg[node].mx);
    if (seg[R].mx > seg[node].mx)
        chmin_on_max(R, seg[node].mx);
}

void build(ll l, ll r, ll node, const vll& arr) {
    if (l == r) {
        ll v = (l < (ll)arr.size() ? arr[l] : 0);
        seg[node].sum = seg[node].mx = v;
        seg[node].smax = NINF;
        seg[node].cnt = 1;
        return;
    }
    build(l, mid, L, arr);

```

```

        build(mid + 1, r, R, arr);
        seg[node] = merge(seg[L], seg[R]);
    }
    void range_set(ll l, ll r, ll node, ll ql, ll qr, ll x) {
        if (r < ql || qr < l) return;
        if (ql <= l && r <= qr) { apply_set(node, l, r, x); return; }
        push(l, r, node);
        range_set(l, mid, L, ql, qr, x);
        range_set(mid + 1, r, R, ql, qr, x);
        seg[node] = merge(seg[L], seg[R]);
    }
    void range_add(ll l, ll r, ll node, ll ql, ll qr, ll x) {
        if (r < ql || qr < l) return;
        if (ql <= l && r <= qr) { apply_add(node, l, r, x); return; }
        push(l, r, node);
        range_add(l, mid, L, ql, qr, x);
        range_add(mid + 1, r, R, ql, qr, x);
        seg[node] = merge(seg[L], seg[R]);
    }
    bool try_apply(ll node, ll l, ll r, ll param) {
        if (operation_type == MOD) {
            if (seg[node].mx < param) return true;
            if (seg[node].smax < param) {
                ll nm = seg[node].mx % param;
                if (nm >= seg[node].smax) { chmin_on_max(node, nm); return true; }
            }
            return false;
        } else if (operation_type == DIV) {
            if (param <= 1 || seg[node].mx == 0) return true;
            if (seg[node].smax == NINF) {
                ll nm = seg[node].mx / param;
                apply_set(node, l, r, nm);
                return true;
            }
            return false;
        } else {
            if (seg[node].mx <= 1) return true;
            if (seg[node].smax <= 1) {
                ll nm = (ll)std::sqrt((long double)seg[node].mx);
                if (nm >= seg[node].smax) { chmin_on_max(node, nm); return true; }
            }
            return false;
        }
    }
    void range_op(ll l, ll r, ll node, ll ql, ll qr, ll param) {
        if (r < ql || qr < l) return;
        if (ql <= l && r <= qr && try_apply(node, l, r, param)) return;
        if (l == r) {
            ll v = seg[node].mx;
            if (operation_type == MOD) v %= param;
            else if (operation_type == DIV) v /= param;
            else v = (ll)std::sqrt((long double)v);
            seg[node].sum = seg[node].mx = v;
            seg[node].smax = NINF; seg[node].cnt = 1;
            return;
        }
        push(l, r, node);
        range_op(l, mid, L, ql, qr, param);
        range_op(mid + 1, r, R, ql, qr, param);
        seg[node] = merge(seg[L], seg[R]);
    }
    ll query_sum(ll l, ll r, ll node, ll ql, ll qr) {
        if (r < ql || qr < l) return 0;
        if (ql <= l && r <= qr) return seg[node].sum;
        push(l, r, node);

```

```

        return query_sum(l, mid, L, ql, qr) + query_sum(mid + 1, r, R, ql, qr);
    }
}

ll query_max(ll l, ll r, ll node, ll ql, ll qr) {
    if (r < ql || qr < l) return NINF;
    if (ql <= l && r <= qr) return seg[node].mx;
    push(l, r, node);
    return std::max(query_max(l, mid, L, ql, qr),
                    query_max(mid + 1, r, R, ql, qr));
}

public:
    segment_tree(const vll& arr, OpType op_type) {
        operation_type = op_type;
        ll n = (ll)arr.size();
        sz = 1; while (sz < n) sz <= 1;
        seg.assign(2 * sz, Node());
        lazy_add.assign(2 * sz, 0);
        lazy_set.assign(2 * sz, NO_SET);
        build(0, sz - 1, 0, arr);
    }
    void update(ll l, ll r, ll x = 0) {
        if (operation_type == DIV && x == 0) return;
        range_op(0, sz - 1, 0, l, r, x);
    }
    void add(ll l, ll r, ll x) { range_add(0, sz - 1, 0, l, r, x); }
    void set(ll l, ll r, ll x) { range_set(0, sz - 1, 0, l, r, x); }
    ll query_sum(ll l, ll r) { return query_sum(0, sz - 1, 0, l, r); }
    ll query_max(ll l, ll r) { return query_max(0, sz - 1, 0, l, r); }
}

#undef L
#undef R
#undef mid
};

// segment_tree mod_tree(my_array, segment_tree::OpType::MOD);

```

---

## 59. Fenwick\_Range\_Update

```

struct FenwickTree {
    int n;
    vector<ll> tree1;
    vector<ll> tree2;
    void _update_internal(vector<ll>& tree, int idx, ll delta) {
        idx++; // 1-based indexing
        while (idx <= n) {
            tree[idx] += delta;
            idx += idx & -idx;
        }
    }
    ll _query_internal(const vector<ll>& tree, int idx) {
        ll result = 0;
        idx++; // 1-based indexing
        while (idx > 0) {
            result += tree[idx];
            idx -= idx & -idx;
        }
        return result;
    }
}

```

```

FenwickTree(const vector<ll>& arr) {
    n = arr.size();
    tree1.assign(n + 1, 0);
    tree2.assign(n + 1, 0);
    ll prev = 0;
    for (int i = 0; i < n; i++) {
        ll delta = arr[i] - prev;
        _update_internal(tree1, i, delta);
        _update_internal(tree2, i, delta * i);
        prev = arr[i];
    }
}

FenwickTree(int sz) {
    n = sz;
    tree1.assign(n + 1, 0);
    tree2.assign(n + 1, 0);
}

void update(int l, int r, ll val) {
    _update_internal(tree1, l, val);
    _update_internal(tree2, l, val * l);
    if (r + 1 < n) {
        _update_internal(tree1, r + 1, -val);
        _update_internal(tree2, r + 1, -val * (r + 1));
    }
}

ll query(int idx) {
    ll sum_d = _query_internal(tree1, idx);
    ll sum_dj = _query_internal(tree2, idx);
    return (idx + 1) * sum_d - sum_dj;
}

ll query(int l, int r) {
    if(l) return (query(r) - query(l - 1));
    else return query(r);
}
};

```

---

## 60. Persistent\_Trie

```

template<int SIGMA = 26, char OFFSET = 'a'>
struct PersistentTrie {
    struct Node {
        int ch[SIGMA];
        ll pass, end;
        Node() : pass(0), end(0) { memset(ch, 0, sizeof ch); }
    };
    vector<Node> pool; // 1-based; 0 = null
    vector<int> roots; // root per version
    PersistentTrie() {
        pool.push_back(Node()); // idx 0 = null
        roots.push_back(newNode()); // version 0: empty root
    }
    void insert(const string& s) { roots.push_back(apply_delta(roots.back(), s, +1)); }
    void erase(const string& s) {
        if (count_word(s, (int)roots.size()-1) == 0) roots.push_back(roots.back());
        else roots.push_back(apply_delta(roots.back(), s, -1));
    }
    ll count_prefix(const string& s, int version) const {

```

```

    int cur = safe_root(version);■
    if (!cur) return 0;■
    for (char c : s) {■
        int idx = mapChar(c);■
        if (idx < 0) return 0;■
        cur = pool[cur].ch[idx];■
        if (!cur) return 0;■
    }■
    return pool[cur].pass;■
}■
ll count_word(const string& s, int version) const {■
    int cur = safe_root(version);■
    if (!cur) return 0;■
    for (char c : s) {■
        int idx = mapChar(c);■
        if (idx < 0) return 0;■
        cur = pool[cur].ch[idx];■
        if (!cur) return 0;■
    }■
    return pool[cur].end;■
}■
ll range_count_word(const string& s, int l, int r) const {■
    l = max(l, 0), r = min(r, (int)roots.size()-1);■
    if (l > r) return 0;■
    return count_word(s, r) - (l ? count_word(s, l-1) : 0);■
}■
ll range_count_prefix(const string& s, int l, int r) const {■
    l = max(l, 0), r = min(r, (int)roots.size()-1);■
    if (l > r) return 0;■
    return count_prefix(s, r) - (l ? count_prefix(s, l-1) : 0);■
}■
bool kth(int version, long long k, string& out) const {■
    int cur = safe_root(version);■
    if (!cur) return false;■
    if (k <= 0 || pool[cur].pass < k) return false;■
    out.clear();■
    while (true) {■
        if (pool[cur].end >= k) return true;■
        k -= pool[cur].end;■
        bool moved = false;■
        for (int i = 0; i < SIGMA; ++i) {■
            int v = pool[cur].ch[i];■
            if (!v) continue;■
            if (pool[v].pass >= k) {■
                out.push_back((char)(i + OFFSET));■
                cur = v;■
                moved = true;■
                break;■
            } else k -= pool[v].pass;■
        }■
        if (!moved) return false;■
    }■
}■
}■
■
int versions() const { return (int)roots.size(); }■
■
private:■
int newNode(const Node* copy = nullptr) {■
    pool.push_back(copy ? *copy : Node());■
    return (int)pool.size() - 1;■
}■
int clone(int id) { return newNode(&pool[id]); }■
static int mapChar(char c) {■
    int idx = (int)c - (int)OFFSET;■
    return (0 <= idx && idx < SIGMA) ? idx : -1;■
}■

```

```

}■
int safe_root(int version) const {■
    if (version < 0 || version >= (int)roots.size()) return 0;■
    return roots[version];■
}■
int apply_delta(int root, const string& s, ll delta) {■
    int newRoot = clone(root);■
    int cur = newRoot;■
    pool[cur].pass += delta;■
    for (char c : s) {■
        int idx = mapChar(c);■
        if (idx < 0) return newRoot;■
        int oldChild = pool[cur].ch[idx];■
        int newChild = oldChild ? clone(oldChild) : newNode();■
        pool[cur].ch[idx] = newChild;■
        cur = newChild;■
        pool[cur].pass += delta;■
    }■
    pool[cur].end += delta;■
    return newRoot;■
}■
};

```

---

## 61. Persistent\_Binary\_Trie

```

template<int MAXLOG = 31>■
struct PersistentBinaryTrie {■
    struct Node {■
        int ch[2];■
        int cnt;■
        Node() : ch{0,0}, cnt(0) {}■
    };■
    vector<Node> pool;■
    vector<int> roots;■
    PersistentBinaryTrie() {■
        pool.reserve(1<<20);■
        pool.push_back(Node());■
        roots.push_back(newNode());■
    }■
    void add(uint32_t x) {■ // Type 0■
        roots.push_back(insertPath(roots.back(), x));■
    }■
    void pop_last(int k) {■ // Type 2■
        int keep = max(0, (int)roots.size()-1 - k);■
        roots.resize(keep + 1);■
    }■
    uint32_t max_xor(uint32_t x, int L, int R) const { // Type 1■
        int u = rootAt(R), v = rootAt(L-1);■
        uint32_t ans = 0;■
        for (int b = MAXLOG; b >= 0; --b) {■
            int bit = (x >> b) & 1;■
            int uAlt = pool[u].ch[bit ^ 1], vAlt = pool[v].ch[bit ^ 1];■
            if (countNode(uAlt) - countNode(vAlt) > 0) {■
                ans |= (1u << b);■
            }
        }
    }
};

```

```

        u = uAlt; v = vAlt;■
    } else {■
        u = pool[u].ch[bit];■
        v = pool[v].ch[bit];■
    }■
}■
return ans;■
}■
■
uint32_t min_xor(uint32_t x, int L, int R) const {■
    int u = rootAt(R), v = rootAt(L - 1);■
    uint32_t ans = 0;■
    for (int b = MAXLOG; b >= 0; --b) {■
        int bit = (x >> b) & 1;■
        ■
        int uSame = pool[u].ch[bit], vSame = pool[v].ch[bit];■
        int uDiff = pool[u].ch[bit^1], vDiff = pool[v].ch[bit^1];■
        if (countNode(uSame) - countNode(vSame) > 0) {■
            u = uSame; v = vSame;■
        } else {■
            u = uDiff; v = vDiff;■
            ans |= (1u << b);■
        }■
        if (!u && !v) break;■
    }■
    return ans;■
}■
■
long long count_leq(uint32_t x, int L, int R) const { // Type 3■
    int u = rootAt(R), v = rootAt(L-1);■
    long long res = 0;■
    for (int b = MAXLOG; b >= 0; --b) {■
        int bit = (x >> b) & 1;■
        if (bit == 1) {■
            int u0 = pool[u].ch[0], v0 = pool[v].ch[0];■
            res += (long long)countNode(u0) - countNode(v0);■
            u = pool[u].ch[1]; v = pool[v].ch[1];■
        } else {■
            u = pool[u].ch[0]; v = pool[v].ch[0];■
        }■
        if (!u && !v) break;■
    }■
    res += (long long)countNode(u) - countNode(v);■
    return res;■
}■
■
long long kth_smallest(int L, int R, long long k) const { // Type 4 (1-indexed)■
    int u = rootAt(R), v = rootAt(L-1);■
    long long total = (long long)countNode(u) - countNode(v);■
    if (k <= 0 || k > total) return -1;■
    ■
    uint32_t val = 0;■
    for (int b = MAXLOG; b >= 0; --b) {■
        int u0 = pool[u].ch[0], v0 = pool[v].ch[0];■
        long long left = (long long)countNode(u0) - countNode(v0);■
        if (k <= left) {■
            u = u0; v = v0;■
        } else {■
            k -= left;■
            u = pool[u].ch[1]; v = pool[v].ch[1];■
            val |= (1u << b);■
        }■
    }■
    return (long long)val;■
}■

```

```

    long long size() const { return countNode(roots.back()); }■
    int versions() const { return (int)roots.size(); }■
■
private:■
    int newNode(const Node* cp = nullptr) {■
        pool.push_back(cp ? *cp : Node());■
        return (int)pool.size() - 1;■
    }■
    int cloneNode(int id) { return newNode(id ? &pool[id] : nullptr); }■
■
    int countNode(int id) const { return id ? pool[id].cnt : 0; }■
■
    int rootAt(int idx) const {■
        if (idx < 0) return 0;■
        if (idx >= (int)roots.size()) return roots.back();■
        return roots[idx];■
    }■
■
    int insertPath(int prevRoot, uint32_t x) {■
        int nr = cloneNode(prevRoot);■
        pool[nr].cnt++;■
        int curNew = nr, curOld = prevRoot;■
        for (int b = MAXLOG; b >= 0; --b) {■
            int bit = (x >> b) & 1;■
            int oldChild = curOld ? pool[curOld].ch[bit] : 0;■
            int newChild = cloneNode(oldChild);■
            pool[curNew].ch[bit] = newChild;■
            curNew = newChild;■
            curOld = oldChild;■
            pool[curNew].cnt++;■
        }■
        return nr;■
    }■
};

```

---

## 62. Persistent\_Binary\_Trie\_ver

```

template<int MAXLOG = 31>■
struct PersistentBinaryTrie {■
    struct Node {■
        int ch[2];■
        int cnt;■
        Node() : ch{0,0}, cnt(0) {}■
    };■
■
    vector<Node> pool;■
    vector<int> roots;■
■
    struct Timeline {■
        vector<int> pref;■
    };■
    vector<Timeline> copies;■
■
    PersistentBinaryTrie() {■
        pool.reserve(1<<20);■
        pool.push_back(Node());■
        roots.push_back(newNode());■
        copies.push_back(Timeline());■
        copies[0].pref.push_back(roots[0]);■
    }
};

```

```

}■
■
void add(uint32_t x) { roots.push_back(insertPath(roots.back(), x)); }■
void pop_last(int k) {■
    int keep = max(0, (int)roots.size()-1 - k);■
    roots.resize(keep + 1);■
}■
uint32_t max_xor(uint32_t x, int L, int R) const {■
    return max_xor_on_copy(0, x, L, R);■
}■
uint32_t min_xor(uint32_t x, int L, int R) const {■
    return min_xor_on_copy(0, x, L, R);■
}■
long long count_leq(uint32_t x, int L, int R) const {■
    return count_leq_on_copy(0, x, L, R);■
}■
long long kth_smallest(int L, int R, long long k) const {■
    return kth_smallest_on_copy(0, L, R, k);■
}■
long long size() const { return countNode(roots.back()); }■
int versions() const { return (int)roots.size(); }■
■
int new_copy() {■
    copies.push_back(Timeline());■
    copies.back().pref.push_back(roots[0]); ■
    return (int)copies.size()-1;■
}■
int clone_copy(int cid) {■
    copies.push_back(copies[cid]);■
    return (int)copies.size()-1;■
}■
void add_to_copy(int cid, uint32_t x) {■
    int last = copies[cid].pref.back();■
    copies[cid].pref.push_back(insertPath(last, x));■
}■
void pop_last_in_copy(int cid, int k) {■
    int keep = max(0, (int)copies[cid].pref.size()-1 - k);■
    copies[cid].pref.resize(keep + 1);■
}■
int size_of_copy(int cid) const { return (int)copies[cid].pref.size()-1; }■
uint32_t max_xor_on_copy(int cid, uint32_t x, int L, int R) const {■
    int u = copyRootAt(cid, R), v = copyRootAt(cid, L-1);■
    uint32_t ans = 0;■
    for (int b = MAXLOG; b >= 0; --b) {■
        int bit = (x>>b)&1;■
        int uAlt = pool[u].ch[bit^1], vAlt = pool[v].ch[bit^1];■
        if (countNode(uAlt) - countNode(vAlt) > 0) {■
            ans |= (1u<<b);■
            u = uAlt; v = vAlt;■
        } else {■
            u = pool[u].ch[bit];■
            v = pool[v].ch[bit];■
        }■
    }■
    return ans; ■
}■
uint32_t min_xor_on_copy(int cid, uint32_t x, int L, int R) const {■
    int u = copyRootAt(cid, R), v = copyRootAt(cid, L-1);■
    uint32_t ans = 0;■
    for (int b = MAXLOG; b >= 0; --b) {■
        int bit = (x>>b)&1;■
        int uSame = pool[u].ch[bit], vSame = pool[v].ch[bit];■
        int uDiff = pool[u].ch[bit^1], vDiff = pool[v].ch[bit^1];■
        if (countNode(uSame) - countNode(vSame) > 0) {■
            u = uSame; v = vSame; ■
        }
    }
}

```

```

        } else {
            u = uDiff; v = vDiff;
            ans |= (1u<<b);
        }
        if (!u && !v) break;
    }
    return ans;
}

long long count_leq_on_copy(int cid, uint32_t x, int L, int R) const {
    int u = copyRootAt(cid, R), v = copyRootAt(cid, L-1);
    long long res = 0;
    for (int b = MAXLOG; b >= 0; --b) {
        int bit = (x>>b)&1;
        if (bit) {
            int u0 = pool[u].ch[0], v0 = pool[v].ch[0];
            res += (long long)countNode(u0) - countNode(v0);
            u = pool[u].ch[1]; v = pool[v].ch[1];
        } else {
            u = pool[u].ch[0]; v = pool[v].ch[0];
        }
        if (!u && !v) break;
    }
    res += (long long)countNode(u) - countNode(v);
    return res;
}

long long kth_smallest_on_copy(int cid, int L, int R, long long k) const {
    int u = copyRootAt(cid, R), v = copyRootAt(cid, L-1);
    long long total = (long long)countNode(u) - countNode(v);
    if (k <= 0 || k > total) return -1;
    uint32_t val = 0;
    for (int b = MAXLOG; b >= 0; --b) {
        int u0 = pool[u].ch[0], v0 = pool[v].ch[0];
        long long left = (long long)countNode(u0) - countNode(v0);
        if (k <= left) {
            u = u0; v = v0;
        } else {
            k -= left;
            u = pool[u].ch[1]; v = pool[v].ch[1];
            val |= (1u<<b);
        }
    }
    return (long long)val;
}

private:
    int newNode(const Node* cp=nullptr) {
        pool.push_back(cp ? *cp : Node());
        return (int)pool.size()-1;
    }
    int cloneNode(int id) { return newNode(id ? &pool[id] : nullptr); }
    int countNode(int id) const { return id ? pool[id].cnt : 0; }

    int rootAt(int idx) const {
        if (idx < 0) return 0;
        if (idx >= (int)roots.size()) return roots.back();
        return roots[idx];
    }
    int copyRootAt(int cid, int idx) const {
        if (idx < 0) return 0;
        const auto& pref = copies[cid].pref;
        if (idx >= (int)pref.size()) return pref.back();
        return pref[idx];
    }

    int insertPath(int prevRoot, uint32_t x) {

```

```

    int nr = cloneNode(prevRoot);
    pool[nr].cnt++;
    int curNew = nr, curOld = prevRoot;
    for (int b = MAXLOG; b >= 0; --b) {
        int bit = (x>>b)&1;
        int oldChild = curOld ? pool[curOld].ch[bit] : 0;
        int newChild = cloneNode(oldChild);
        pool[curNew].ch[bit] = newChild;
        curNew = newChild;
        curOld = oldChild;
        pool[curNew].cnt++;
    }
    return nr;
}
};

```

---

## 63. DSU\_with\_rollback

```

struct DSU {
    int n, comps;
    vector<int> parent, sz;
    vector<pair<int,int>> hist;
    DSU(int n_=0){ init(n_); }
    void init(int n_) {
        n = n_;
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        for (int i = 1; i <= n; ++i) parent[i] = i;
        hist.clear();
        comps = n;
    }
    int find(int x) const {
        while (x != parent[x]) x = parent[x];
        return x;
    }
    bool merge(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) {
            hist.emplace_back(-1, -1);
            return false;
        }
        if (sz[a] > sz[b]) swap(a, b);
        hist.emplace_back(a, b);
        parent[a] = b;
        sz[b] += sz[a];
        --comps;
        return true;
    }
    void _rollback_one_step() {
        if (hist.empty()) return;
        auto [a, b] = hist.back();
        hist.pop_back();
        if (a == -1) return;
        parent[a] = a;
        sz[b] -= sz[a];
        ++comps;
    }
    void rollback(int t) {
        while ((int)hist.size() > t) {

```

```

        _rollback_one_step();
    }
}
void krollback(int k) {
    for (int i = 0; i < k; ++i) {
        _rollback_one_step();
    }
}
int snapshot() const { return (int)hist.size(); }
bool same(int a, int b) const { return find(a) == find(b); }
int size_of(int a) const { return sz[find(a)]; }
int components() const { return comps; }
};

```

---

## 64. Kruskal\_Tree

```

struct Edge {int u, v; ll w;};
struct KruskalTree {
    int n, m;
    vector<Edge> edges;
    int tot;
    vector<vector<int>>> g;
    vector<ll> val;
    vector<int> isLeaf;
    vector<int> dsu;
    int findp(int x){ return dsu[x]==x ? x : dsu[x]=findp(dsu[x]); }
    int LOG;
    vector<int> depth, tin, tout, szLeaf;
    vector<vector<int>>> up;
    int timer = 0;
    KruskalTree(int n, int m): n(n), m(m), edges(m) {}
    void add_edge(int i, int u, int v, ll w){ edges[i] = {u,v,w}; }
    void build(bool addSuperRoot = true) {
        const ll INF = (ll)4e18;
        g.assign(n + m + 2, {});
        val.assign(n + m + 2, 0);
        isLeaf.assign(n + m + 2, 0);
        dsu.resize(n + m + 2);
        iota(dsu.begin(), dsu.end(), 0);

        tot = n;
        for (int i=1; i<=n; ++i) {
            val[i] = -(ll)4e18;
            isLeaf[i] = 1;
        }
        sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b){
            if (a.w != b.w) return a.w < b.w;
            return tie(a.u,a.v) < tie(b.u,b.v);
        });
        for (auto &e : edges) {
            int ru = findp(e.u), rv = findp(e.v);
            if (ru == rv) continue;
            int z = ++tot;
            val[z] = e.w;
            g[z].push_back(ru);
            g[z].push_back(rv);
            dsu[ru] = z;
            dsu[rv] = z;
        }
    }
};

```

```

    dsu[z] = z;
}
int root = 0;
if (addSuperRoot) {
    root = ++tot;
    val[root] = INF;
    vector<int> roots;
    for (int i=1; i<=tot-1; ++i) if (findp(i) == i) roots.push_back(i);
    for (int r : roots) g[root].push_back(r);
}

LOG = 1;
while ((1<<LOG) <= tot) ++LOG;
up.assign(LOG, vector<int>(tot+1, 0));
depth.assign(tot+1, 0);
tin.assign(tot+1, 0);
tout.assign(tot+1, 0);
szLeaf.assign(tot+1, 0);

if (addSuperRoot) dfs(root, 0);
else {
    for (int i=1; i<=tot; ++i) if (up[0][i]==0) dfs(i, 0);
}
for (int k=1; k<LOG; ++k)
    for (int v=1; v<=tot; ++v)
        up[k][v] = up[k-1][v] ? up[k-1][ up[k-1][v] ] : 0;
}

void dfs(int v, int p){
    up[0][v] = p;
    depth[v] = p ? depth[p] + 1 : 0;
    tin[v] = ++timer;
    if (g[v].empty()) {
        szLeaf[v] = 1;
    } else {
        szLeaf[v] = 0;
        for (int u : g[v]) {
            dfs(u, v);
            szLeaf[v] += szLeaf[u];
        }
    }
    tout[v] = timer;
}

int lca(int a, int b){
    if (a==0 || b==0) return 0;
    if (depth[a] < depth[b]) swap(a,b);
    int diff = depth[a] - depth[b];
    for (int k=LOG-1; k>=0; --k)
        if (diff & (1<<k)) a = up[k][a];
    if (a==b) return a;
    for (int k=LOG-1; k>=0; --k){
        if (up[k][a] != up[k][b]) {
            a = up[k][a];
            b = up[k][b];
        }
    }
    return up[0][a];
}

ll bottleneck(int u, int v){
    int w = lca(u, v);
    return (w ? val[w] : (ll)4e18);
}

int climbByWeight(int v, ll x){
    if (val[v] > x) return 0;

```

```

        for (int k=LOG-1; k>=0; --k){
            int to = up[k][v];
            if (to && val[to] <= x) v = to;
        }
        return v;
    }
    int reachableCountLeq(int u, ll x){
        int node = climbByWeight(u, x);
        return node ? szLeaf[node] : 0;
    }
};

```

---

## 65. Persistent\_DSU

```

struct PersistentDSU {
    int n;
    int cur_time;
    const int INF = numeric_limits<int>::max() / 2;

    vector<int> parent;
    vector<int> parent_time;
    vector<vector<pair<int,int>>> size_hist;

    explicit PersistentDSU(int n)
        : n(n), cur_time(0), parent(n), parent_time(n, INF), size_hist(n) {
        for (int i = 0; i < n; ++i) {
            parent[i] = i;
            size_hist[i].push_back({0, 1});
        }
    }

    int time() const { return cur_time; }
    int find(int x, int t) const {
        assert(0 <= t && t <= cur_time);
        while (parent_time[x] <= t) x = parent[x];
        return x;
    }

    int size(int x, int t) const {
        int r = find(x, t);
        const auto& h = size_hist[r];
        int lo = 0, hi = (int)h.size();
        while (lo < hi) {
            int mid = (lo + hi) >> 1;
            if (h[mid].first <= t) lo = mid + 1;
            else hi = mid;
        }
        return h[lo - 1].second;
    }

    bool connected(int a, int b, int t) const {
        return find(a, t) == find(b, t);
    }

    void unite(int a, int b) {
        int t = cur_time;
        int ra = find(a, t), rb = find(b, t);
        if (ra == rb) {
            ++cur_time;
            return;
        }
        int sa = size_hist[ra].back().second;

```

```

        int sb = size_hist[rb].back().second; ■
        if (sa < sb) swap(ra, rb), swap(sa, sb); ■
        parent[rb] = ra; ■
        parent_time[rb] = t + 1; ■
        size_hist[ra].push_back({t + 1, sa + sb}); ■
        ++cur_time; ■
    } ■
    ■
    int query_connect_time(int a, int b) const { ■
        if (!connected(a, b, cur_time)) { ■
            return -1; ■
        } ■
        int lo = 0; ■
        int hi = cur_time; ■
        int ans = cur_time; ■
    }
    ■
    while (lo <= hi) { ■
        int mid = (lo + hi) >> 1; ■
        if (connected(a, b, mid)) { ■
            ans = mid; ■
            hi = mid - 1; ■
        } else { ■
            lo = mid + 1; ■
        } ■
    } ■
    return ans; ■
} ■
int find_now(int x) const { return find(x, cur_time); } ■
int size_now(int x) const { return size(x, cur_time); } ■
bool connected_now(int a, int b) const { return connected(a, b, cur_time); } ■
};

```

---

## 66. Weighted\_DSU

```

struct DSU{ ■
    ll sz; ■
    vll parent, group; ■
    vll dis; ■
    ■
    DSU(ll n){ ■
        sz = n+1; ■
        parent.resize(sz+1); ■
        group.resize(sz+1, 1); ■
        dis.resize(sz+1); ■
        FOR1(i, 1, n) parent[i] = i; ■
    } ■
    pair<ll, ll> find(ll node){ ■
        if(parent[node]==node) return {node, 0}; ■
        auto t = find(parent[node]); ■
        parent[node] = t.first; ■
        dis[node] += t.second; ■
        return {parent[node], dis[node]}; ■
    } ■
    bool merge(ll u, ll v, ll w){ ■
        auto [le1, dx] = find(u); ■
        auto [le2, dy] = find(v); ■
        if(le1==le2) return (dy-dx==w); ■
        if(group[le1] > group[le2]) { ■
            parent[le2] = le1; ■

```

```

        group[le1] += group[le2];
        dis[le2] = w + dx - dy;
    } else {
        parent[le1] = le2;
        group[le2] += group[le1];
        dis[le1] = dy - dx - w;
    }
    return true;
}
bool samegroup(ll v, ll u){
    return find(v).first==find(u).first;
}
ll getsize(ll v){
    return group[find(v).first];
}
};

```

---

## 67. DSU\_range\_merge

```

struct DSU{
    ll sz;
    vi parent,group;

    DSU(ll n){
        sz = n+1;
        parent.resize(sz+1);
        group.resize(sz+1,1);
        FOR1(i,1,n) parent[i] = i;
    }
    ll find(ll node){
        if(parent[node]==node) return node;
        return find(parent[node]);
    }
    void merge(ll u, ll v){
        ll le1 = find(u);
        ll le2 = find(v);
        if(le1==le2) return;
        if(group[le1]>group[le2]) swap(le1, le2);
        parent[le1] = le2;
        group[le2] += group[le1];
    }
    bool samegroup(ll v, ll u){
        return find(v)==find(u);
    }
    ll getsize(ll v){
        return group[find(v)];
    }
};

ll topbit(ll x){
    return 63 - __builtin_clzll(x);
}

struct RangeParallelDSU{
    ll sz, lg;
    vector<DSU> dsu;

    RangeParallelDSU(ll n){
        sz = n;
    }
}

```

```

        lg = 0;
        while((1LL<<lg) <= sz) lg++;
        dsu.reserve(lg);
        for(ll i = 0; i < lg; i++) dsu.emplace_back(sz);
    }

    template<typename F>
    void merge(ll x, ll y, ll len, F f){
        if(len <= 0) return;
        if(len == 1){
            inner(x, y, 0, f);
            return;
        }
        ll k = topbit(len - 1);
        inner(x, y, k, f);
        inner(x + len - (1LL<<k), y + len - (1LL<<k), k, f);
    }

    ll find(ll v){
        return dsu[0].find(v);
    }

    bool samegroup(ll a, ll b){
        return dsu[0].samegroup(a, b);
    }

    ll getsize(ll v){
        return dsu[0].getsize(v);
    }

private:
    template<typename F>
    void inner(ll x, ll y, ll k, F f){
        if(k == 0){
            ll le1 = dsu[0].find(x);
            ll le2 = dsu[0].find(y);
            if(le1 == le2) return;
            if(dsu[0].group[le1] > dsu[0].group[le2]) swap(le1, le2);
            dsu[0].parent[le1] = le2;
            dsu[0].group[le2] += dsu[0].group[le1];
            f(le2, le1);
            return;
        }
        if(dsu[k].samegroup(x, y)) return;
        dsu[k].merge(x, y);
        inner(x, y, k - 1, f);
        ll add = 1LL << (k - 1);
        inner(x + add, y + add, k - 1, f);
    }
};

/* in main
auto on_merge_callback = [&](ll new_root, ll old_root) {
    num_components--;
};
*/

```

---

## 68. mo\_with\_rollback

```

const int N = 2e5 + 5; ■
int blksz; // in main ■
■
struct Query { ■
    int l, r, id, blk; ■
    bool operator<(const Query& other) const { ■
        if (blk != other.blk) return blk < other.blk; ■
        return r < other.r; ■
    } ■
}; ■
■
// ----- ■
// ===== ■
// ----- ■
■
■
ll a [N]; ■
struct RollbackState { ■
    int parent[N], sz[N]; ■
    int components; ■
    ll first[N], last[N], ans; ■
    struct hss { ■
        ll val, f, l, mx; ■
        hss() { ■
            val = f = l = mx = -1; ■
        } ■
        hss(ll a, ll b, ll c, ll d) { ■
            val = a; ■
            f = b; ■
            l = c; ■
            mx = d; ■
        } ■
    }; ■
    vector<hss> history; ■
    void init(int n) { ■
        components = n; ■
        history.clear(); ■
        ans = 0; ■
        for(int i = 1; i <= n; i++) { ■
            parent[i] = i; ■
            sz[i] = 1; ■
            first[i] = 1e18; ■
            last[i] = -1e18; ■
        } ■
    } ■
    int find(int i) { ■
        while (i != parent[i]) i = parent[i]; ■
        return i; ■
    } ■
    void add(ll idx) { ■
        ll val = a[idx]; ■
        history.pb(hss(val, first[val], last[val], ans)); ■
        first[val] = min(first[val], idx); ■
        last[val] = max(last[val], idx); ■
        ans = max(ans, last[val]-first[val]); ■
    } ■
    int snapshot() { ■
        return history.size(); ■
    } ■
    void rollback(int snap) { ■
        while (history.size() > snap) { ■

```

```

        auto[val,f,l,mx] = history.back();
        history.pop_back();
        if (val != -1) {
            first[val] = f;
            last[val] = l;
            ans = mx;
        }
    }
}
int get_answer() {
    return ans;
}

}
state;
// -----
//
// -----
int ans[N];
vector<Query> queries;
void solve_mo(int n, int q_count) {
    if (queries.empty()) return;
    blkksz = max(1, (int)(n / sqrt(q_count)));
    for(auto &q : queries) q.blk = q.l / blkksz;
    sort(queries.begin(), queries.end());
    int last = -1;
    int r_ptr = 0;
    int end = 0;
    for (const auto& q : queries) {
        if (q.blk != last) {
            last = q.blk;
            state.init(n);
            end = (last + 1) * blkksz - 1;
            r_ptr = end;
        }
        if (q.r <= end) {
            int snap = state.snapshot();
            for (int i = q.l; i <= q.r; i++) state.add(i);
            ans[q.id] = state.get_answer();
            state.rollback(snap);
        } else {
            while (r_ptr < q.r) {
                r_ptr++;
                state.add(r_ptr);
            }
            int snap = state.snapshot();
            for (int i = end + 1; i >= q.l; i--) {
                state.add(i);
            }
            ans[q.id] = state.get_answer();
            state.rollback(snap);
        }
    }
}
}

```

---

## 69. mo\_with\_update

```
const int N = 2e5 + 5;
```

```

const int MAX_VAL = 1000005; ■
■
ll sq;■
ll v[N], ans[N], vis[MAX_VAL];■
ll n, q, ret = 0;■
■
struct query {■
    ll l, r, t, idx, blkL, blkR;■
    query(ll _l, ll _r, ll _t, ll _idx) {■
        l = _l; r = _r; t = _t; idx = _idx;■
        blkL = l / sq;■
        blkR = r / sq;■
    }■
    bool operator < (const query &q) const {■
        if (blkL != q.blkL) return blkL < q.blkL;■
        if (blkR != q.blkR) return blkR < q.blkR;■
        return t < q.t;■
    }■
};■
■
struct update {■
    ll idx, newVal;■
};■
■
vector<query> qu;■
vector<update> ups;■
■
void add(ll idx) {■
    if (!vis[v[idx]]) ret+=v[idx];■
    vis[v[idx]]++;■
}■
■
void remove(ll idx) {■
    vis[v[idx]]--;■
    if (!vis[v[idx]]) ret-=v[idx];■
}■
■
void change(ll u_idx, ll &l, ll &r) {■
    ll idx = ups[u_idx].idx;■
    bool inRange = (idx >= l && idx <= r);■
    if (inRange) remove(idx);■
    swap(v[idx], ups[u_idx].newVal);■
    if (inRange) add(idx);■
}■
■
void mo_build() {■
    sort(qu.begin(), qu.end());■
    ll l = 1, r = 0, t = 0;■
    ■
    for (const auto &q : qu) {■
        while (t < q.t) change(t++, l, r);■
        while (t > q.t) change(--t, l, r);■
        while (l > q.l) add(--l);■
        while (r < q.r) add(++r);■
        while (l < q.l) remove(l++);■
        while (r > q.r) remove(r--);■
        ans[q.idx] = ret;■
    }■
}■
■
■
// void Hello_World()■
// {■
//     cin>>n;■
//     sq = pow(n, 2.0/3.0); ■

```

```

//      if(sq == 0) sq = 1;
//      for(int i=1; i<=n; i++) cin >> v[i];
//      int t_idx = 0, q_idx = 0;
//      cin>>q;
//      for(int i=0; i<q; i++) {
//          char type; cin >> type;
//          if(type == 'Q') {
//              int l, r; cin >> l >> r;
//              qu.push_back(query(l, r, t_idx, q_idx++));
//          } else {
//              int idx, val; cin >> idx >> val;
//              ups.push_back({idx, val});
//              t_idx++;
//          }
//      }
//      mo_build();
//      for(int i=0; i<q_idx; i++) cout << ans[i] << "\n";
//  }

```

---

## 70. Suffix\_Array

```

struct suffarray {
    string s;
    int n;
    vector<int> p, c, lcp;
    vector<vector<int>> st; // Sparse Table
    vector<int> lg; // Log values
    suffarray(string _s) : s(_s + "$"), n(s.length()) {
        p.resize(n); c.resize(n); lcp.resize(n);
        build_sa();
        build_lcp();
        build_st();
    }
    // Builds Suffix Array in O(N log N)
    void build_sa() {
        vector<pair<char, int>> a(n);
        for (int i = 0; i < n; i++) a[i] = {s[i], i};
        sort(a.begin(), a.end());
        for (int i = 0; i < n; i++) p[i] = a[i].second;
        c[p[0]] = 0;
        for (int i = 1; i < n; i++)
            c[p[i]] = c[p[i - 1]] + (a[i].first != a[i - 1].first);
        int k = 0;
        while ((1 << k) < n) {
            for (int i = 0; i < n; i++) p[i] = (p[i] - (1 << k) + n) % n;
            count_sort();
            vector<int> new_c(n);
            new_c[p[0]] = 0;
            for (int i = 1; i < n; i++) {
                pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << k)) % n]};
                pair<int, int> now = {c[p[i]], c[(p[i] + (1 << k)) % n]};
                new_c[p[i]] = new_c[p[i - 1]] + (prev != now);
            }
            c = new_c;
            k++;
        }
    }
    // Helper for build_sa
    void count_sort() {

```

```

vector<int> cnt(n, 0);
for (int x : c) cnt[x]++;
vector<int> pos(n);
pos[0] = 0;
for (int i = 1; i < n; i++) pos[i] = pos[i - 1] + cnt[i - 1];
vector<int> new_p(n);
for (int x : p) {
    int i = c[x];
    new_p[pos[i]] = x;
    pos[i]++;
}
p = new_p;
}
// Builds LCP Array in O(N) using Kasai's Algo
void build_lcp() {
    int k = 0;
    for (int i = 0; i < n - 1; i++) {
        int pi = c[i];
        int j = p[pi - 1];
        while (s[i + k] == s[j + k]) k++;
        lcp[pi] = k;
        if (k > 0) k--;
    }
}
// Builds Sparse Table for RMQ in O(N log N)
void build_st() {
    lg.resize(n + 1);
    lg[1] = 0;
    for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
    int K = lg[n];
    st.assign(n, vector<int>(K + 1));
    for (int i = 0; i < n; i++) st[i][0] = lcp[i];
    for (int j = 1; j <= K; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
}
// Returns min value in range [L, R]
int query(int L, int R) {
    int j = lg[R - L + 1];
    return min(st[L][j], st[R - (1 << j) + 1][j]);
}

// Returns LCP of suffixes starting at i and j in O(1)
int get_lcp(int i, int j) {
    if (i == j) return n - i - 1;
    int L = c[i], R = c[j];
    if (L > R) swap(L, R);
    return query(L + 1, R);
}
// Compares substrings s[l1..r1] and s[l2..r2] in O(1)
// Returns: -1 if s1 < s2, 1 if s1 > s2, 0 if equal
int compare_substr(int l1, int r1, int l2, int r2) {
    int len1 = r1 - l1 + 1, len2 = r2 - l2 + 1;
    int common = get_lcp(l1, l2);
    common = min(common, min(len1, len2));
    if (common == len1 && common == len2) return 0;
    if (common == len1) return -1;
    if (common == len2) return 1;
    return (s[l1 + common] < s[l2 + common]) ? -1 : 1;
}

int countOccurrences(string &pat) {
    int n = p.size();
    int m = pat.size();
    int l = 0, r = n - 1;

```

```

int first = -1;
while (l <= r) {
    int mid = l + (r - l) / 2;
    int res = s.compare(p[mid], m, pat);
    if (res >= 0) {
        first = mid;
        r = mid - 1;
    } else {
        l = mid + 1;
    }
}
if (first == -1 || s.compare(p[first], m, pat) != 0) {
    return 0;
}
l = 0; r = n - 1;
int last = n;
while (l <= r) {
    int mid = l + (r - l) / 2;
    int res = s.compare(p[mid], m, pat);
    if (res > 0) {
        last = mid;
        r = mid - 1;
    } else {
        l = mid + 1;
    }
}
return last - first;
}
};

```

---

## 71. Suffix\_Automaton

```

struct SuffixAutomaton {
    struct node {
        int len, link, firstpos;
        map<char, int> nxt;
    };
    int sz, last;
    vector<node> t;
    vector<int> terminal;
    vector<long long> dp;
    vector<vector<int>> g;
    SuffixAutomaton() {}
    SuffixAutomaton(int n) {
        t.resize(2 * n); terminal.resize(2 * n, 0);
        dp.resize(2 * n, -1); sz = 1; last = 0;
        g.resize(2 * n);
        t[0].len = 0; t[0].link = -1; t[0].firstpos = 0;
    }
    void extend(char c) {
        int p = last;
        if (t[p].nxt.count(c)) {
            int q = t[p].nxt[c];
            if (t[q].len == t[p].len + 1) {
                last = q;
                return;
            }
            int clone = sz++;
            t[clone] = t[q];

```

```

        t[clone].len = t[p].len + 1;
        t[q].link = clone;
        last = clone;
        while (p != -1 && t[p].nxt[c] == q) {
            t[p].nxt[c] = clone;
            p = t[p].link;
        }
        return;
    }
    int cur = sz++;
    t[cur].len = t[last].len + 1;
    t[cur].firstpos = t[cur].len;
    p = last;
    while (p != -1 && !t[p].nxt.count(c)) {
        t[p].nxt[c] = cur;
        p = t[p].link;
    }
    if (p == -1) t[cur].link = 0;
    else {
        int q = t[p].nxt[c];
        if (t[p].len + 1 == t[q].len) t[cur].link = q;
        else {
            int clone = sz++;
            t[clone] = t[q];
            t[clone].len = t[p].len + 1;
            while (p != -1 && t[p].nxt[c] == q) {
                t[p].nxt[c] = clone;
                p = t[p].link;
            }
            t[q].link = t[cur].link = clone;
        }
    }
    last = cur;
}

void build_tree() {
    for (int i = 1; i < sz; i++) g[t[i].link].push_back(i);
}

void build(string &s) {
    for (auto x: s) {
        extend(x);
        terminal[last] = 1;
    }
    build_tree();
}

long long cnt(int i) { //number of times i-th node occurs in the string
    if (dp[i] != -1) return dp[i];
    long long ret = terminal[i];
    for (auto &x: g[i]) ret += cnt(x);
    return dp[i] = ret;
}

};

```

---

# Contest Snippets

## Index

1. dijkstra
2. include
3. sieve\_algo
4. fast\_power\_mod
5. fast\_power
6. mod\_op
7. nck\_with\_mod
8. nck\_without\_mod
9. prefix\_sum\_2d
10. findSubarrayCount
11. DSU\_
12. mint\_mod
13. ordered\_set\_call
14. prime\_div\_in)map
15. prefix\_and\_or\_xor
16. Hashing
17. Trie
18. Binary\_Trie
19. Z\_Function
20. number\_of\_prime\_factors\_of\_a\_number
21. xor\_from\_1\_to\_n
22. sparse\_table
23. mo\_in\_array
24. segment\_tree
25. sqrt\_deco
26. lazy\_segment\_(q,p)
27. lazy\_segment\_(q,q)
28. XOR\_Basis
29. segment\_tree\_2d

30. Li\_Chao\_Tree
31. LCA
32. NTT
33. Matrix\_def
34. Fenwick\_tree
35. Lucas\_Theorem
36. Aho\_Corasick
37. KMP and go state
38. Merge\_Sort\_Tree
39. mobius
40. Median\_query
41. Median\_Sliding
42. 2-sat sheet
43. Tarjan for SCC
44. Tarjan for Bridges
45. Tarjan for Articulation Points
46. Centroid\_Decomposition\_eg
47. Heavy\_Light\_Decomposition\_eg
48. stirling\_2d\_type
49. persistent\_segment\_tree
50. Persistent\_Segment\_Tree\_with\_Lazy\_Propagation
51. Dynamic\_Segment\_Tree
52. small\_segment\_tree
53. Segment\_Tree\_Merging
54. xor\_segment\_tree
55. xor\_segment\_tree\_Non-commutative\_case
56. segment\_tree\_beats\_min\_add
57. segment\_tree\_beats\_max\_add
58. segment\_tree\_beats\_mod,div,sqrt
59. Fenwick\_Range\_Update
60. Persistent\_Trie
61. Persistent\_Binary\_Trie
62. Persistent\_Binary\_Trie\_ver

- 63. DSU\_with\_rollback
- 64. Kruskal\_Tree
- 65. Persistent\_DSU
- 66. Weighted\_DSU
- 67. DSU\_range\_merge
- 68. mo\_with\_rollback
- 69. mo\_with\_update
- 70. Suffix\_Array
- 71. Suffix\_Automaton